

Universidad Autónoma Gabriel René Moreno

FACULTAD DE INGENIERÍA EN CIENCIAS DE LA
COMPUTACIÓN Y TELECOMUNICACIONES



SISTEMA DE INFORMACIÓN PARA LA GESTIÓN Y ADMISIÓN DEL CURSO PREUNIVERSITARIO (CUP) DE LA FICCT

INTEGRANTES:

Soto Montaña Gabriel 222058102

Docente: Ing. Angélica Garzón Cuéllar

Materia: Sistemas De Información 1

Gestión: 1-2026

Grupo: 38

Índice de contenido

1	PERFIL	4
1.1	INTRODUCCIÓN	4
1.2	OBJETIVOS	5
1.2.1	Objetivo General	5
1.2.2	Objetivos específicos	5
1.3	DESCRIPCIÓN DEL PROBLEMA	6
1.4	ALCANCE	6
1.4.1	Módulo de Autenticación	6
1.4.2	Módulo de Registro de Postulantes	7
1.4.3	Módulo de Exámenes	7
1.4.4	Módulo de Asignación de Grupos	8
1.4.5	Módulo de Reportes	8
1.4.6	Panel Administrativo (Dashboard)	8
1.4.7	Módulo de Gestión de Docentes	9
2	MARCO TEÓRICO	10
2.1	Marco Conceptual (Dominio de Admisión Universitaria)	10
2.2	Marco Metodológico (Ingeniería de Software)	10
2.3	Marco Tecnológico	11
3	MODELO DE NEGOCIO	12
3.1	DIAGRAMA DE ACTIVIDADES	12

3.1.1	Diagrama de Actividad: Proceso de Inscripción y Registro.....	12
3.1.2	Proceso de Registro de Notas y Aprobación.	14
3.1.3	Proceso de Admisión Final (Distribución por Cupos).....	15
3.1.4	Contratación y Asignación de Docentes	16
4	FLUJO DE TRABAJO: CAPTURA DE REQUISITOS.....	17
4.1	Identificar actores del sistema	17
4.1.1	Actores internos del sistema.....	18
4.1.2	Actores externos del sistema.....	19
4.2	Lista de casos de uso	20
4.3	Priorizar casos de uso	21
4.3.1	Ciclo #1 Iteración: Arquitectura Base, Persistencia y Mitigación de Riesgos Algorítmicos	25
4.4	Detallar caso de uso	26
4.4.1	Ciclo #1	26
4.5	Estructurar caso de uso.....	48
4.5.1	Ciclo #1	48
5	FLUJO DE TRABAJO: ANÁLISIS	49
5.1	Análisis de arquitectura.....	49
5.1.1	Identificar paquetes.....	49
5.1.2	Relacionar paquetes y casos de uso	51
5.1.3	Vista de casos de uso	53
5.2	Diagrama de comunicación.....	56
5.2.1	Ciclo #1	56

5.3	Análisis de clases	64
5.3.1	Ciclo #1	64
5.4	Analizar paquetes.....	71
5.4.1	Ciclo #1	71
6	FLUJO DE TRABAJO: DISEÑO	72
6.1	Diseño de arquitectura.....	72
6.1.1	Diseño físico (diagrama de despliegue)	72
6.1.2	Diseño lógico (diagrama organizado en capas).....	73
6.2	Diseño de datos.....	75
6.2.1	Diseño de datos lógico	75
6.2.2	Diseño de datos físico	76
6.2.3	Población de datos.....	94
6.2.4	Consultas	108
6.3	Diseñar un caso de uso (diagrama de secuencia).....	141
6.3.1	Ciclo #1	141
7	FLUJO DE TRABAJO: IMPLEMENTACION.....	142

1 PERFIL

1.1 INTRODUCCIÓN

En la actualidad, la gestión eficiente de los procesos académicos es un pilar fundamental para el desarrollo de las instituciones de educación superior. La Facultad de Ingeniería en Ciencias de la Computación y Telecomunicaciones (FICCT) de la Universidad Autónoma Gabriel René Moreno (UAGRM) recibe semestralmente a una gran cantidad de bachilleres que aspiran a ingresar a una de sus cuatro carreras a través del Curso Preuniversitario (CUP). El manejo manual o semiautomatizado de las inscripciones, la asignación de aulas para miles de postulantes, la distribución de cargas horarias de los docentes y, sobre todo, el cálculo de los cupos de

admisión, representa un desafío logístico significativo. Por lo tanto, el presente proyecto se enfoca en el desarrollo de una Aplicación Web en PHP orientada a automatizar de manera integral el proceso de registro, evaluación y admisión del CUP.

1.2 OBJETIVOS

1.2.1 Objetivo General

Desarrollar una aplicación web en lenguaje PHP con base de datos PostgreSQL para administrar y automatizar el proceso de inscripción, evaluación, asignación de grupos y admisión de los estudiantes postulantes al Curso Preuniversitario (CUP) de la FICCT.

1.2.2 Objetivos específicos

1. Implementar un módulo de autenticación y seguridad para la administración de usuarios (autoridades, docentes, operadores y postulantes).
2. Desarrollar un módulo de registro de postulantes que capture sus requisitos (Cédula de identidad, título de bachiller y opciones de carrera).
3. Programar un algoritmo de cálculo que asigne a los estudiantes inscritos en grupos con una capacidad máxima de 70 alumnos por aula.
4. Implementar un sistema de asignación docente que valide perfiles académicos (Maestría y Diplomado en Educación Superior) y garantice una distribución máxima de 4 grupos sin cruce de horarios.
5. Desarrollar el módulo de exámenes para calcular promedios finales basados en 3 evaluaciones en las 4 materias (Matemáticas, Física, Computación e Inglés).
6. Construir el algoritmo central de admisión que asigne a los estudiantes aprobados a su primera o segunda opción de carrera dependiendo del límite de cupos y su orden de mérito (promedio general).
7. Generar reportes estadísticos e indicadores de gestión sobre índices de aprobación e historial de gestiones.

1.3 DESCRIPCIÓN DEL PROBLEMA

El actual proceso de inscripción y admisión del CUP enfrenta cuellos de botella severos debido a la alta demanda (superando en ocasiones los 1000 postulantes). La recolección física de documentos genera filas extensas y propensión a extravíos. Asimismo, el cálculo matemático para la habilitación de la cantidad de grupos necesarios y la asignación manual de docentes sin generar solapamientos de horarios es un proceso lento y susceptible al error humano.

El problema más crítico reside en la etapa final de evaluación: tabular manualmente tres exámenes por estudiante en cuatro materias, validar la condición estricta de aprobación (nota mayor o igual a 60 en todas), y distribuir a los cientos de aprobados ordenándolos por promedio para llenar los cupos exactos de su primera o segunda opción de carrera. Este procedimiento manual no solo retrasa la publicación de la lista oficial de admitidos, sino que dificulta la generación de métricas y estadísticas de rendimiento comparativo entre gestiones para la decanatura.

1.4 ALCANCE

El Sistema Web de Administración del Curso Preuniversitario (CUP) abarcará en su totalidad la logística operativa, el registro documental y la evaluación académica, estructurándose en 6 Módulos Funcionales clave:

1.4.1 Módulo de Autenticación

Permite el acceso seguro al sistema mediante credenciales validadas, garantizando que el Usuario Administrador ingrese de forma protegida a la plataforma. Gestiona las sesiones activas para proteger la integridad de los datos de admisión y evitar accesos no autorizados. Datos principales y funcionalidades:

- **Datos:** Usuario, Contraseña (encriptada), Estado de sesión.

- **Funcionalidades:** Inicio de sesión seguro, Cerrar sesión, Recuperación básica de contraseña (opcional).
- **Validaciones:** Usuario y contraseña obligatorios, control estricto de sesiones.

1.4.2 Módulo de Registro de Postulantes

Es el componente central para la recolección de datos de los estudiantes que aspiran ingresar a la FICCT. Permite inscribir y administrar toda la información personal y académica del postulante, controlando validaciones clave como la unicidad del carnet de identidad. Datos principales y funcionalidades:

- **Datos requeridos:** CI, Nombres y Apellidos, Fecha de nacimiento, Sexo, Dirección, Teléfono, Correo electrónico, Colegio de procedencia, Ciudad, Carrera a la que postula, Título de bachiller.
- **Funcionalidades:** Registrar postulante, Modificar datos, Eliminar registro, Buscar postulante, Listar postulantes.
- **Validaciones:** No permitir CI duplicado, validar formato de correo electrónico, validar campos vacíos.

1.4.3 Módulo de Exámenes

Gestiona el registro y procesamiento de las calificaciones obtenidas por los postulantes. Incorpora la lógica de negocio algorítmica para promediar automáticamente las notas y determinar si el postulante está Aprobado o Reprobado según el umbral de la facultad, considerando la primera y segunda opción de carrera. Datos principales y funcionalidades:

- **Datos (Notas):** Computación, Matemáticas, Inglés y Física (Rango de 0 a 100).
- **Reglas de Negocio:** Solo 3 exámenes por estudiante. El estado se calcula automáticamente (APROBADO si el promedio es ≥ 60 , REPROBADO si es < 60). Se evalúa el cupo de la 1ra opción, y si está lleno, pasa a la 2da opción de carrera.
- **Funcionalidades:** Registrar notas, Editar notas, Mostrar promedio calculado.

1.4.4 Módulo de Asignación de Grupos

Automatiza la organización académica del curso de nivelación. Su objetivo es distribuir de manera eficiente a los postulantes inscritos ejecutando un algoritmo que calcula dinámicamente la cantidad de grupos necesarios para cumplir con el límite máximo de estudiantes por aula establecido por la facultad. Datos principales y funcionalidades:

- **Datos y Cálculos:** Cantidad total de inscritos, Cantidad de grupos habilitados (Aplicando la fórmula algorítmica de techo: $\text{CEIL}(\text{TotalInscritos} / \text{LímitePorGrupo})$).
- **Regla:** Cada grupo admite un límite estricto de estudiantes (ej. máximo 70 u 80 según parametrización).
- **Funcionalidades:** Mostrar total de inscritos, Mostrar cantidad de grupos, Mostrar estudiantes asignados por grupo.

1.4.5 Módulo de Reportes

Proporciona información consolidada y estadística para apoyar la toma de decisiones de la administración académica. Genera listas detalladas y resúmenes de rendimiento que pueden ser visualizados en el sistema. Datos principales e indicadores:

- **Reportes obligatorios:** Lista general de postulantes, Postulantes aprobados, Postulantes reprobados, Promedios generales, Cantidad de grupos habilitados, Estadísticas por materia, Docentes por grupos, Grupos con mayor cantidad de aprobados.
- **Funcionalidades (Opcionales):** Exportar a PDF, Exportar a Excel, e integración con IA para generación de reportes por comandos de voz.

1.4.6 Panel Administrativo (Dashboard)

Actúa como el centro de control principal para el administrador. Centraliza el acceso a todas las funciones del sistema mediante un menú intuitivo y presenta un resumen ejecutivo en tiempo real de los indicadores clave del proceso de admisión. Datos principales:

- **Interfaz:** Menú de navegación y Dashboard principal.

- **Indicadores Estadísticos (KPIs):** Total de inscritos, Total de aprobados, Total de reprobados, Total de grupos habilitados.

1.4.7 Módulo de Gestión de Docentes

Administra el ciclo completo de contratación y asignación académica de los docentes del CUP. El sistema valida que cada candidato cumpla con los requisitos mínimos establecidos por la FICCT antes de habilitarlo para recibir una carga horaria. Garantiza además que ningún docente sea asignado a más grupos de los permitidos ni que existan cruces de horario en su distribución.

- **Datos requeridos del docente:** CI, Nombres y Apellidos, Especialidad profesional, Título de Maestría, Diplomado en Educación Superior, Correo electrónico, Teléfono.
- **Funcionalidades:** Registrar docente, Modificar datos, Eliminar registro, Buscar docente, Listar docentes, Asignar grupos y horarios, Visualizar carga horaria asignada.
- **Reglas de negocio:** El docente debe poseer título de Maestría y Diplomado en Educación Superior para ser contratado. Cada docente puede ser asignado desde 1 hasta un máximo de 4 grupos. El sistema no permite asignación si existe cruce de horario.
- **Validaciones:** No permitir CI duplicado. Verificar existencia de título de Maestría (campo obligatorio). Verificar existencia de Diplomado en Educación Superior (campo obligatorio). Bloquear asignación si el docente ya tiene 4 grupos activos. Detectar y alertar cruce de horarios antes de confirmar asignación.
- **Carga masiva de cuentas de usuario (Excel/CSV):** El sistema permitirá importar lotes de cuentas de docentes, autoridades y coordinadores desde archivos Excel o CSV entregados por la administración académica al inicio de cada gestión. El sistema genera automáticamente las credenciales de acceso y asigna el perfil de rol correspondiente (RBAC) a cada cuenta importada.

2 MARCO TEÓRICO

2.1 Marco Conceptual (Dominio de Admisión Universitaria)

En esta sección se definen los pilares operativos y de control que rigen la actividad académica y administrativa del proceso de admisión, traduciendo las necesidades de la facultad en conceptos formales de sistemas.

- **Sistema de Procesamiento de Transacciones (TPS):** Un TPS es un tipo de sistema de información cuya función principal es recolectar, almacenar, modificar y recuperar la información generada por las transacciones de una organización. Para el presente proyecto del CUP, este concepto es fundamental, ya que el sistema debe procesar diariamente la inscripción de miles de postulantes, la asignación masiva de aulas y el registro continuo de calificaciones, garantizando que ninguna transacción concurrente altere el resultado de los exámenes.
- **Control de Accesos Basado en Roles (RBAC):** Es un método de seguridad que restringe el acceso al sistema basándose en el rol o cargo que ocupa el usuario dentro de la organización. En el Sistema del CUP, el RBAC es el pilar de la privacidad y seguridad: garantiza que un *Postulante* solo tenga permisos de lectura sobre sus propias notas, mientras que el *Docente* posee permisos de escritura exclusiva sobre su carga horaria, aislando el control absoluto únicamente para el rol de *Administrador*.
- **Inteligencia de Negocios (Business Intelligence - BI):** Comprende las estrategias y tecnologías utilizadas por las empresas para el análisis de datos. En el contexto de la FICCT, la aplicación de BI justifica la creación del Panel Administrativo (Dashboard), el cual transforma los datos crudos de inscripciones y notas en indicadores gráficos de rendimiento y reportes estadísticos, agilizando la toma de decisiones del coordinador.

2.2 Marco Metodológico (Ingeniería de Software)

Para garantizar la calidad y el cumplimiento de los estrictos requisitos del proceso de admisión, el proyecto se rige bajo estándares formales de la ingeniería de software.

- **Proceso Unificado de Desarrollo de Software (PUDS):** El PUDS es un marco de desarrollo de software formal. Para la construcción de la aplicación web del CUP, se ha adoptado esta metodología debido a sus cuatro características esenciales:
 1. *Iterativo e Incremental:* El sistema no se programa de una sola vez, sino en "ciclos" de desarrollo (entregas parciales) que van sumando funcionalidades hasta tener el producto completo.
 2. *Centrado en la Arquitectura:* Prioriza la construcción de una base de datos y un esqueleto sólido antes de diseñar las pantallas.
 3. *Dirigido por Casos de Uso:* Todo el código que se escribe nace exclusivamente para resolver una necesidad de los actores (Postulantes, Docentes, Administradores).
- **Lenguaje Unificado de Modelado (UML 2.5):** Es el lenguaje estándar a nivel mundial para visualizar, especificar y documentar los artefactos de un sistema de software. En este proyecto, UML se utiliza para abstraer la complejidad del proceso de admisión en planos lógicos (Diagramas de Casos de Uso, Secuencia y Clases) antes de escribir el código fuente.

2.3 Marco Tecnológico

El éxito de la sistematización del CUP no reside únicamente en la programación, sino en la selección de un ecosistema tecnológico moderno que garantice alta disponibilidad y seguridad de los datos.

- **Desarrollo Backend con PHP:** PHP es un lenguaje de programación de uso general de código del lado del servidor. En el Sistema del CUP, PHP será el motor lógico responsable de recibir las peticiones, procesar el algoritmo matemático de asignación de grupos (CEIL(Inscritos/Límite)) y calcular los promedios académicos para determinar el estado de aprobación de los postulantes de forma segura.
- **Bases de Datos Relacionales (PostgreSQL) y Propiedades ACID:** PostgreSQL es un sistema de gestión de bases de datos relacional y robusto. Su selección se justifica por

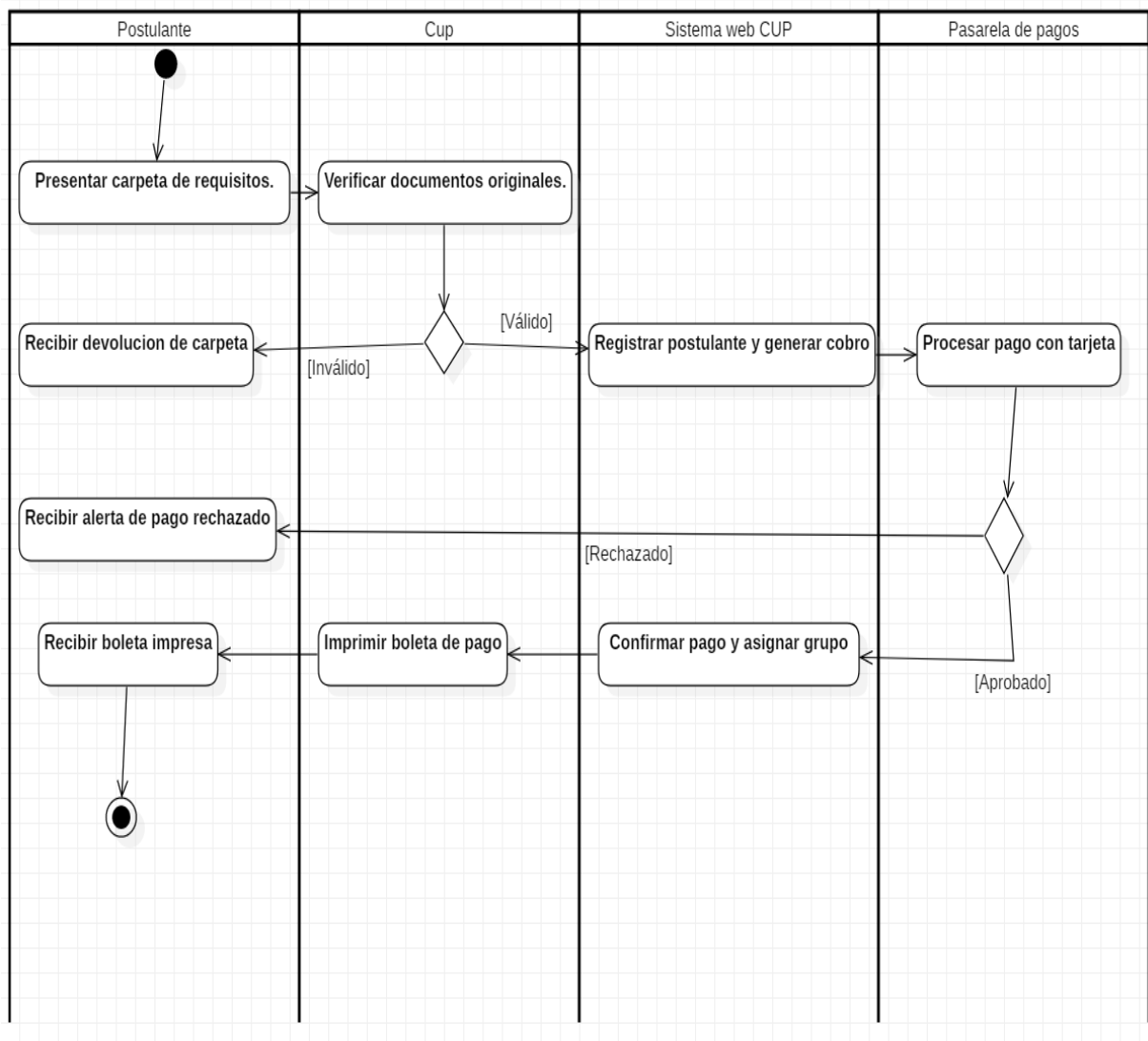
el cumplimiento estricto de las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad). Este concepto teórico es innegociable para el CUP, ya que asegura que las notas de los exámenes y las asignaciones de cupos jamás se pierdan ni se corrompan ante fallos de red o cortes eléctricos.

- **Computación en la Nube (Cloud Computing):** Se refiere a la entrega de servicios informáticos a través de Internet (la nube). Desplegar el sistema del CUP en la nube (AWS, Azure u otros) elimina la dependencia de servidores físicos locales (localhost), garantizando que los miles de postulantes puedan acceder a la plataforma desde sus domicilios las 24 horas del día.
- **Desarrollo Frontend Responsive (HTML5, CSS3, JavaScript):** Conjunto de tecnologías estándar para la creación de interfaces web. Su implementación bajo el paradigma "Responsive Web Design" asegura que el sistema adapte visualmente sus formularios y reportes al tamaño de pantalla del usuario, permitiendo que el postulante se inscriba desde su teléfono móvil con la misma fluidez que el administrador en una computadora de escritorio.

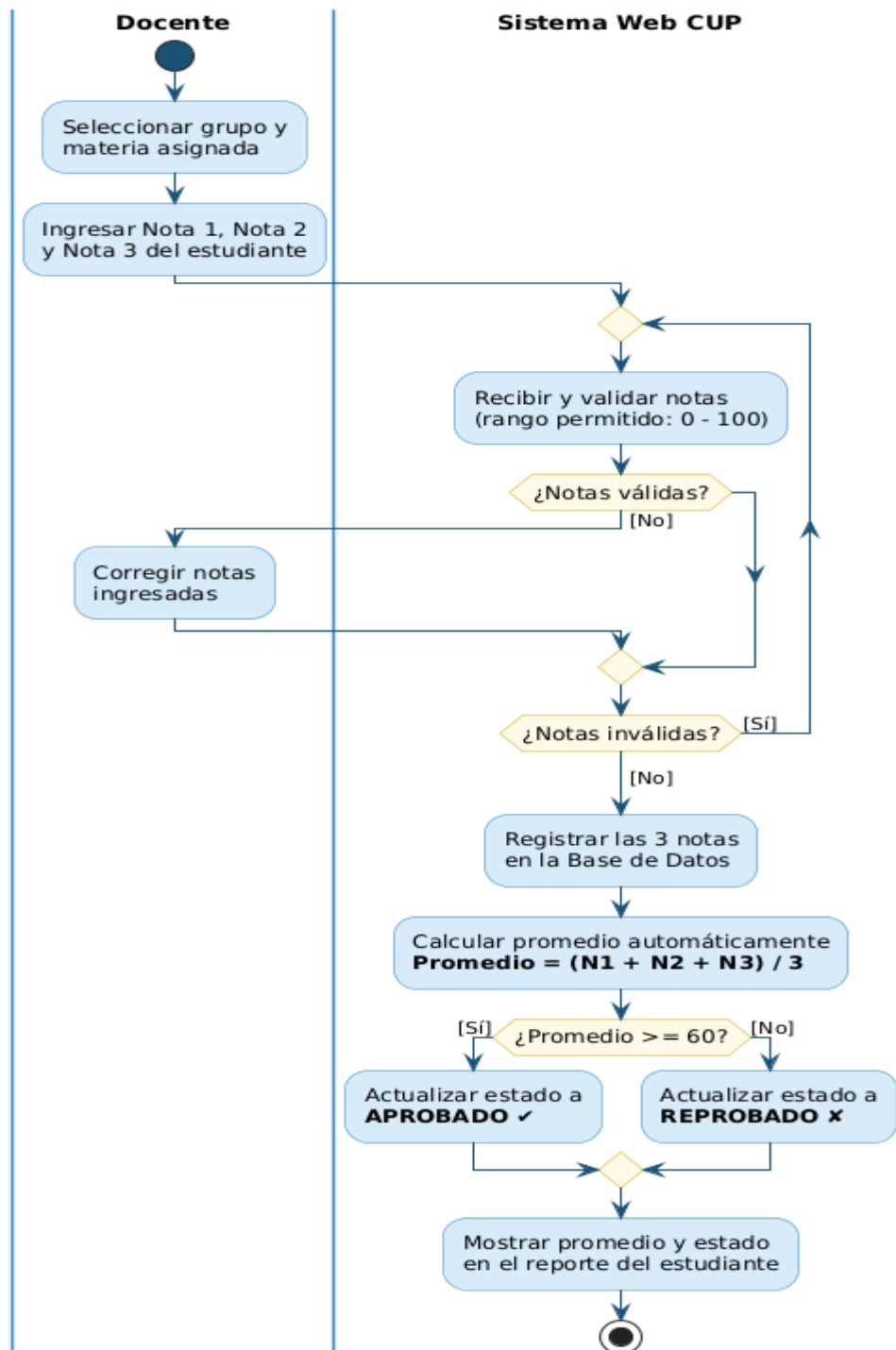
3 MODELO DE NEGOCIO

3.1 DIAGRAMA DE ACTIVIDADES

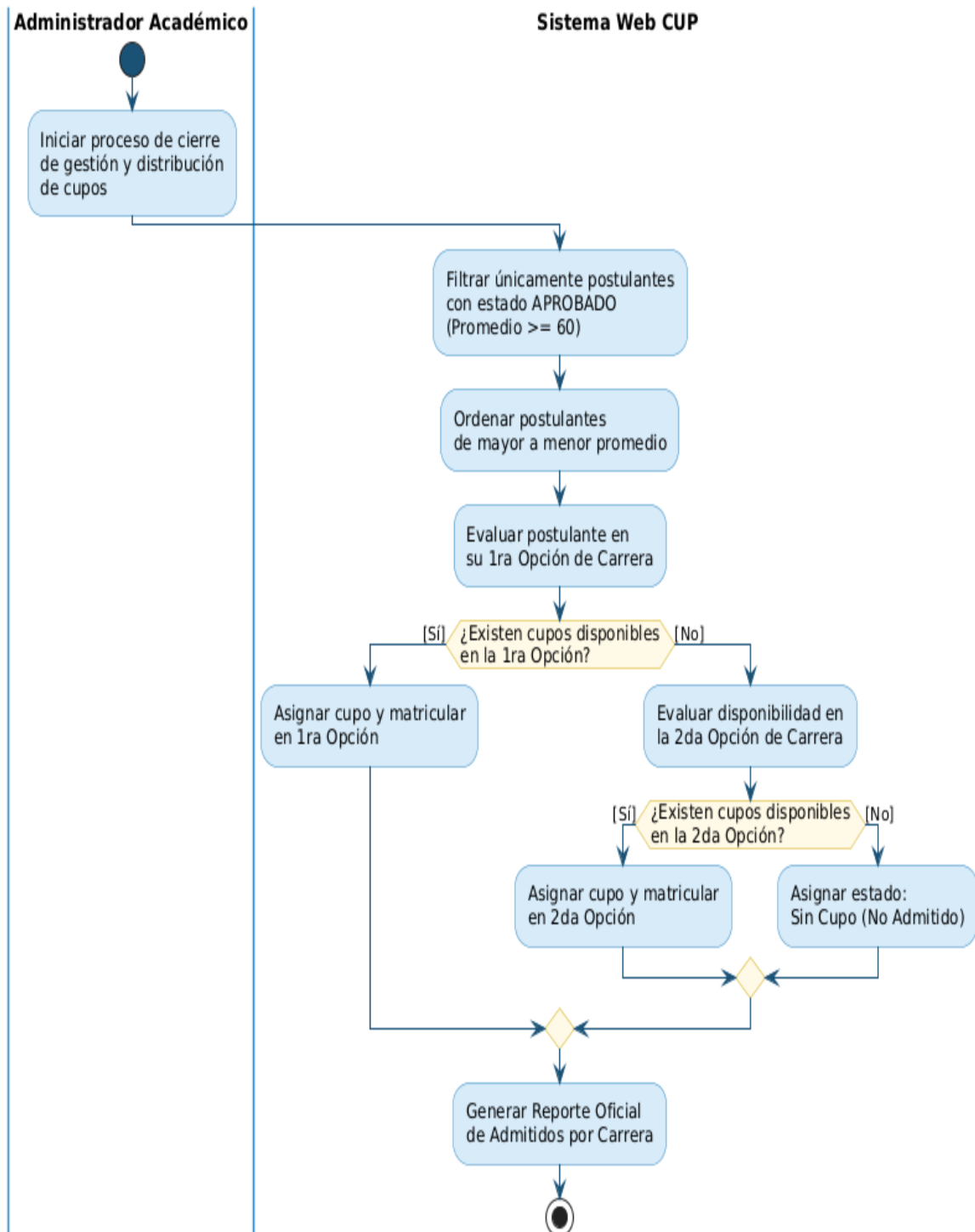
3.1.1 Diagrama de Actividad: Proceso de Inscripción y Registro



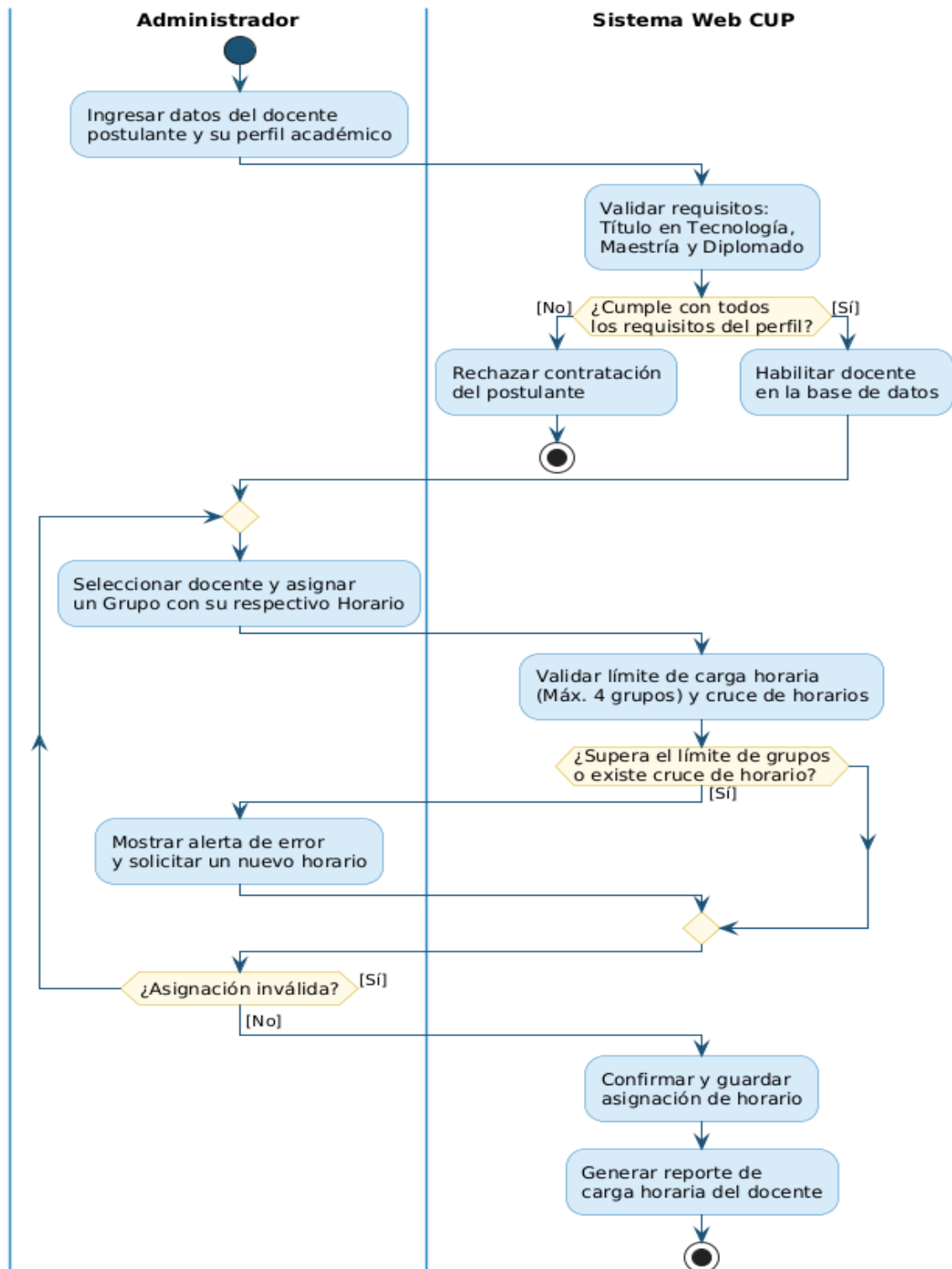
3.1.2 Proceso de Registro de Notas y Aprobación.



3.1.3 Proceso de Admisión Final (Distribución por Cupos).

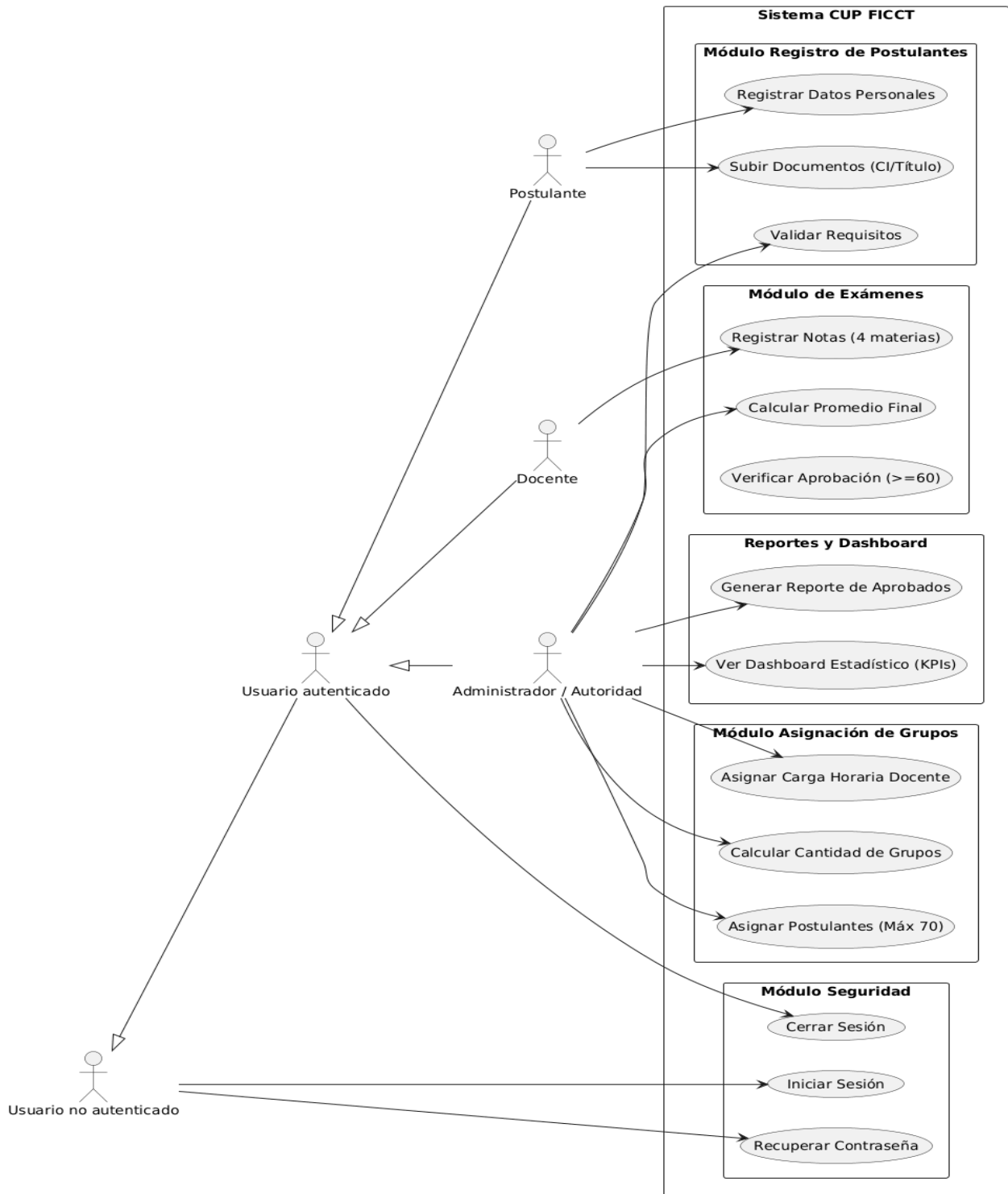


3.1.4 Contratación y Asignación de Docentes



4 FLUJO DE TRABAJO: CAPTURA DE REQUISITOS

4.1 Identificar actores del sistema



4.1.1 Actores internos del sistema

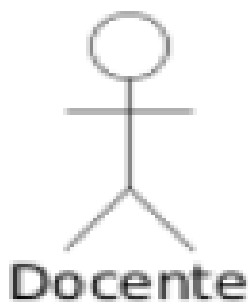
- **Administrador**



Administrador / Autoridad:

- *Rol Sistémico:* Actor con control total sobre el ecosistema del sistema CUP. Gestiona la contratación de docentes, la distribución de cupos por carrera, la asignación de grupos y horarios, y supervisa el proceso de admisión final.
- *Impacto en Vida Real:* Inicia el cierre de gestión, genera reportes oficiales de admitidos, visualiza el dashboard estadístico y garantiza que los cupos por carrera no sean superados.

- **Docente**



Docente:

- *Rol Sistémico:* Responsable del registro de notas de los postulantes en el sistema y de su propia carga académica. Ingresa las 3 calificaciones por estudiante, supervisa el proceso de evaluación dentro de su grupo asignado, y es sujeto de contratación y asignación por parte del Administrador (Módulo de Gestión de Docentes).
- *Impacto en Vida Real:* Accede al módulo de exámenes para registrar y validar notas (rango 0–100). El sistema calcula automáticamente el promedio y determina el estado APROBADO o REPROBADO. También puede visualizar su carga horaria asignada (grupos y materias).

- **Usuario Autenticado**



Usuario Autenticado:

- *Rol Sistémico:* Actor base del sistema que agrupa los privilegios compartidos entre el Docente y el Administrador. Representa a cualquier usuario con sesión activa y credenciales válidas.
- *Impacto en Vida Real:* Puede acceder a los módulos habilitados según su rol, cerrar sesión y operar dentro del sistema con permisos controlados.

4.1.2 Actores externos del sistema

- **Postulante**



Postulante:

- *Rol Sistémico:* Actor externo que inicia el proceso de admisión al CUP. Registra sus datos personales, sube documentos requeridos (CI/Título de bachiller), rinde el examen y recibe su resultado de admisión.
- *Impacto en Vida Real:* Es el sujeto principal del proceso. Su promedio final determina si accede a un cupo en su 1ra o 2da opción de carrera, o si queda sin cupo disponible.

- **Usuario no autenticado**



Usuario No Autenticado:

- *Rol Sistémico:* Actor externo que interactúa únicamente con el módulo de seguridad. No tiene acceso a ninguna funcionalidad del sistema hasta completar el proceso de autenticación.
- *Impacto en Vida Real:* Puede iniciar sesión con sus credenciales o recuperar su contraseña mediante el sistema. Una vez autenticado, se convierte en Usuario Autenticado con los permisos de su rol asignado.

4.2 Lista de casos de uso

Módulo 1: Seguridad y Autenticación

- **CU01:** Iniciar Sesión en la Plataforma
- **CU02:** Cerrar Sesión Activa
- **CU03:** Recuperar Contraseña de Usuario
- **CU04:** Cambiar Contraseña Propia
- **CU05:** Gestionar Cuentas y Roles (RBAC)

Módulo 2: Registro y Gestión de Postulantes

- **CU06:** Validar Requisitos Documentales (Título/CI)
- **CU07: Procesar Pago de Inscripción (Interacción con Pasarela)**
- **CU08:** Registrar Nuevo Postulante (Datos personales y académicos)
- **CU09:** Cargar Documentos de Respaldo Digitalizados
- **CU10:** Modificar Información del Postulante
- **CU11:** Eliminar Registro de Postulante (Baja lógica)
- **CU12:** Buscar y Listar Postulantes Inscritos

Módulo 3: Evaluación Académica (Exámenes)

- **CU13:** Registrar Notas de Materias (Computación, Matemáticas, Inglés, Física)
- **CU14:** Editar Calificaciones de Exámenes (Máximo 3 evaluaciones)
- **CU15:** Calcular Promedio Final Automáticamente
- **CU16:** Verificar Estado de Aprobación (Promedio ≥ 60)
- **CU17:** Ejecutar Algoritmo de Admisión por Cupos (1ra y 2da opción)

Módulo 4: Organización Logística (Asignación de Grupos y Aulas)

- **CU18:** Configurar Parámetros de Capacidad de Aula (Máx. 70/80)
- **CU19:** Calcular Cantidad de Grupos Necesarios (Algoritmo CEIL)
- **CU20:** Asignar Postulantes a Grupos y Aulas

Módulo 5: Reportes e Inteligencia de Negocio

- **CU23:** Generar Lista General de Postulantes
- **CU24:** Generar Reporte de Postulantes Aprobados y Reprobados
- **CU25:** Generar Estadísticas de Rendimiento por Materia
- **CU26:** Generar Reporte de Asignación Docente por Grupo
- **CU27:** Exportar Reportes a Formato PDF o Excel
- **CU28:** Consultar Reportes Académicos Mediante Comandos de Voz (IA)

Módulo 7: Gestión de Docentes

- **CU21:** Registrar Nuevo Docente y Validar Títulos (Maestría/Diplomado)
- **CU22:** Asignar Carga Horaria y Grupos a Docentes (Máx. 4 grupos)
- **CU31:** Importar Cuentas de Usuario desde Excel/CSV (Carga Masiva)

Módulo 6: Panel Administrativo (Dashboard)

- **CU29:** Visualizar Dashboard de Indicadores Clave (KPIs)
- **CU30:** Consultar Bitácora de Auditoría del Sistema (TPS)

4.3 Priorizar casos de uso

ID	Caso de Uso	Prioridad	Justificación
CU01	Iniciar Sesión en la Plataforma	Alto	Arquitectónico: Base para el RBAC y seguridad del sistema.
CU06	Validar Requisitos Documentales	Alto	Riesgo: Primer filtro del flujo de negocio. Define la entrada al sistema.
CU07	Procesar Pago de Inscripción	Alto	Riesgo/Arquitectónico: Dependencia con actor externo (Pasarela). Condición de estado crítica.

CU08	Registrar Nuevo Postulante	Alto	Arquitectónico: Define la entidad "Postulante" y el modelo de datos principal.
CU13	Registrar Notas de Materias	Alto	Valor: Corazón del sistema. Entrada para la evaluación académica.
CU15	Calcular Promedio Final Automáticamente	Alto	Riesgo Crítico: Algoritmo central de aprobación exigido en el examen.
CU17	Ejecutar Algoritmo de Admisión por Cupos	Alto	Riesgo Crítico: Lógica compleja de 1ra/2da opción y orden de mérito.
CU19	Calcular Cantidad de Grupos Necesarios	Alto	Riesgo Crítico: Algoritmo matemático CEIL para logística de la facultad.
CU20	Asignar Postulantes a Grupos y Aulas	Alto	Arquitectónico: Define la organización física y lógica de la inscripción.
CU22	Asignar Carga Horaria y Grupos a Docentes	Alto	Riesgo: Validación de perfiles y restricción de máximo 4 grupos.
CU02	Cerrar Sesión Activa	Media	Necesario para la integridad, pero de baja complejidad técnica.
CU05	Gestionar Cuentas y Roles (RBAC)	Media	Administrativo. Importante pero no es el núcleo de la admisión.

CU09	Cargar Documentos de Respaldo	Media	Complemento digital del registro de postulantes.
CU10	Modificar Información del Postulante	Media	Mantenimiento de datos (CRUD). Riesgo técnico bajo.
CU12	Buscar y Listar Postulantes Inscritos	Media	Consulta esencial para la operación diaria.
CU14	Editar Calificaciones de Exámenes	Media	Mantenimiento de la entidad "Nota". Importante para corrección de errores.
CU16	Verificar Estado de Aprobación	Media	Lógica derivada del CU15 (Cálculo de promedios).
CU18	Configurar Parámetros de Capacidad	Media	Configuración de reglas de negocio (70 u 80 alumnos).
CU21	Registrar Nuevo Docente	Media	Mantenimiento de datos. Requisito para la asignación de carga.
CU23	Generar Lista General de Postulantes	Media	Salida de información básica requerida.
CU24	Generar Reporte de Aprobados y Reprobados	Media	Resultado del proceso académico. Valor informativo alto.
CU29	Visualizar Dashboard de Indicadores (KPIs)	Media	Arquitectónico: Punto de integración de datos para BI.

CU03	Recuperar Contraseña de Usuario	Bajo	Funcionalidad de soporte opcional según el alcance.
CU04	Cambiar Contraseña Propia	Bajo	Mejora de seguridad de usuario final, no crítica para el flujo.
CU11	Eliminar Registro de Postulante	Bajo	Acción poco frecuente. Se recomienda baja lógica.
CU25	Generar Estadísticas por Materia	Bajo	Refinamiento de reportes. No detiene la operación central.
CU26	Generar Reporte de Asignación Docente	Bajo	Reporte operativo secundario.
CU27	Exportar Reportes a PDF o Excel	Bajo	Funcionalidad opcional de formato de salida.
CU28	Consultar Reportes por Voz (IA)	Bajo	Opcional: Valor estético o de innovación, no funcional base.
CU31	Importar Cuentas desde Excel/CSV	Alto	Requerimiento explícito del enunciado: generación automática de cuentas en lotes por gestión académica.
CU30	Consultar Bitácora de Auditoría (TPS)	Bajo	Función técnica para control interno y rastreo de transacciones.

4.3.1 Ciclo #1 Iteración: Arquitectura Base, Persistencia y Mitigación de Riesgos Algorítmicos

Justificación (PUDS/RUP): Esta iteración tiene como objetivo establecer la **Línea Base de la Arquitectura**. Mitiga el riesgo técnico mediante la implementación de la persistencia de datos en **PostgreSQL** para la entidad central (Postulante) y la resolución de los algoritmos matemáticos críticos (**CEIL** para grupos y promedios de notas). Siguiendo a Jacobson, se ataca primero la "parte difícil" para estabilizar el sistema antes de la construcción masiva.

- **Actores Implicados:** Administrador / Autoridad, Postulante, Docente, Pasarela de Pagos (Actor Sistema Externo).
- **R.F (Casos de Uso a implementar):**

ID	Caso de Uso	Prioridad
CU01	Iniciar Sesión en la Plataforma	Alto
CU02	Cerrar Sesión Activa	Alto
CU05	Gestionar Cuentas de Usuario y Roles (RBAC)	Alto
CU07	Procesar Pago de Inscripción (Interacción Pasarela)	Alto
CU08	Registrar Nuevo Postulante (Persistencia Base)	Alto
CU13	Registrar Notas de Materias (Computación, Matemáticas, etc.)	Alto
CU15	Calcular Promedio Final Automáticamente (Algoritmo 1)	Alto
CU16	Verificar Estado de Aprobación (Promedio $\geq$ 60)	Alto
CU19	Calcular Cantidad de Grupos Necesarios (Algoritmo CEIL)	Alto
CU20	Asignar Postulantes a Grupos y Aulas	Alto

CU29	Visualizar Dashboard de Indicadores Clave (KPIs)	Media
CU30	Consultar Bitácora de Transacciones (Auditoría del TPS)	Media
CU04	Cambiar Contraseña Propia	Bajo
CU03	Recuperar Contraseña de Usuario	Bajo
CU17	Ejecutar Algoritmo de Admisión por Cupos (1ra y 2da opción)	Alto
CU31	Importar Cuentas de Usuario desde Excel/CSV (Carga Masiva)	Alto

4.4 Detallar caso de uso

4.4.1 Ciclo #1

CU01: Iniciar Sesión en la Plataforma

[Prioridad: Alta]

Caso de Uso	CU01 : Iniciar Sesión.
Propósito	Restringir y asegurar el acceso al sistema autenticando la identidad del usuario.
Descripción	Permite que usuarios (Admin, Docente, Postulante) ingresen sus credenciales para obtener un token de acceso según su rol (RBAC).
Actores	Administrador, Docente, Postulante (Usuario autenticado con cualquier rol).
Actor iniciador	Usuario (Cualquier rol registrado).

Precondición	El usuario debe existir en PostgreSQL y su estado debe ser "Activo".
Flujo principal	1. El actor ingresa a la URL del CUP FICCT. 2. El sistema despliega el formulario de Login. 3. El actor introduce Email y Contraseña. 4. El sistema encripta la clave y valida el hash contra la BD. 5. El sistema identifica el rol y carga los permisos (RBAC). 6. El sistema registra el ingreso en la bitácora (TPS). 7. El sistema redirige al Dashboard correspondiente.
Post condición	Sesión activa vinculada al ID de usuario, habilitando transacciones según perfil.
Excepciones	E1: Credenciales inválidas (Notifica error). E2: Usuario bloqueado (Impide acceso).

CU07: Procesar Pago de Inscripción [Prioridad: Alta]

Caso de Uso	CU07 : Procesar Pago de Inscripción.
Propósito	Validar el pago arancelario obligatorio para habilitar el registro.
Descripción	El sistema interactúa con una pasarela de pagos externa para confirmar la transacción del postulante.

Actores	Postulante, Pasarela de Pagos (Sistema Externo).
Actor iniciador	Postulante.
Precondición	El postulante debe haber validado sus documentos (CI/Título).
Flujo principal	1. El actor selecciona "Realizar Pago". 2. El sistema genera un ID de orden y redirige a la pasarela externa. 3. El actor completa el pago en la plataforma externa. 4. La pasarela devuelve confirmación de éxito al sistema. 5. El sistema registra el pago en PostgreSQL (Propiedades ACID). 6. El sistema cambia el estado a "Habilitado para Registro".
Post condición	El estado de pago queda persistido, desbloqueando el CU08.
Excepciones	E1: Transacción rechazada por la pasarela (Permite reintento).

CU08: Registrar Nuevo Postulante [Prioridad: Alta]

Caso de Uso	CU08 : Registrar Postulante.
Prioridad	Alta (Arquitectónico: Define la entidad principal y modelo de datos).
Propósito	Capturar y persistir los datos maestros del aspirante a la FICCT.
Descripción	Registro integral de información personal y académica en la base de datos PostgreSQL.

Actores	Postulante, Administrador (Operador de inscripción).
Actor iniciador	Postulante / Operador.
Precondición	El pago debe estar verificado con éxito en el sistema (CU07).
Flujo principal	1. El actor llena el formulario (CI, Nombres, Colegio, Carreras 1 y 2). 2. El sistema valida que el CI no sea duplicado. 3. El sistema valida formato de correo y campos obligatorios. 4. El sistema guarda el registro en PostgreSQL (Garantizando Integridad ACID). 5. El sistema confirma el registro exitoso.
Post condición	El postulante queda almacenado y listo para asignación de grupos.
Excepciones	E1: CI Duplicado (Muestra alerta de error).

CU15: Calcular Promedio Final Automáticamente [Prioridad: Alta]

Caso de Uso	CU15 : Calcular Promedio Final.
Prioridad	Alta (Riesgo Crítico: Algoritmo central de aprobación exigido).
Propósito	Ejecutar la lógica algorítmica de evaluación académica por materia.
Descripción	El sistema procesa las 3 notas de las 4 materias troncales para obtener el promedio final.

Actores	Sistema Web CUP (actor automático, se dispara tras registrar la 3ra nota).
Actor iniciador	Sistema (Automático tras el registro de la tercera nota).
Precondición	Deben existir las 3 notas cargadas por cada materia (Matemáticas, Física, etc.).
Flujo principal	1. El sistema recupera las notas registradas. 2. El sistema aplica la fórmula: $\text{Promedio} = (\text{Nota1} + \text{Nota2} + \text{Nota3}) / 3$. 3. El sistema actualiza el registro del postulante con su nota final. 4. El sistema verifica si $\text{Promedio} \geq 60$ para asignar estado "Aprobado".
Post condición	Promedio calculado y persistido, habilitando el proceso de admisión por cupos.
Excepciones	E1: Faltan notas por registrar (Detiene el cálculo).

CU19: Calcular Cantidad de Grupos Necesarios [Prioridad: Alta]

Caso de Uso	CU19 : Calcular Cantidad de Grupos.
Prioridad	Alta (Riesgo Crítico: Algoritmo logístico fundamental).
Propósito	Determinar la logística de aulas basándose en la demanda real de inscritos.
Descripción	Aplicación del algoritmo matemático de techo para organizar a los inscritos en grupos.

Actores	Administrador / Autoridad.
Actor iniciador	Administrador / Autoridad.
Precondición	El proceso de inscripción debe estar en fase de auditoría o cierre.
Flujo principal	1. El sistema cuenta el total de alumnos en estado "Inscrito". 2. El sistema aplica la fórmula: $\text{CantGrupos} = \text{CEIL}(\text{TotalInscritos} / 80)$. 3. El sistema muestra la cantidad de grupos resultantes. 4. El Administrador aprueba la creación de los grupos en la base de datos.
Post condición	Grupos creados y habilitados para la asignación de docentes.
Excepciones	E1: Total de inscritos es cero (Notifica advertencia).

CU13: Registrar Notas de Materias

[Prioridad: Alta]

Caso de Uso	CU13 : Registrar Notas de Materias.
Prioridad	Alta (Valor: Entrada de datos para la lógica central de evaluación).
Propósito	Registrar las calificaciones de los exámenes de las materias troncales (Computación, Matemáticas, Inglés y Física).
Descripción	Permite que el Docente o Administrador ingrese hasta 3 evaluaciones por cada una de las 4 materias obligatorias.

Actores	Docente, Administrador.
Actor iniciador	Docente / Administrador.
Precondición	El postulante debe estar debidamente registrado en el sistema y el periodo de evaluación debe estar abierto.
Flujo principal	1. El actor selecciona al postulante de una lista. 2. El sistema despliega el formulario de notas por materia. 3. El actor ingresa el valor de la calificación (0-100). 4. El sistema valida que el valor esté en el rango permitido. 5. El sistema persiste la nota en la tabla de PostgreSQL. 6. El sistema dispara el cálculo automático del promedio (CU15).
Post condición	Notas almacenadas de forma íntegra, listas para el procesamiento académico.
Excepciones	E1: Nota fuera de rango (Muestra error y no guarda). E2: Intento de registrar más de 3 notas por materia (Bloquea acción).

CU16: Verificar Estado de Aprobación

[Prioridad: Alta]

Caso de Uso	CU16 : Verificar Estado de Aprobación.
Prioridad	Alta (Valor: Define el resultado académico inmediato según reglas de la FICCT).
Propósito	Clasificar formalmente al postulante como "Aprobado" o "Reprobado".

Descripción	Basado en el promedio final calculado, el sistema aplica la regla estricta de aprobación de la facultad.
Actores	Sistema (Automático).
Actor iniciador	Sistema (Se dispara tras CU15).
Precondición	El promedio final debe haber sido calculado y persistido en la BD.
Flujo principal	1. El sistema recupera el valor de "promedio_final" del postulante. 2. El sistema aplica la regla de negocio: Si promedio >= 60 ENTONCES estado = 'APROBADO'. 3. El sistema actualiza el campo de estado en el registro del estudiante. 4. El sistema notifica el cambio de estado internamente para reportes futuros.
Post condición	El estado del postulante queda actualizado, habilitándolo para el Algoritmo de Admisión.
Excepciones	E1: Promedio inexistente o nulo (Interrumpe la validación).

CU17:

Ejecutar Algoritmo de Admisión por Cupos

[Prioridad: Alta]

Caso de Uso	CU17 : Ejecutar Algoritmo de Admisión por Cupos.
Prioridad	Alta (Riesgo Crítico: Lógica compleja de 1ra/2da opción y orden de mérito).
Propósito	Distribuir a los postulantes APROBADOS en las carreras disponibles respetando el límite de cupos por gestión y el orden de mérito (promedio mayor tiene prioridad).
Descripción	El sistema procesa la lista de aprobados ordenada por promedio de mayor a menor. Para cada postulante verifica si hay cupo disponible en su 1ra opción de carrera; si no hay cupo, intenta su 2da opción. Si ninguna opción tiene cupo disponible, el postulante queda en lista de espera.
Actores	Sistema Web CUP (actor automático). Administrador / Autoridad (aprueba el cierre del proceso).
Actor iniciador	Administrador (ejecuta el cierre del proceso de admisión).
Precondición	El proceso de evaluación debe estar cerrado. Todos los promedios deben estar calculados y persistidos (CU15). Cada carrera debe tener

	configurado su cupo máximo de admitidos por gestión.
Flujo principal	1. El Administrador selecciona "Ejecutar proceso de admisión". 2. El sistema recupera la lista de postulantes con estado APROBADO. 3. El sistema ordena la lista de mayor a menor por promedio_final. 4. Para cada postulante en orden de mérito: a. El sistema verifica si la 1ra opción de carrera tiene cupo disponible. b. Si SÍ: asigna al postulante a la 1ra opción y descuenta 1 cupo. c. Si NO: el sistema verifica la 2da opción de carrera. d. Si la 2da opción tiene cupo: asigna al postulante y descuenta 1 cupo. e. Si ninguna opción tiene cupo: el postulante queda en estado Lista de Espera. 5. El sistema persiste las asignaciones finales en la base de datos. 6. El sistema genera el reporte oficial de admitidos por carrera.
Postcondición	Cada postulante APROBADO tiene asignada una carrera o consta en Lista de Espera. Los cupos por carrera quedan actualizados. El reporte de admitidos queda disponible para publicación.
Excepciones	E1: Total de aprobados es cero (el sistema notifica y no ejecuta el algoritmo). E2: Una carrera tiene cupo configurado en cero (el sistema advierte al Administrador antes de ejecutar). E3: Empate de promedios en el último cupo disponible (el sistema aplica criterio secundario: fecha de inscripción más temprana tiene prioridad).

CU20: Asignar Postulantes a Grupos y Aulas [Prioridad: Alta]

Caso de Uso	CU20 : Asignar Postulantes a Grupos.
Prioridad	Alta (Arquitectónico: Define la organización operativa básica).
Propósito	Distribuir físicamente a los estudiantes inscritos en las aulas habilitadas.
Descripción	Proceso logístico que vincula a cada postulante con un grupo específico, respetando el límite de capacidad.
Actores	Administrador / Sistema Web CUP (asignación automática).
Actor iniciador	Administrador / Sistema.
Precondición	Se debe haber ejecutado previamente el cálculo de grupos necesarios (CU19).
Flujo principal	1. El sistema recupera la lista de estudiantes inscritos sin grupo asignado. 2. El sistema distribuye a los estudiantes secuencialmente en los grupos habilitados. 3. El sistema valida que ningún grupo supere el límite máximo de 70 estudiantes. 4. El sistema guarda la relación postulante-grupo en la base de datos. 5. Se confirma la asignación exitosa.
Post condición	Postulantes organizados en grupos, listos para la asistencia a clases.
Excepciones	E1: Capacidad de grupos excedida (Requiere habilitar nuevo grupo).

CU05: Gestionar Cuentas de Usuario y Roles (RBAC) [Prioridad: Alta]

Caso de Uso	CU05 : Gestionar Cuentas y Roles.
Prioridad	Alta (Arquitectónico: Pilar de la seguridad y privacidad de datos).
Propósito	Administrar perfiles de usuario y asignar permisos específicos basados en el rol (Admin, Docente, Postulante).
Descripción	Creación y configuración de credenciales de acceso para garantizar que los docentes solo vean su carga horaria y los postulantes sus notas.
Actores	Administrador General.
Actor iniciador	Administrador General.
Precondición	El Administrador debe estar autenticado con privilegios máximos.
Flujo principal	1. El actor accede al panel de Gestión de Usuarios. 2. El actor selecciona "Crear Nuevo Usuario" o "Editar Rol". 3. El sistema permite vincular al usuario con un perfil (RBAC). 4. El sistema asigna los permisos de lectura/escritura correspondientes. 5. El sistema registra la acción en la bitácora de auditoría. 6. El sistema confirma la actualización del perfil.
Post condición	Perfiles de seguridad actualizados en PostgreSQL, garantizando la privacidad de la información.

Excepciones	E1: Rol no definido en el sistema (Impide la asignación).
--------------------	---

CU02: Cerrar Sesión Activa

[Prioridad: Media]

Caso de Uso	CU02 : Cerrar Sesión.
Prioridad	Media (Importante para la integridad de la sesión y seguridad).
Propósito	Finalizar de forma segura la interacción del usuario con la plataforma para evitar accesos no autorizados.
Descripción	Permite que el usuario autenticado invalide su token de acceso y salga del entorno del sistema.
Actores	Usuario autenticado (cualquier rol: Administrador, Docente, Postulante).
Actor iniciador	Usuario autenticado (Cualquier rol).
Precondición	Debe existir una sesión activa en el navegador.
Flujo principal	1. El actor selecciona la opción "Cerrar Sesión" en el menú de perfil. 2. El sistema destruye la sesión del lado servidor. 3. El sistema limpia las cookies/tokens locales en el navegador. 4. El sistema redirige al actor a la pantalla de Login (CU01).
Post condición	La sesión queda invalidada; el acceso a rutas protegidas es bloqueado.
Excepciones	E1: Falla de red (La sesión expira por tiempo o el usuario debe limpiar caché).

CU30: Consultar Bitácora de Transacciones (TPS)**[Prioridad: Media]**

Caso de Uso	CU30 : Consultar Bitácora de Transacciones.
Prioridad	Media (Arquitectónico: Garantiza la auditabilidad y las propiedades ACID).
Propósito	Monitorear y auditar las acciones críticas realizadas por los usuarios en el sistema.
Descripción	Permite al administrador visualizar un registro histórico (log) de inserciones, modificaciones y eliminaciones.
Actores	Administrador, Tabla PostgreSQL (bitacora_transacciones).
Actor iniciador	Administrador.
Precondición	El administrador debe tener permisos de auditoría.
Flujo principal	1. El actor accede al módulo de Auditoría. 2. El sistema consulta la tabla de bitácora en PostgreSQL. 3. El sistema muestra la lista de transacciones con fecha, hora, IP y usuario iniciador. 4. El actor puede filtrar por fecha o tipo de operación (Login, Registro, Notas).
Post condición	Visibilidad del rastro de auditoría para control interno.
Excepciones	E1: Tabla vacía (Muestra mensaje: "Sin registros para el periodo").

CU04: Cambiar Contraseña Propia**[Prioridad: Bajo]**

Caso de Uso	CU04 : Cambiar Contraseña.
Prioridad	Bajo (Funcionalidad de soporte y mantenimiento de credenciales).
Propósito	Permitir que el usuario actualice sus credenciales por seguridad.
Descripción	El usuario ingresa su clave actual y define una nueva cumpliendo parámetros de seguridad.
Actores	Usuario autenticado (cualquier rol).
Actor iniciador	Usuario autenticado.
Precondición	El usuario debe estar logueado satisfactoriamente.
Flujo principal	1. El actor selecciona "Cambiar Contraseña". 2. El actor ingresa contraseña actual y la nueva contraseña (dos veces). 3. El sistema valida que la contraseña actual coincida con el hash de la BD. 4. El sistema encripta la nueva clave y actualiza PostgreSQL. 5. El sistema confirma el cambio exitoso.
Post condición	Credenciales actualizadas; se requiere la nueva clave en el próximo login.
Excepciones	E1: Contraseña actual incorrecta (Bloquea el cambio). E2: Las nuevas contraseñas no coinciden.

CU03: Recuperar Contraseña de Usuario [Prioridad: Bajo]

Caso de Uso	CU03 : Recuperar Contraseña.
Prioridad	Bajo (Opcional según el alcance del proyecto).
Propósito	Restablecer el acceso al sistema para usuarios que olvidaron su clave.
Descripción	Proceso mediante el cual el sistema envía un enlace de reinicio o genera una clave temporal vía correo electrónico.
Actores	Usuario no autenticado, Servidor de Correo Externo.
Actor iniciador	Usuario (Admin, Docente, Postulante).
Precondición	El correo electrónico ingresado debe existir en la base de datos.
Flujo principal	1. El actor pulsa "¿Olvidó su contraseña?" en el login. 2. El actor introduce su correo electrónico registrado. 3. El sistema valida el correo y genera un token de seguridad temporal. 4. El sistema envía un email con el enlace de recuperación. 5. El actor accede al enlace y define una nueva clave.
Post condición	El acceso del usuario es restaurado con una nueva contraseña.
Excepciones	E1: Correo no registrado (Notifica que el usuario no existe).

CU17: Ejecutar Algoritmo de Admisión por Cupos**[Prioridad: Alto]**

Caso de Uso	CU17 : Ejecutar Algoritmo de Admisión por Cupos.
Propósito	Asignar automáticamente postulantes a carreras según sus puntajes, opciones preferidas y cupos disponibles.
Descripción	El Administrador dispara el algoritmo de admisión que recorre todos los postulantes aprobados, los ordena por puntaje descendente y los asigna a su 1ra o 2da opción de carrera según disponibilidad de cupos. El resultado queda registrado en la BD.
Actores	Administrador.
Precondición	Existen postulantes con estado "Aprobado" y cupos configurados por carrera. El periodo de admisión está activo.
Flujo principal	1. Administrador inicia el proceso de admisión. 2. Sistema recupera postulantes aprobados y los ordena por puntaje. 3. Sistema recorre la lista asignando cada postulante a su 1ra opción si hay cupo; si no, a la 2da. 4. Sistema persiste resultados y genera reporte de asignación.

Flujo alternativo	Si ambas opciones de un postulante están sin cupo, el sistema lo marca como "Sin plaza" y notifica al administrador.
Postcondición	Todos los postulantes aprobados tienen un estado de asignación registrado (carrera asignada o sin plaza). Los cupos de cada carrera se actualizaron.
Prioridad	Alto

CU31: Importar Cuentas de Usuario desde Excel/CSV [Prioridad: Alto]

Caso de Uso	CU31 : Importar Cuentas de Usuario desde Excel/CSV.
Propósito	Permitir la carga masiva de cuentas de docentes y postulantes desde archivos Excel o CSV, evitando el registro manual uno a uno.
Descripción	El Administrador sube un archivo Excel o CSV con columnas predefinidas (nombre, CI, correo, rol, carrera). El sistema valida el formato, detecta duplicados y crea las cuentas en bloque generando contraseñas temporales automáticas.
Actores	Administrador.

Precondición	El administrador tiene el archivo Excel/CSV con el formato correcto de columnas. Tiene sesión activa con rol Administrador.
Flujo principal	1. Administrador sube el archivo Excel/CSV. 2. Sistema valida estructura de columnas y formato. 3. Sistema detecta duplicados por CI o correo. 4. Sistema crea cuentas nuevas en bloque con contraseña temporal. 5. Sistema devuelve resumen: X creadas, Y duplicadas, Z con error.
Flujo alternativo	Si el archivo tiene formato incorrecto o columnas faltantes, el sistema rechaza la importación y muestra los errores específicos por fila.
Postcondición	Las cuentas válidas del archivo quedan creadas en el sistema con contraseña temporal. El administrador tiene el reporte de resultados de la importación.
Prioridad	Alto

CU29: Visualizar Dashboard de Indicadores (KPIs) [Prioridad: Media]

Caso de Uso	CU29 : Visualizar Dashboard.
Prioridad	Media (Arquitectónico: Punto de integración de datos para la toma de decisiones).

Propósito	Facilitar el control administrativo mediante un resumen ejecutivo en tiempo real de los indicadores clave del proceso de admisión.
Descripción	Panel gráfico que transforma los datos crudos de inscripciones y notas en indicadores estadísticos para agilizar la toma de decisiones del coordinador.
Actores	Administrador / Autoridades de la FICCT.
Actor iniciador	Administrador.
Precondición	El usuario debe estar autenticado con rol de Administrador y deben existir registros previos en PostgreSQL.
Flujo principal	1. El actor selecciona la opción "Dashboard" en el menú de navegación. 2. El sistema realiza consultas agregadas a las tablas de PostgreSQL (postulantes, notas, grupos). 3. El sistema calcula en tiempo real los 4 KPIs principales: Total inscritos, Total aprobados, Total reprobados y Total grupos habilitados. 4. El sistema procesa los datos mediante lógica de BI para generar gráficos de rendimiento por materia. 5. El sistema renderiza el panel visual adaptado al dispositivo (Responsive).
Post condición	Visibilidad total del estado operativo y académico del CUP para las autoridades.
Excepciones	E1: Base de datos vacía (El sistema muestra los indicadores en cero). E2: Error de concurrencia (El sistema refresca los datos garantizando aislamiento - ACID).

CU31:

Importar Cuentas de Usuario desde Excel/CSV

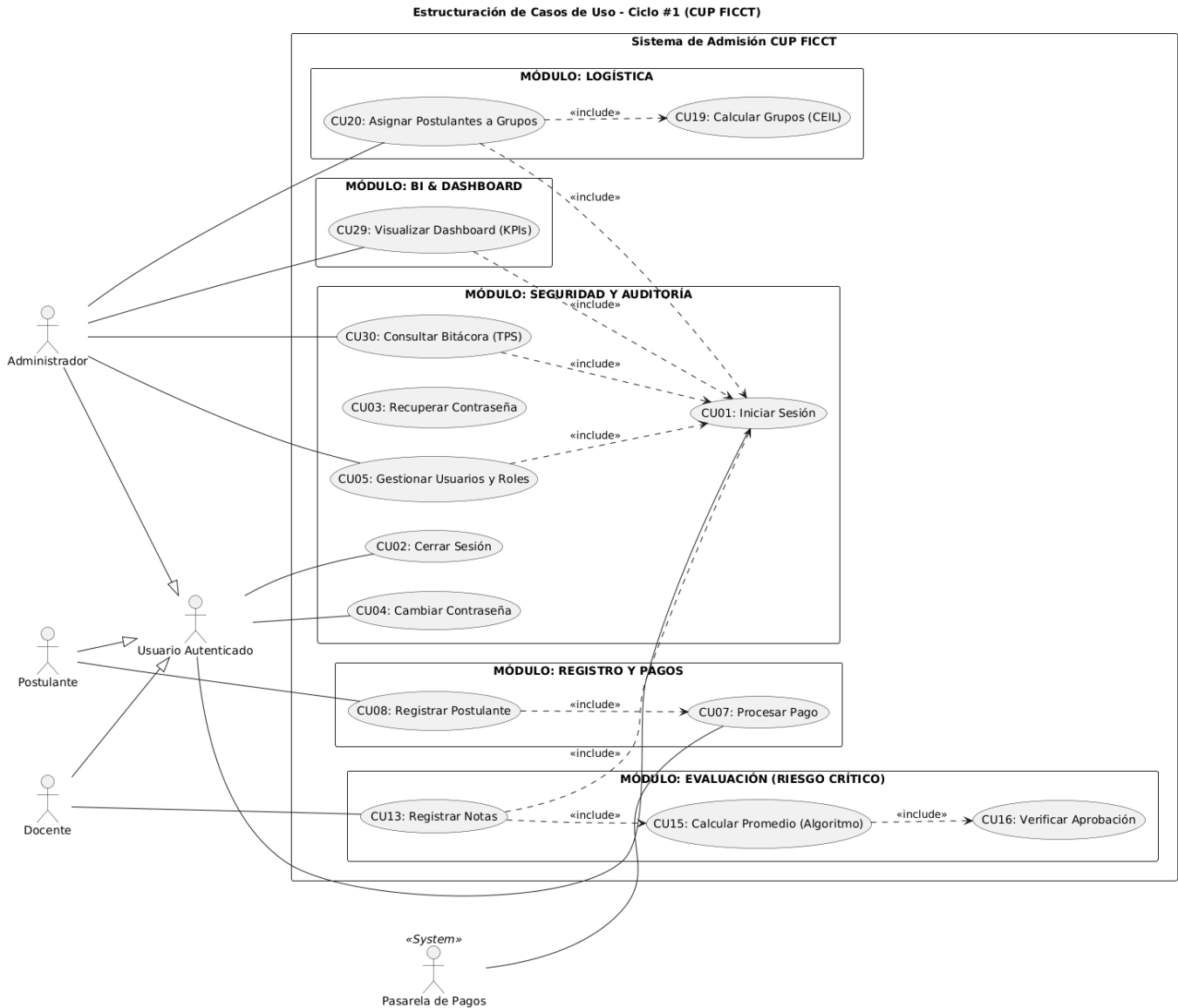
[Prioridad: Alta]

Caso de Uso	CU31 : Importar Cuentas de Usuario desde Excel/CSV.
Prioridad	Alta (Requerimiento explícito del enunciado: generación automática de cuentas en lotes).
Propósito	Permitir que la administración académica cargue masivamente las cuentas de docentes, autoridades y coordinadores al inicio de cada gestión, evitando la creación manual una por una.
Descripción	El Administrador carga un archivo Excel (.xlsx) o CSV con los datos del personal académico. El sistema valida el formato, procesa cada fila, genera automáticamente las credenciales de acceso y asigna el perfil de rol (RBAC) correspondiente a cada cuenta importada.
Actores	Administrador General. Archivo Excel/CSV (fuente de datos externa provista por la administración de la FICCT).
Actor iniciador	Administrador General.
Precondición	El Administrador debe estar autenticado con privilegios máximos. El archivo Excel/CSV debe seguir la plantilla oficial del sistema (columnas: CI, Nombres, Apellidos, Correo, Rol, Especialidad).
Flujo principal	1. El actor accede al módulo "Importación masiva de usuarios". 2. El actor sube el archivo Excel o

	CSV desde su dispositivo. 3. El sistema valida que el formato del archivo sea correcto (extensión y estructura de columnas). 4. El sistema procesa cada fila del archivo: a. Verifica que el CI no esté duplicado en la base de datos. b. Valida que el campo Rol tenga un valor permitido (Docente, Autoridad, Coordinador). c. Genera automáticamente la contraseña temporal encriptada. d. Crea la cuenta de usuario en PostgreSQL con el perfil RBAC asignado. 5. El sistema genera un reporte de importación con: filas exitosas, filas con error y motivo de cada error. 6. El sistema notifica al Administrador el resultado total de la importación.
Postcondición	Las cuentas válidas quedan creadas en el sistema con credenciales generadas y perfil RBAC asignado. Las filas con error quedan registradas en el reporte para corrección manual.
Excepciones	E1: Formato de archivo incorrecto (el sistema rechaza la carga y muestra el formato esperado). E2: CI duplicado en el archivo o en la BD (la fila se marca como error, el resto se procesa). E3: Rol inválido en una fila (la fila se descarta y se reporta). E4: Archivo vacío o sin datos (el sistema notifica y cancela la importación).

4.5 Estructurar caso de uso

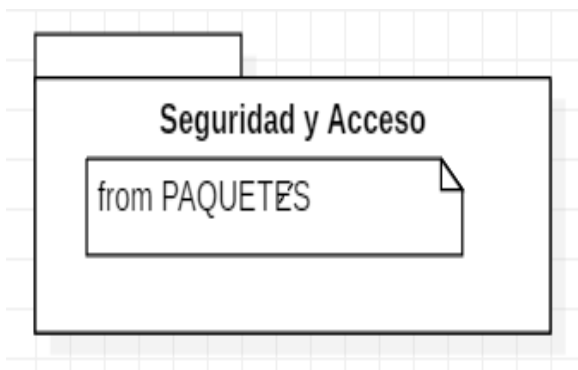
4.5.1 Ciclo #1



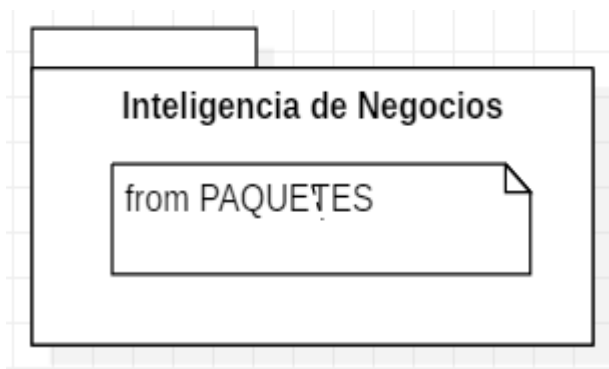
5 FLUJO DE TRABAJO: ANÁLISIS

5.1 Análisis de arquitectura

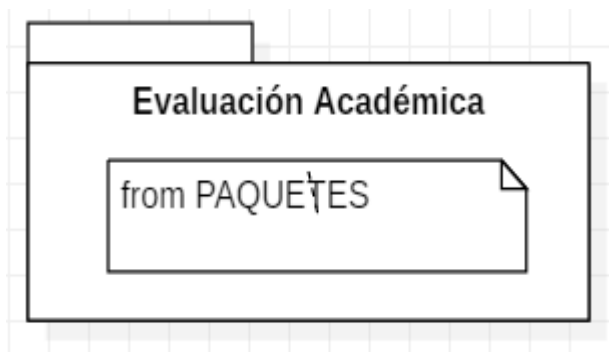
5.1.1 Identificar paquetes



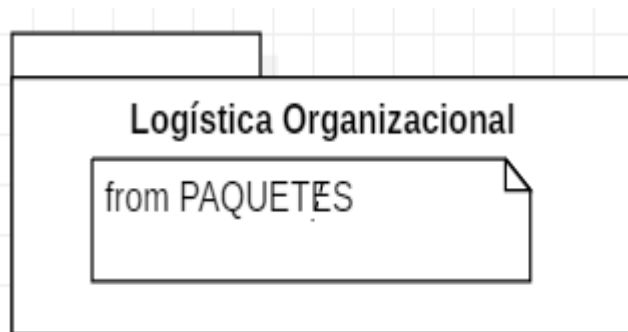
SEGURIDAD Y ACCESO Este paquete comprende los mecanismos de protección y control de acceso al sistema CUP. Su propósito es validar las credenciales del personal administrativo, docentes y postulantes mediante el esquema **RBAC**, gestionando sesiones seguras y la bitácora de transacciones (TPS) para proteger la integridad de los datos académicos.



REGISTRO Y ADMISIÓN Encargado del flujo de inscripción formal del postulante. Gestiona la captura de datos maestros en **PostgreSQL**, la validación de requisitos documentales y la integración con la pasarela de pagos externa, asegurando que solo los bachilleres habilitados financieramente procedan a la fase de evaluación.

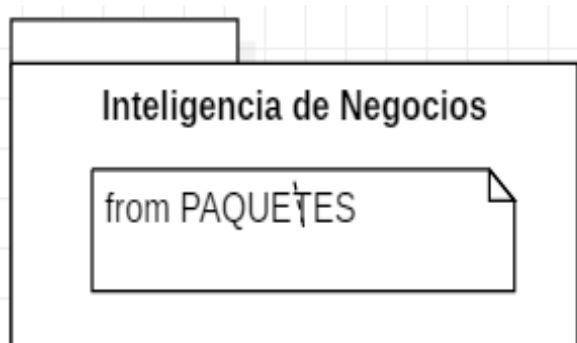


EVALUACIÓN ACADÉMICA Contiene la lógica de negocio central para el procesamiento de calificaciones. Implementa los controladores responsables de gestionar las notas de las 4 materias troncales y ejecutar el **Algoritmo de Promedios** exigido para determinar el estado de aprobación (≥ 60) de forma automatizada.



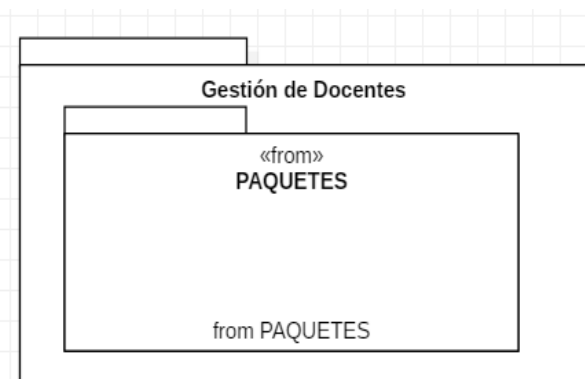
LOGÍSTICA ORGANIZACIONAL

Administra la organización física del curso de nivelación. Su función principal es ejecutar el algoritmo matemático **CEIL** para calcular la cantidad de grupos necesarios basándose en el límite de capacidad de aulas (70 u 80 alumnos) y realizar la asignación masiva de estudiantes inscritos.



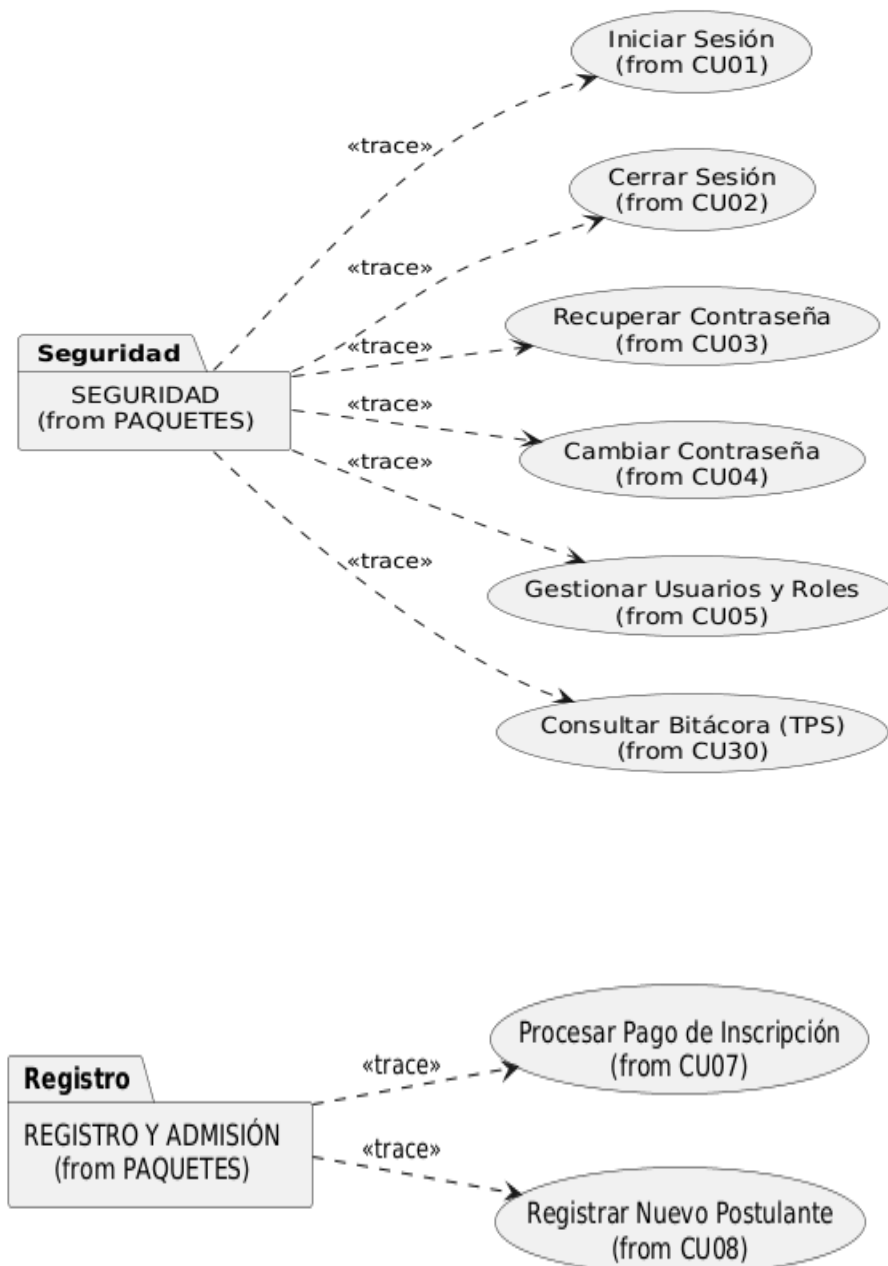
INTELIGENCIA DE NEGOCIOS (BI)

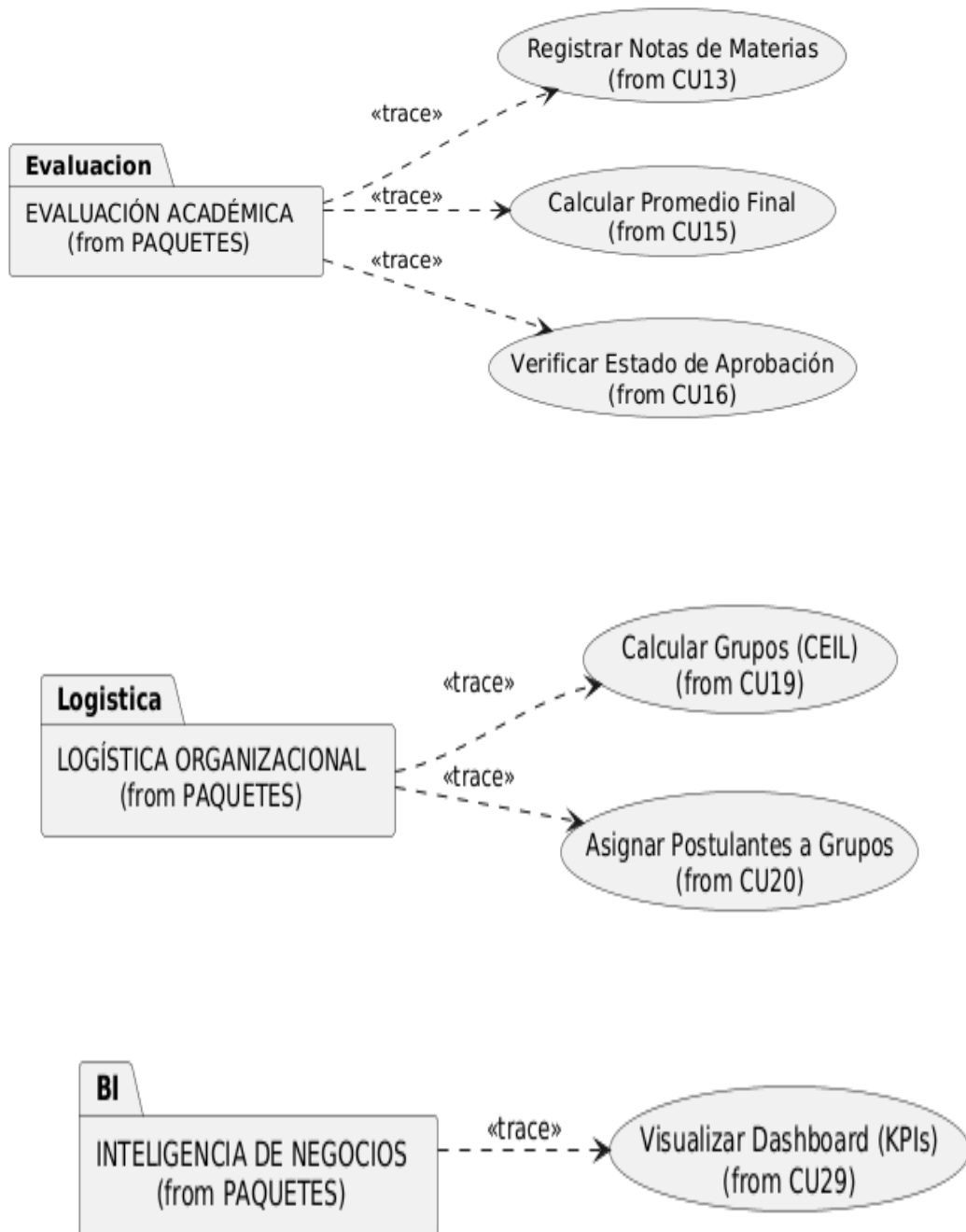
Este paquete transforma los datos transaccionales brutos en información estratégica. Es responsable de la generación de indicadores clave (KPIs) visualizados en el **Dashboard**, permitiendo a las autoridades de la FICCT monitorear en tiempo real los índices de aprobación y la ocupación de grupos.

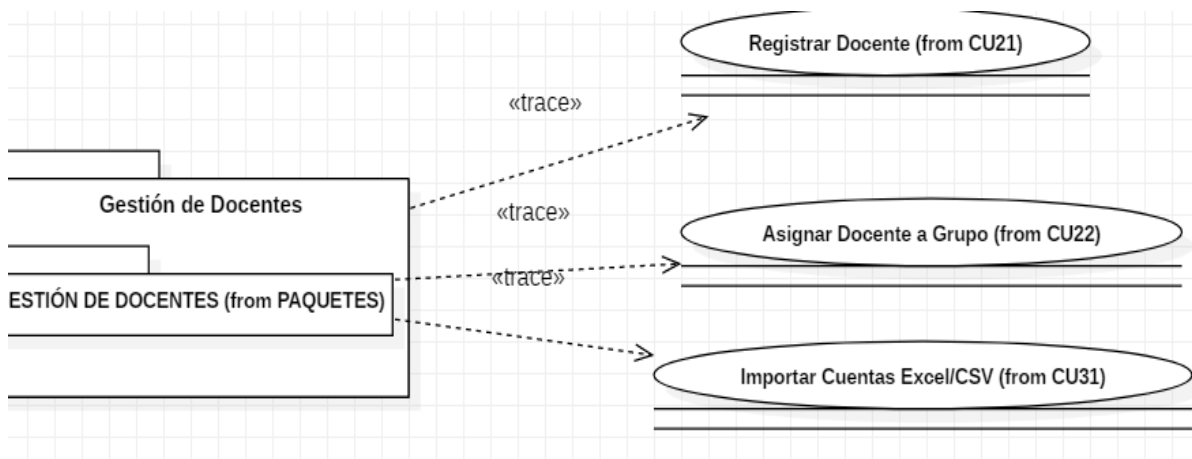


GESTIÓN DE DOCENTES — Administra el ciclo completo de los docentes dentro de la plataforma. Su función principal es permitir al Administrador registrar nuevos docentes (CU21), asignarlos a grupos y materias específicas (CU22) e importar cuentas masivamente desde archivos Excel/CSV (CU31), garantizando que cada grupo del curso de nivelación cuente con un docente asignado antes del inicio de clases.

5.1.2 Relacionar paquetes y casos de uso

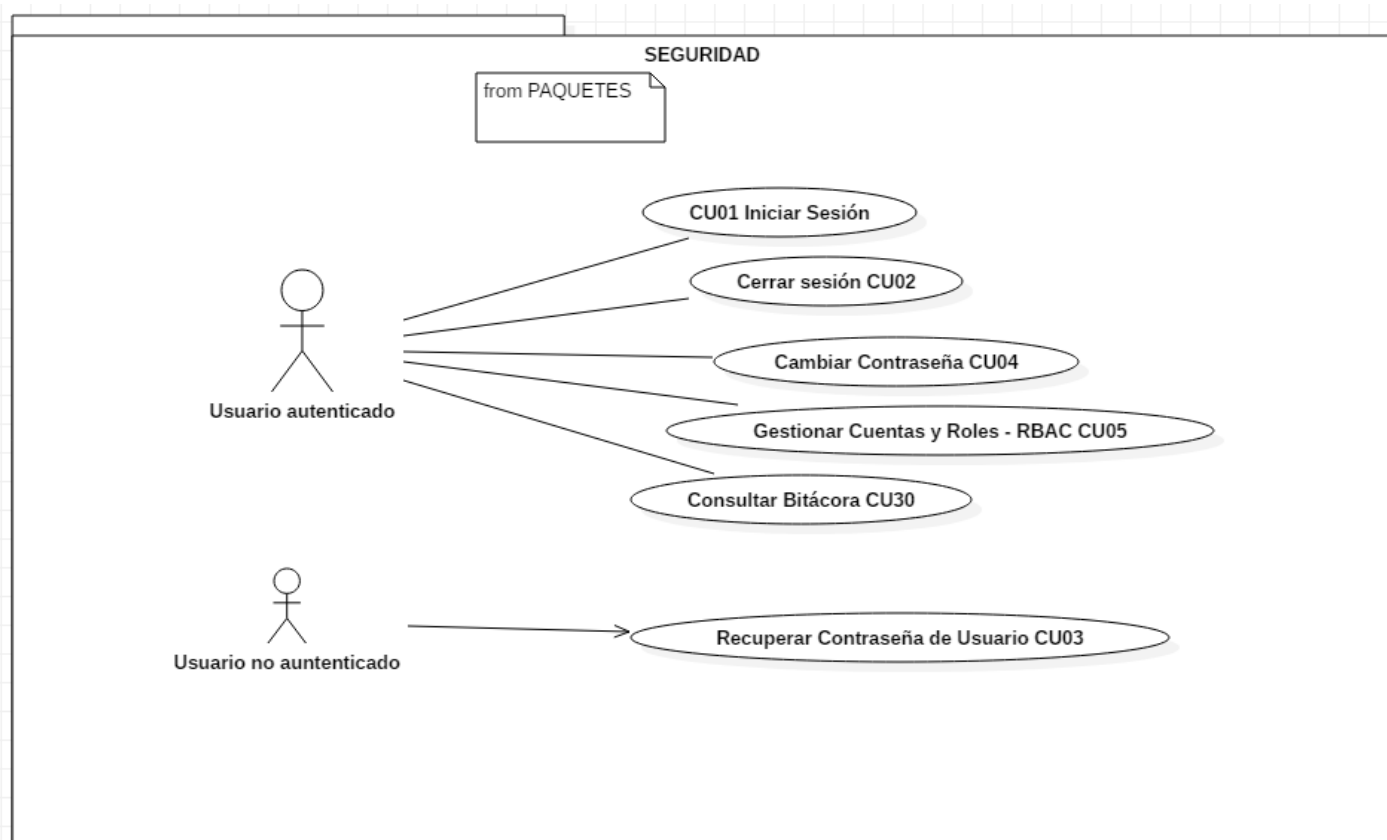




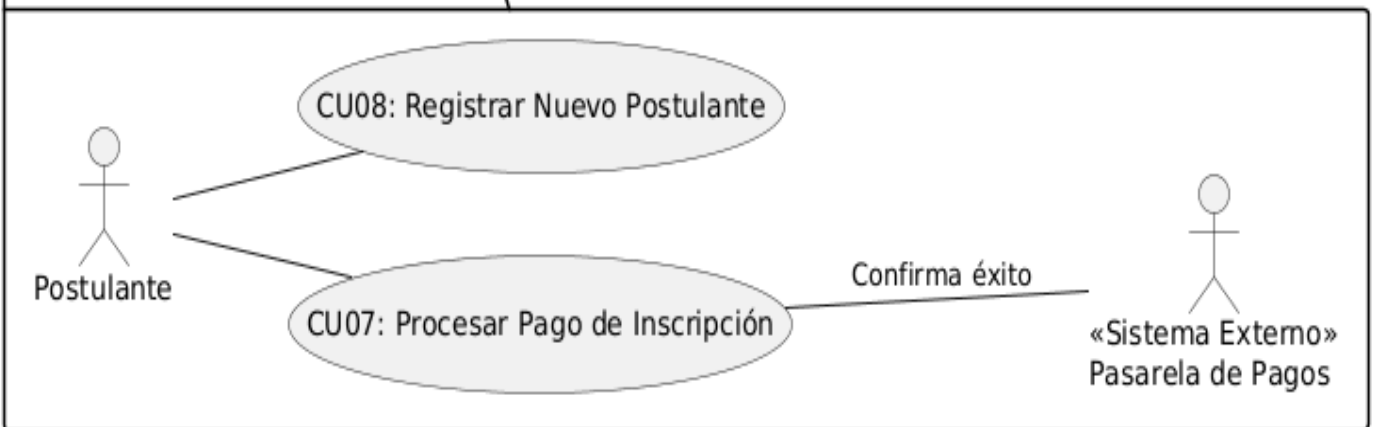


5.1.3 Vista de casos de uso

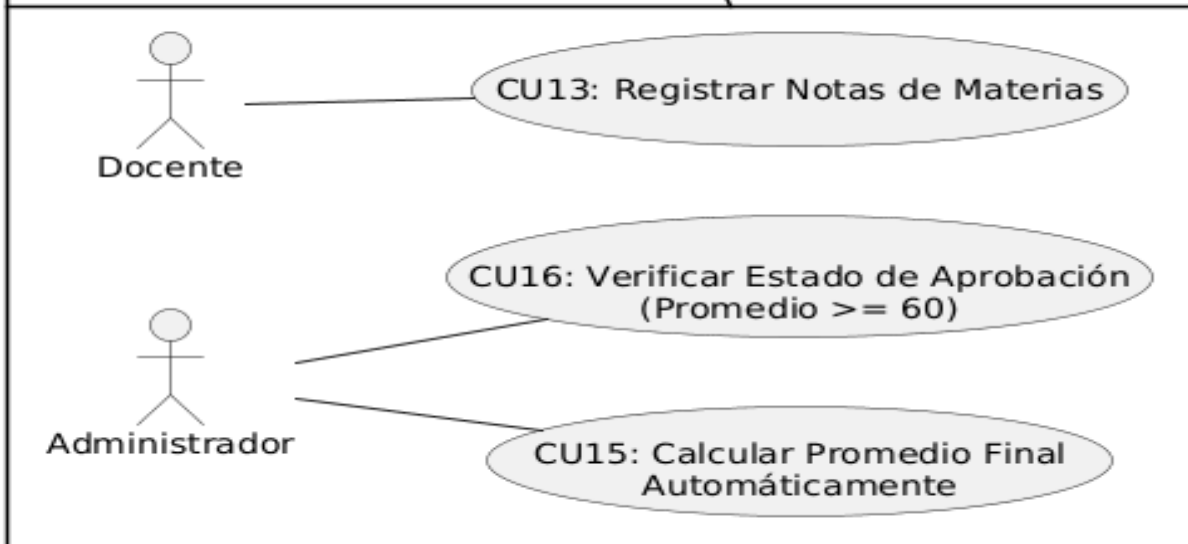
Ciclo #1



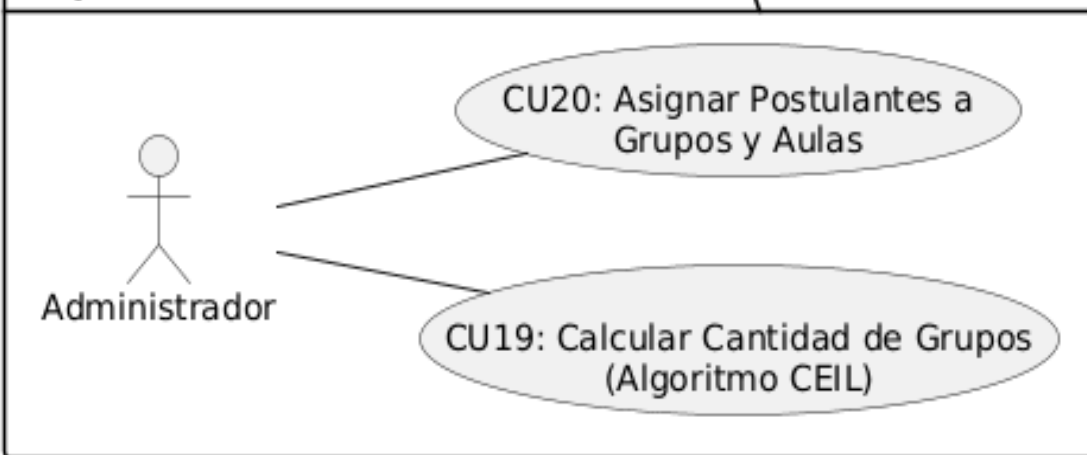
Paquete: REGISTRO Y ADMISIÓN



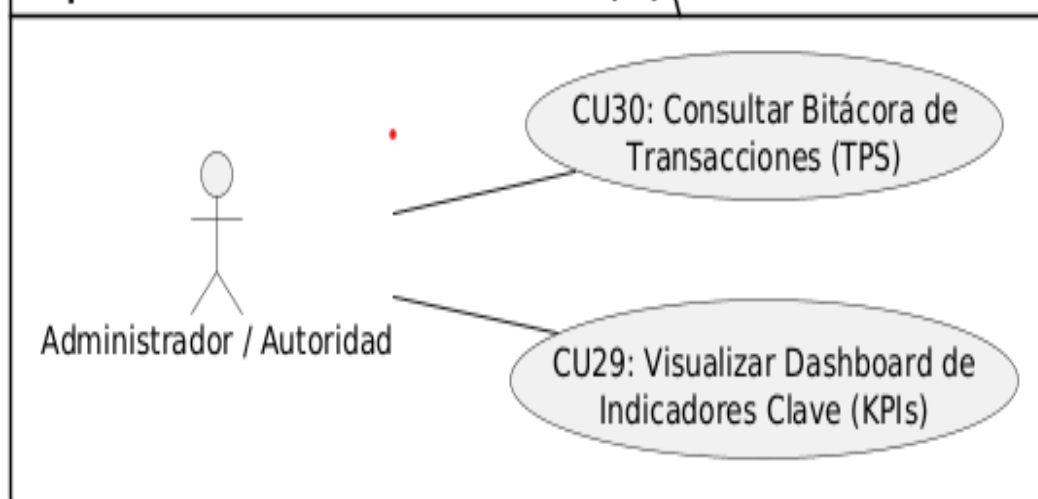
Paquete: EVALUACIÓN ACADÉMICA

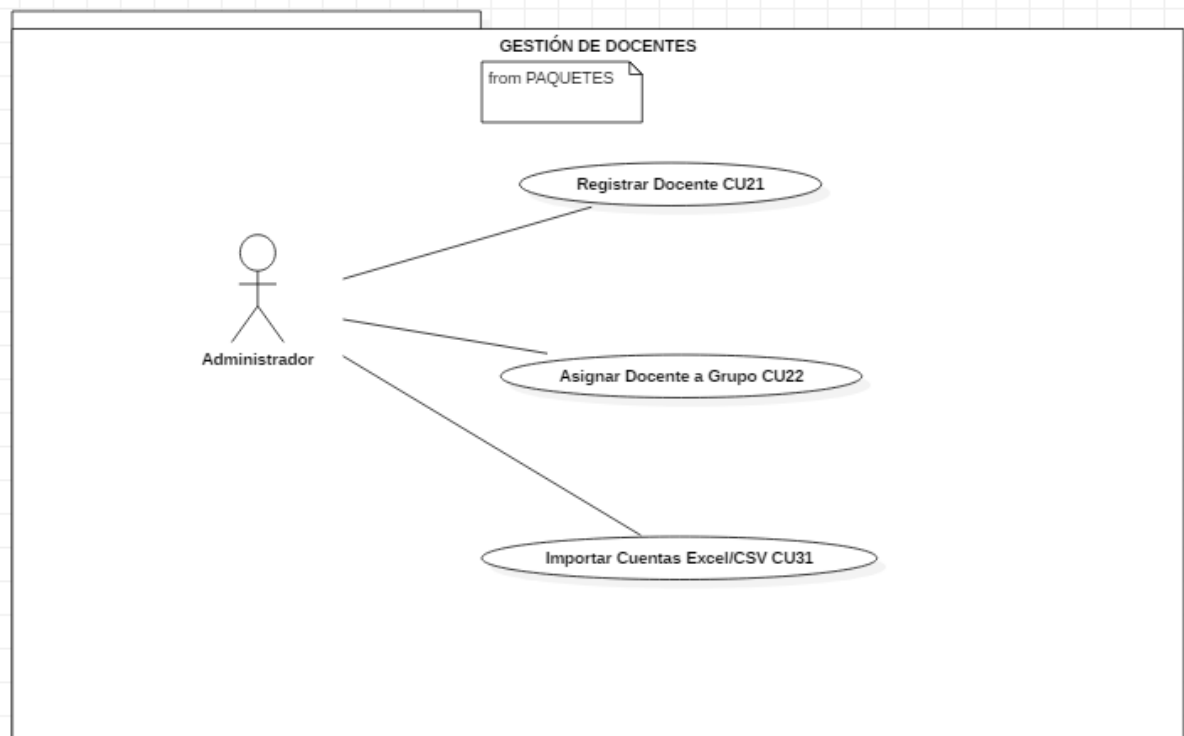


Paquete: LOGÍSTICA ORGANIZACIONAL



Paquete: INTELIGENCIA DE NEGOCIOS (BI)

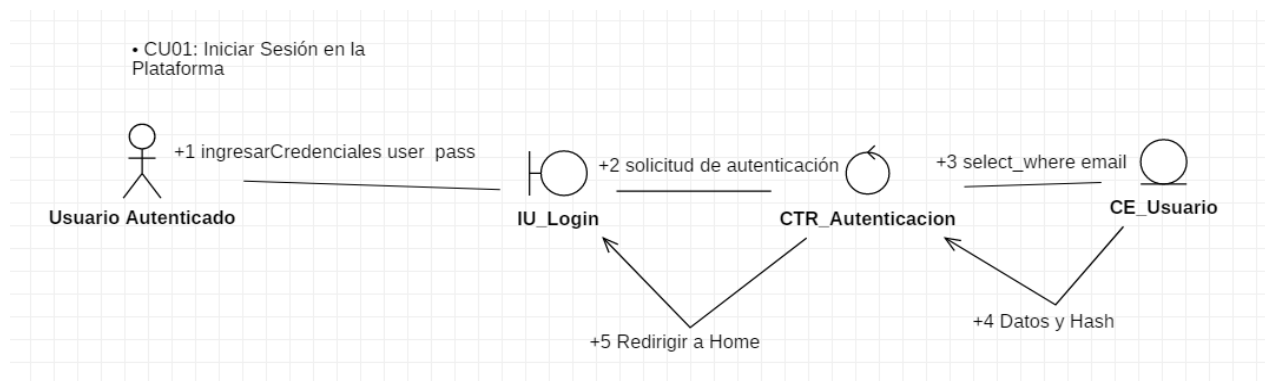




5.2 Diagrama de comunicación

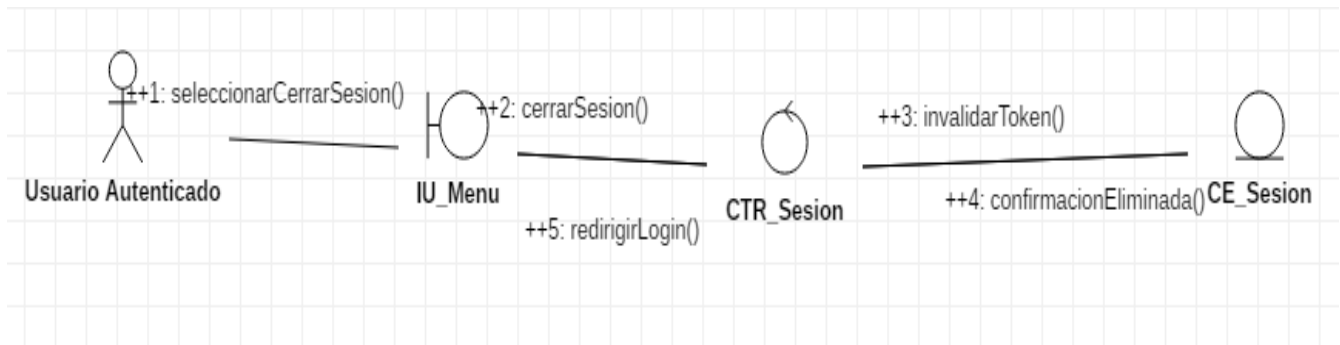
5.2.1 Ciclo #1

CU01: Iniciar sesión en plataforma



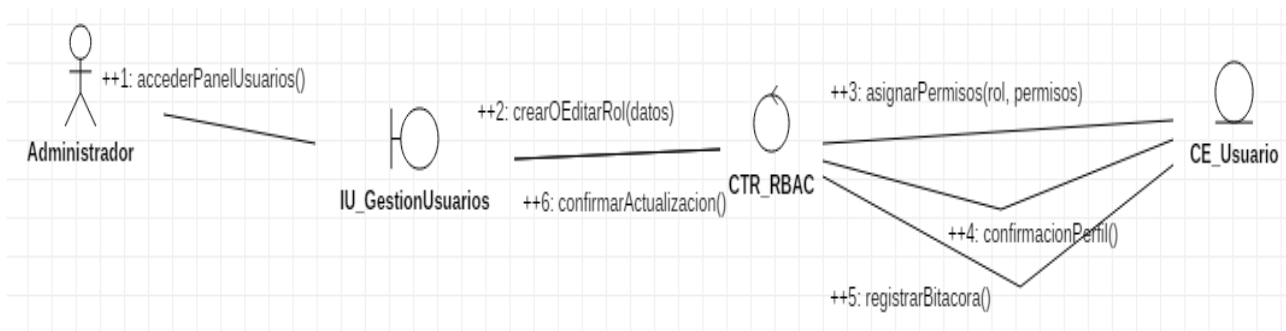
CU01: Iniciar Sesión en la Plataforma: El flujo comienza cuando el **Usuario** envía 1: ingresarCredenciales(user, pass) al objeto <<boundary>> IU_Login. Esta interfaz invoca 2: validarAcceso() en el <<control>> CTR_Auth, el cual ejecuta 3: buscarUsuario() en la <<entity>> CE_Usuario. Tras recibir 4: retornoHash(), el controlador verifica la clave, genera un **token JWT** y ordena 5: redirigirADashboard() a la interfaz.

CU02: Cerrar sesión



CU02: Cerrar Sesión Activa: El usuario solicita la salida desde el menú. El CTR_Auth destruye la sesión en el servidor, invalida el token y solicita a la interfaz limpiar el almacenamiento local y mostrar la pantalla de acceso.

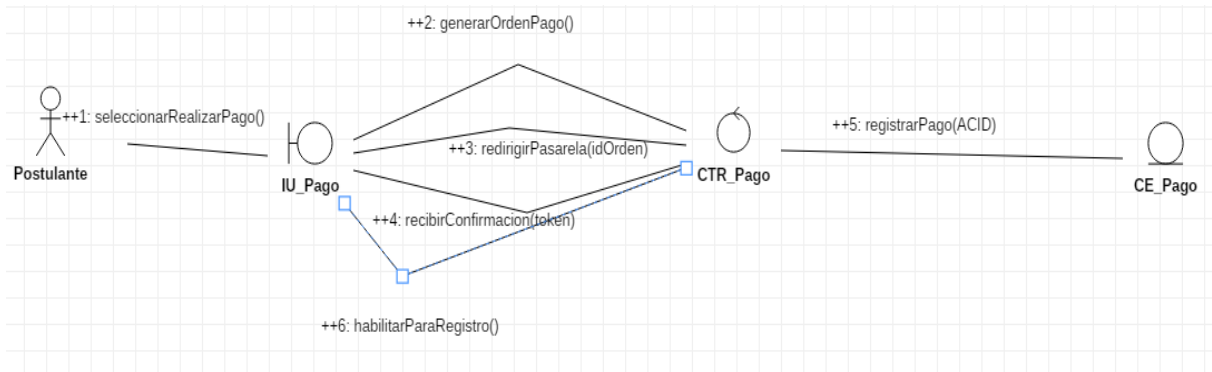
CU05: Gestionar Cuentas y Roles (RBAC)



CU05: Gestionar Cuentas y Roles (RBAC): El Administrador en IU_Usuarios envía parámetros de perfil al <<control>> CTR_Admin. Este interactúa con las entidades CE_Usuario y CE_Rol

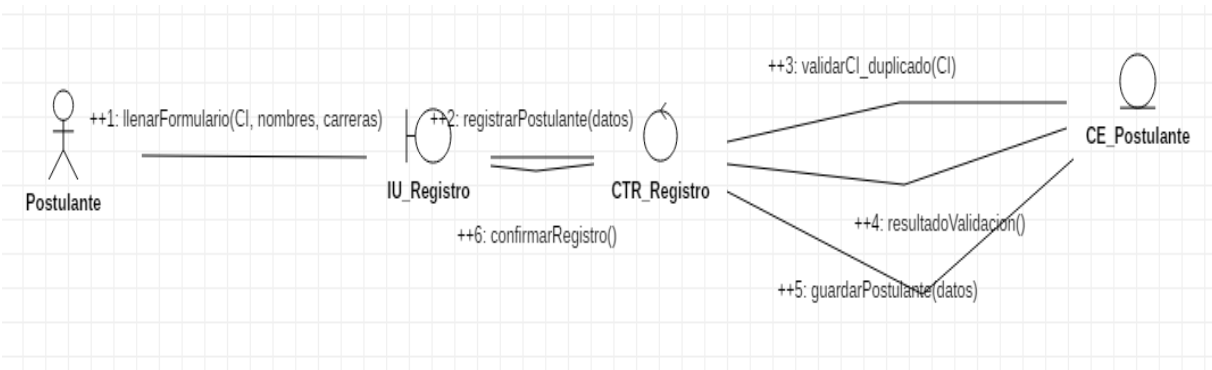
para actualizar privilegios, garantizando que el personal acceda solo a funciones autorizadas (ej. Docentes solo a sus notas).

CU07: Procesar Pago de Inscripción (Interacción con Pasarela)



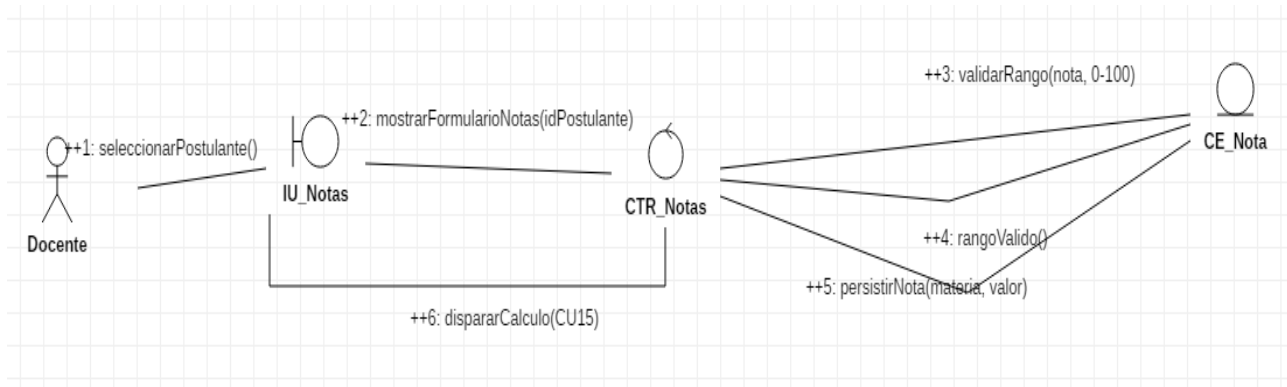
CU07: Procesar Pago de Inscripción: El **Postulante** activa la transacción en la IU_Pago. El <<control>> CTR_Transaccion crea una orden de pago y se comunica con el **Actor Externo Pasarela de Pagos**. Al recibir el mensaje de éxito, el controlador persiste el registro en CE_Pago y actualiza el estado del postulante a "Habilitado".

CU08: Registrar Nuevo Postulante (Datos personales y académicos)



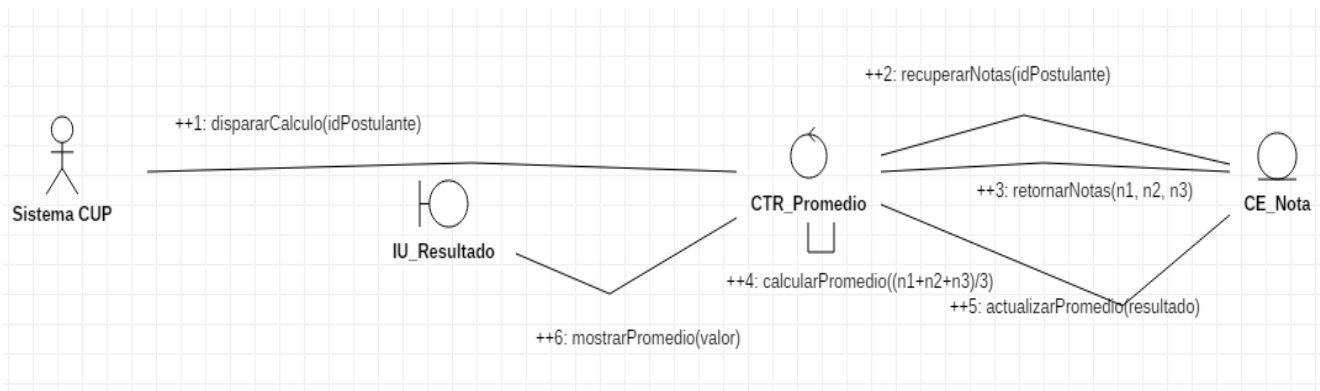
CU08: Registrar Nuevo Postulante: En la IU_Registro, se capturan los datos personales y académicos. El <<control>> CTR_Postulante valida la unicidad del **CI** en la entidad CE_Postulante. Si es correcto, guarda la información y vincula el registro con el pago previo, estableciendo la entidad base del sistema.

CU13: Registrar Notas de Materias (Computación, Matemáticas, Inglés, Física)



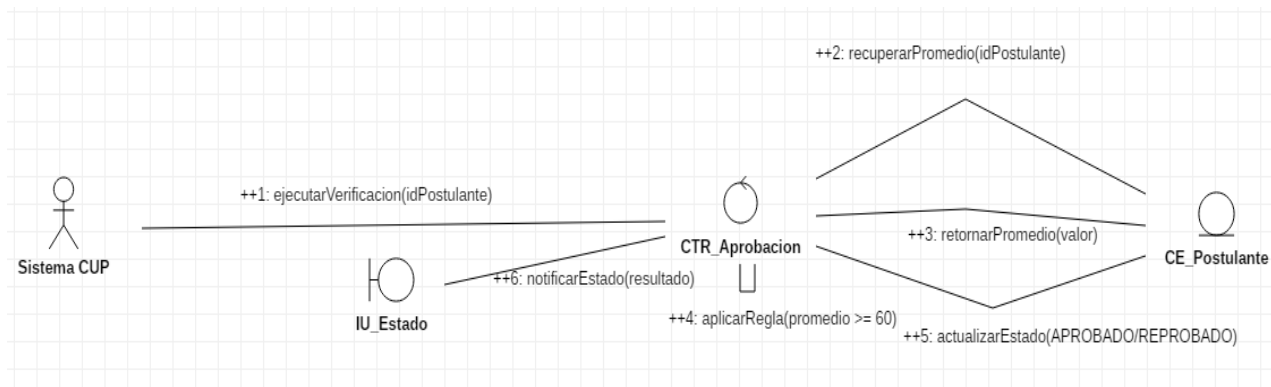
CU13: Registrar Notas de Materias: El **Docente** ingresa calificaciones en IU_Evaluacion. El <<control>> CTR_Academico valida que el rango sea 0-100 y persiste el dato en la entidad CE_Nota. Al terminar, dispara automáticamente el cálculo de promedios.

CU15: Calcular Promedio Final Automáticamente



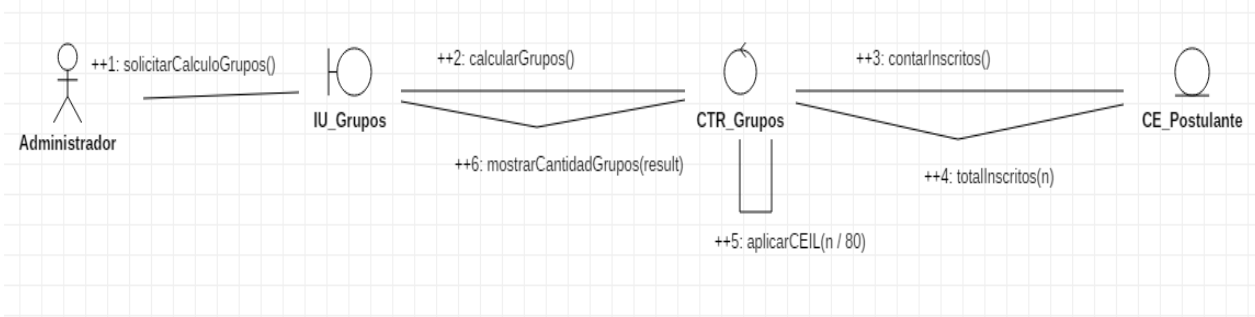
CU15: Calcular Promedio Final Automáticamente: Tras la tercera nota, el sistema activa el controlador para recuperar los valores de CE_Nota. El objeto aplica la fórmula $(N1+N2+N3)/3$ y actualiza el campo promedio_final en la entidad CE_Postulante.

CU16: Verificar Estado de Aprobación (Promedio ≥ 60)



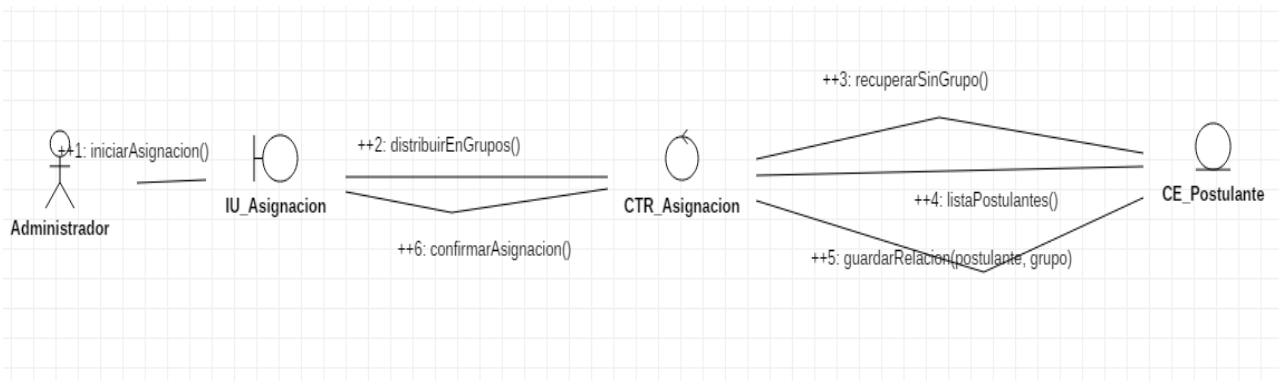
CU16: Verificar Estado de Aprobación: El controlador académico lee el promedio final de CE_Postulante. Aplica la regla: **si promedio ≥ 60 , entonces estado = 'APROBADO'**. El resultado se persiste en el registro del estudiante.

CU19: Calcular Cantidad de Grupos Necesarios (Algoritmo CEIL)



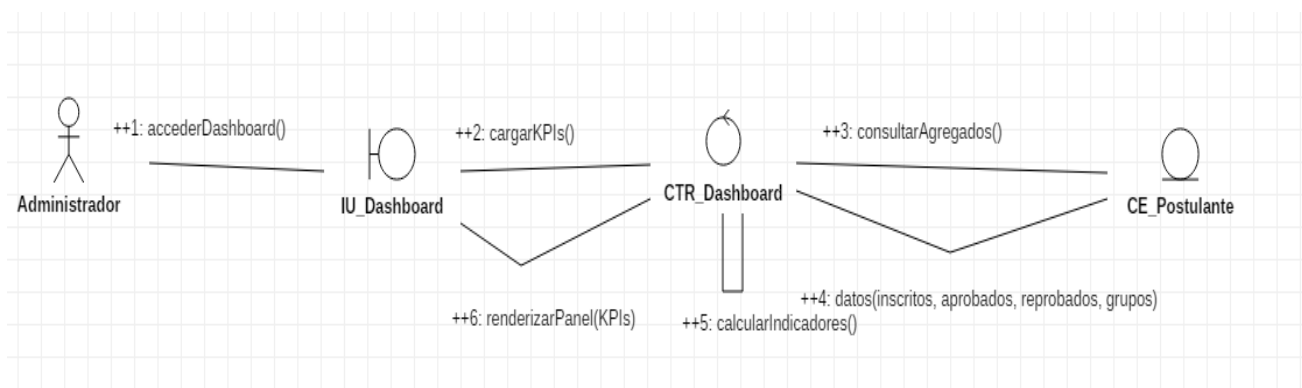
CU19: Calcular Cantidad de Grupos Necesarios: El Administrador solicita el cálculo en IU_Logística. El <<control>> CTR_Distribucion cuenta los inscritos en CE_Postulante, aplica el **Algoritmo CEIL (Total/80)** y crea masivamente las entidades CE_Grupo necesarias.

CU20: Asignar Postulantes a Grupos y Aulas



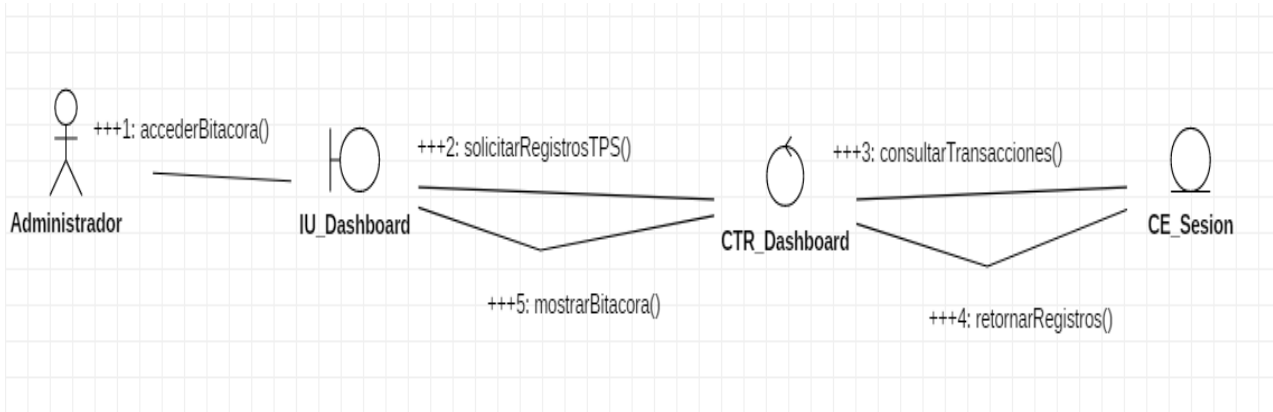
CU20: Asignar Postulantes a Grupos y Aulas: El sistema activa el controlador para recuperar estudiantes sin aula. El controlador recorre los grupos disponibles, valida el límite de **70 alumnos** por aula y persiste la asignación en la entidad CE_Postulante_Grupo.

CU29: Visualizar Dashboard de Indicadores Clave (KPIs)



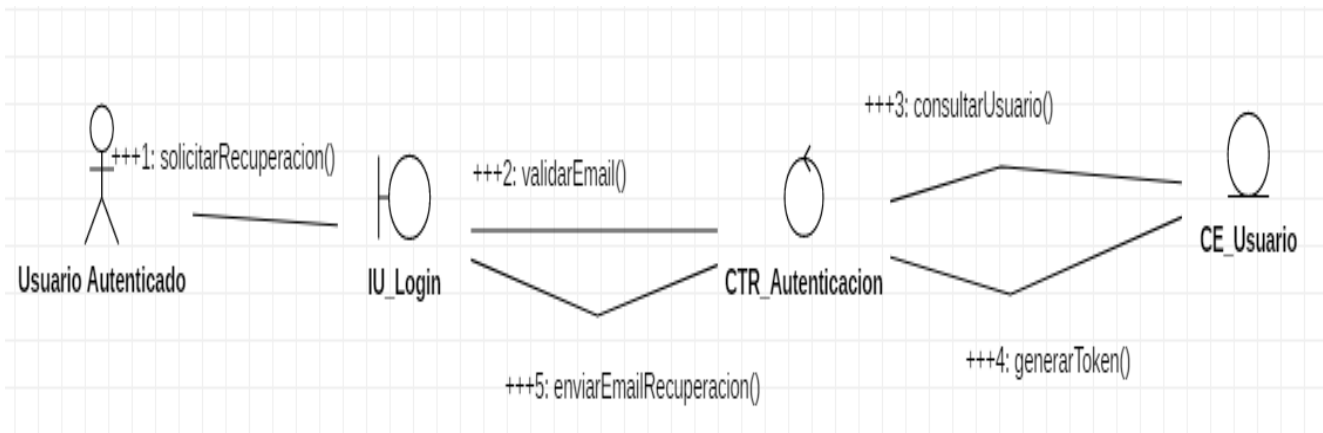
CU29: Visualizar Dashboard de Indicadores (KPIs): Al ingresar a la IU_Dashboard, el <<control>> CTR_BI realiza consultas agregadas (conteo, sumatoria, promedios) sobre las entidades del sistema. Procesa los datos en tiempo real y los envía a la interfaz para generar gráficos de rendimiento y estados de admisión.

CU30: Consultar Bitácora de Auditoría del Sistema (TPS)



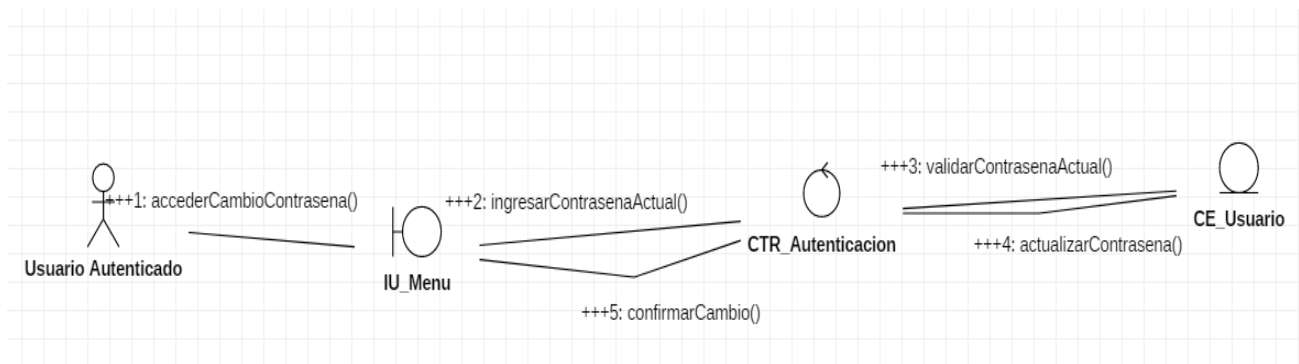
CU30: Consultar Bitácora de Transacciones (TPS): El Administrador accede a IU_Auditoria. El <<control>> CTR_Audit recupera los registros históricos de la entidad CE_TPS, permitiendo visualizar quién, cuándo y qué transacción realizó para auditoría interna.

CU03: Recuperar Contraseña de Usuario



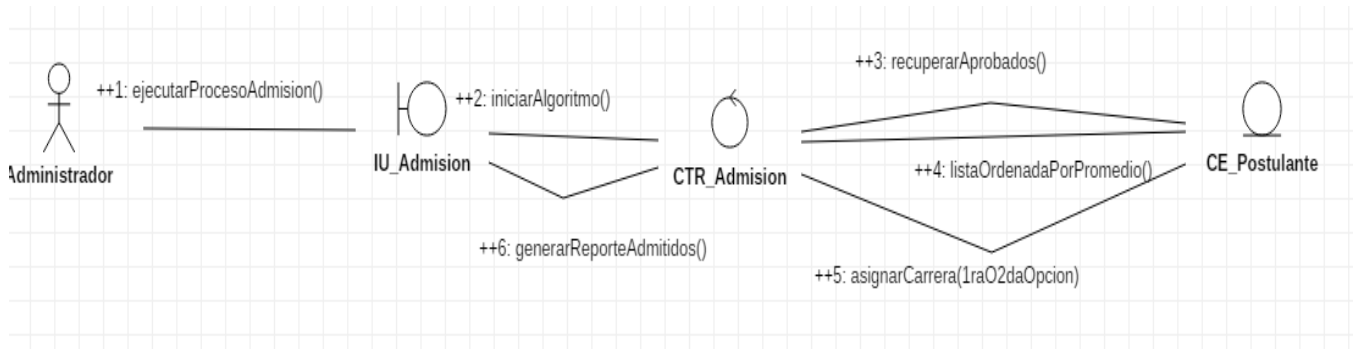
CU03: Recuperar Contraseña de Usuario: La IU_Login recibe el correo y lo envía al CTR_Auth, el cual valida la existencia en CE_Usuario. Si es válido, genera un enlace temporal único y activa el envío de un correo electrónico externo.

CU04: Cambiar Contraseña Propia



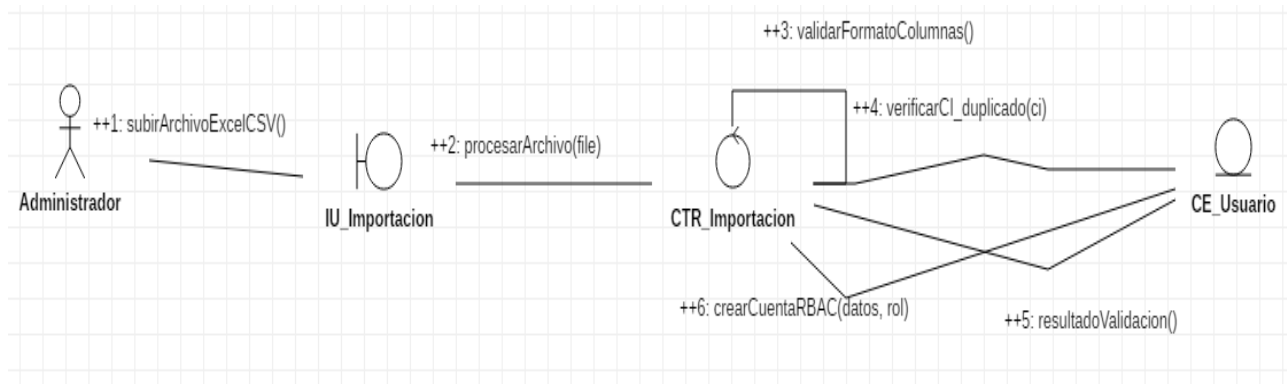
CU04: Cambiar Contraseña Propia: Desde el perfil de usuario, el CTR_Auth recibe la clave antigua y la nueva. Valida el hash actual en la entidad y, tras confirmar, sobrescribe el registro con la nueva clave encriptada

CU17: Ejecutar Algoritmo de Admisión por Cupos (1ra y 2da opción)



CU17: Ejecutar Algoritmo de Admisión por Cupos: El Administrador activa el proceso desde IU_Admision. El controlador ordena a los postulantes por mérito académico, recorre los cupos disponibles por carrera y asigna la 1ra o 2da opción, actualizando la relación en la base de datos.

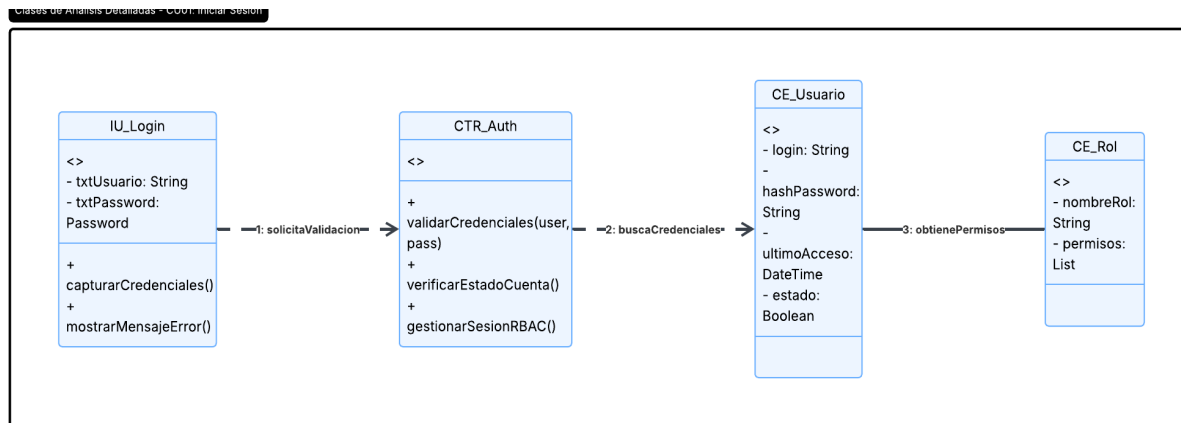
CU31: Importar Cuentas de Usuario desde Excel/CSV (Carga Masiva)



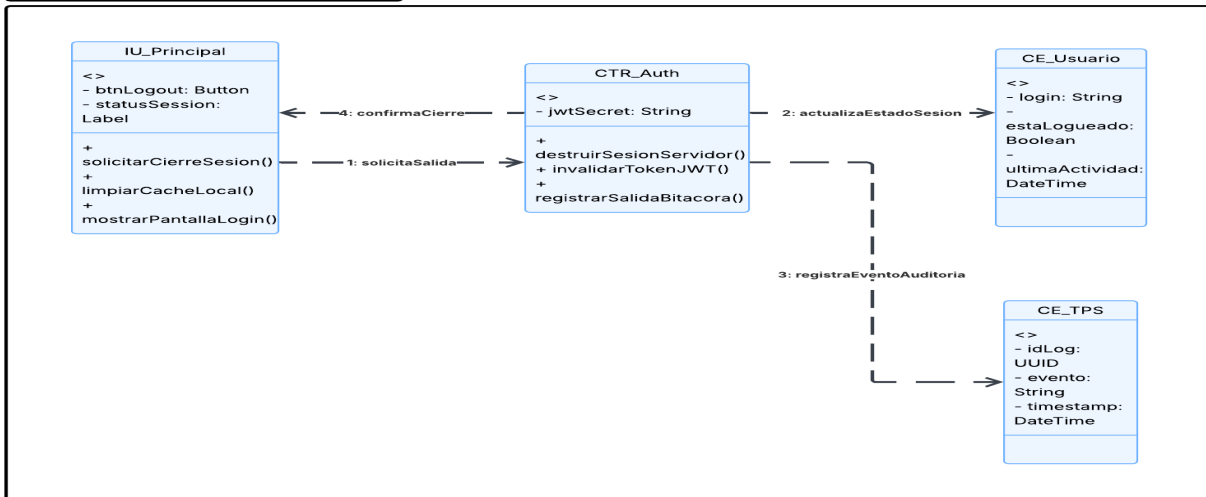
CU31: Importar Cuentas desde Excel/CSV: El Administrador carga un archivo en IU_Importacion. El <<control>> CTR_Usuario procesa los registros masivos, valida duplicados contra la base de datos y crea automáticamente las cuentas de acceso en CE_Usuario asignando los perfiles correspondientes.

5.3 Análisis de clases

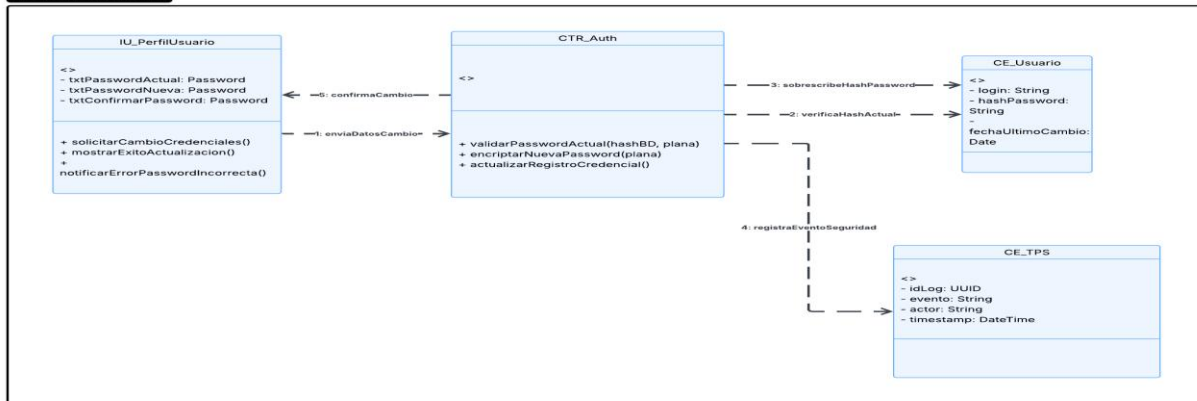
5.3.1 Ciclo #1



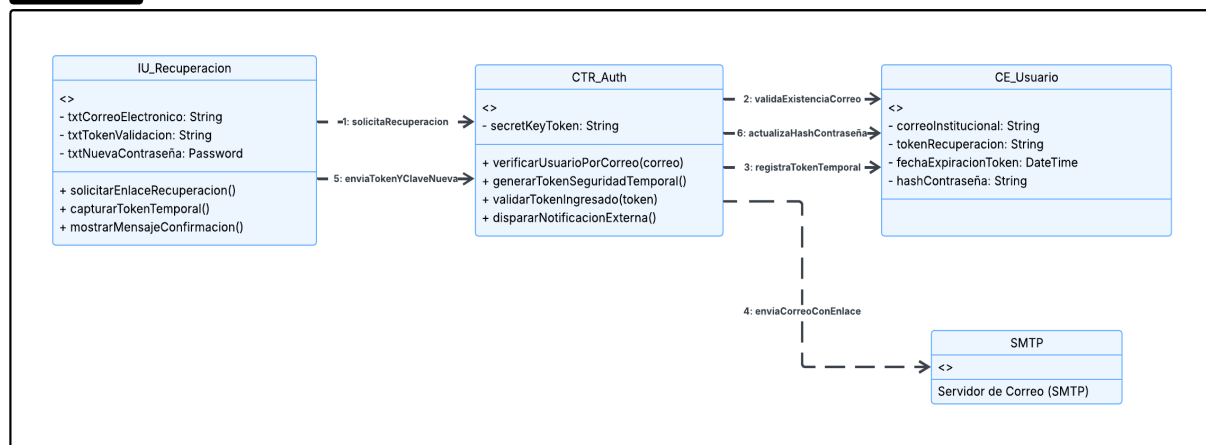
Clases de Análisis Detalladas - CU02: Cerrar Sesión Activa



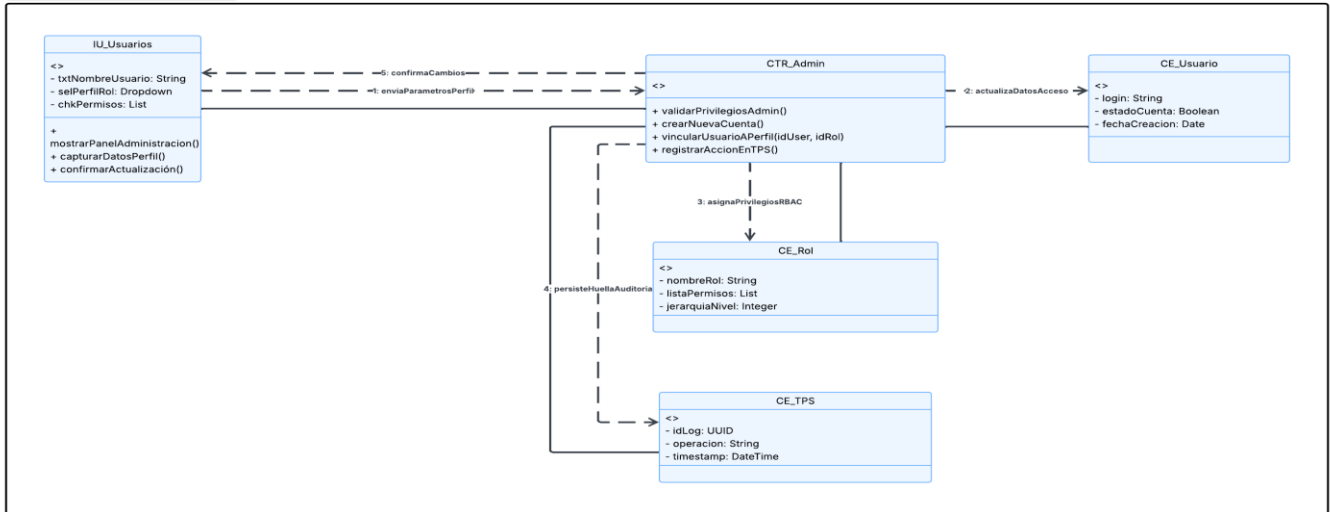
CU04: Cambiar Contraseña Propia



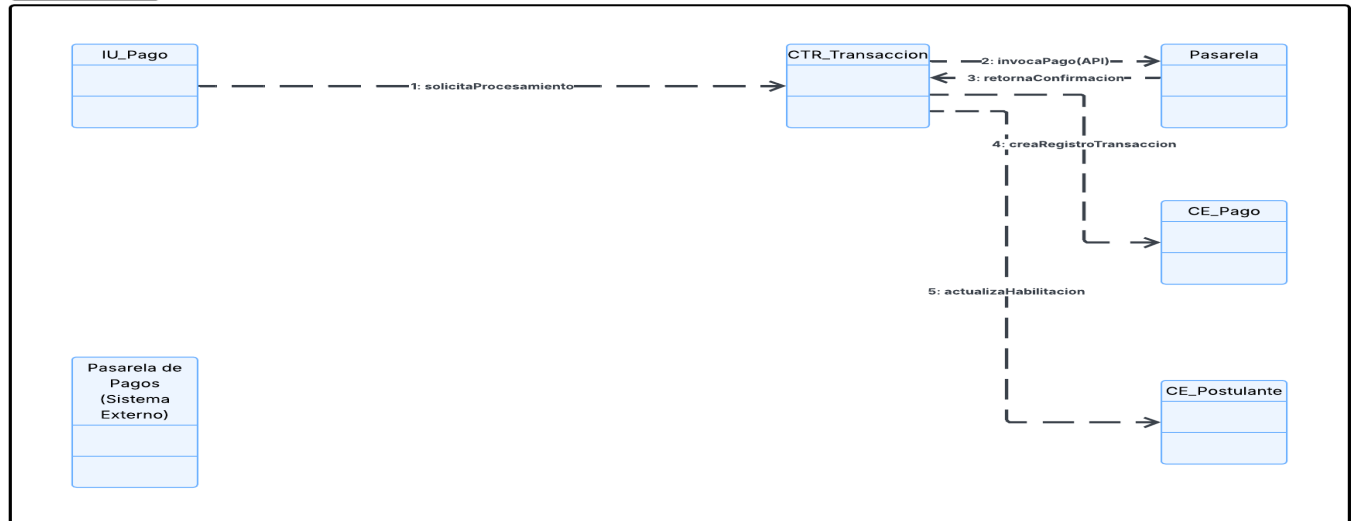
CU03: Recuperar Contraseña



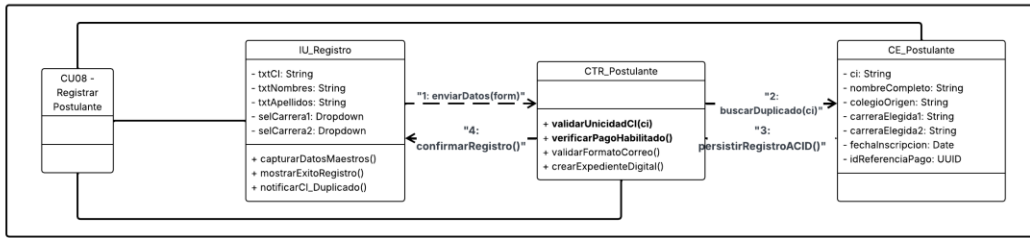
3.3 Realización de Análisis: CU05 - Gestionar Roles (RBAC)



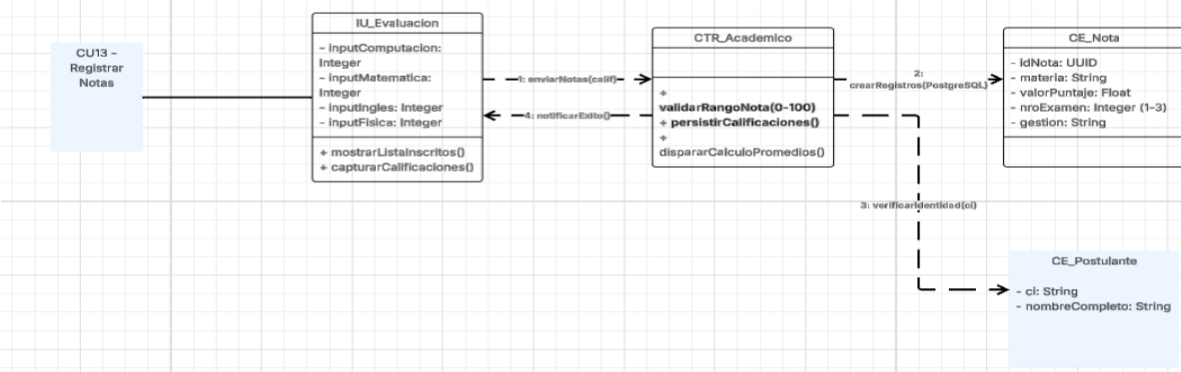
CU07 - Procesar Pago



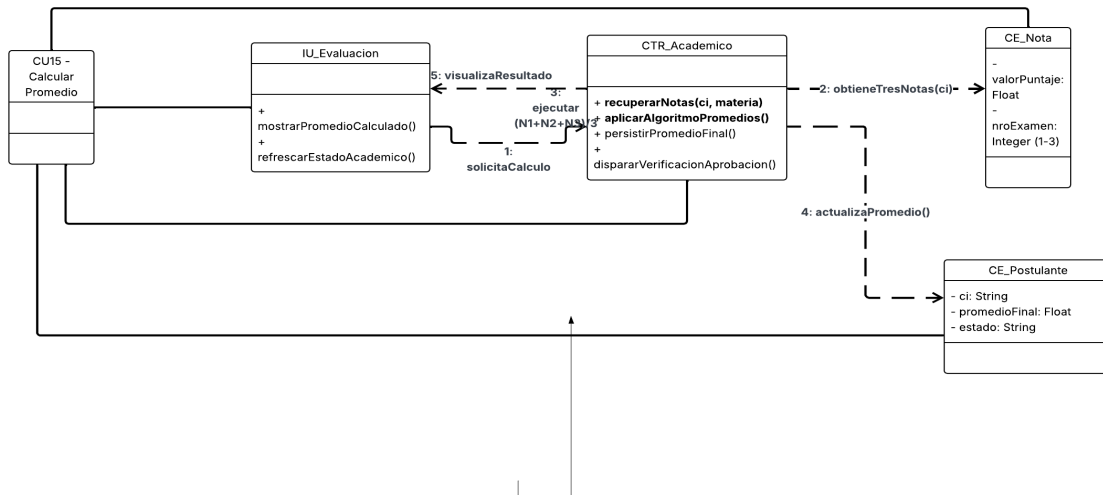
CU08 - Registrar Postulante



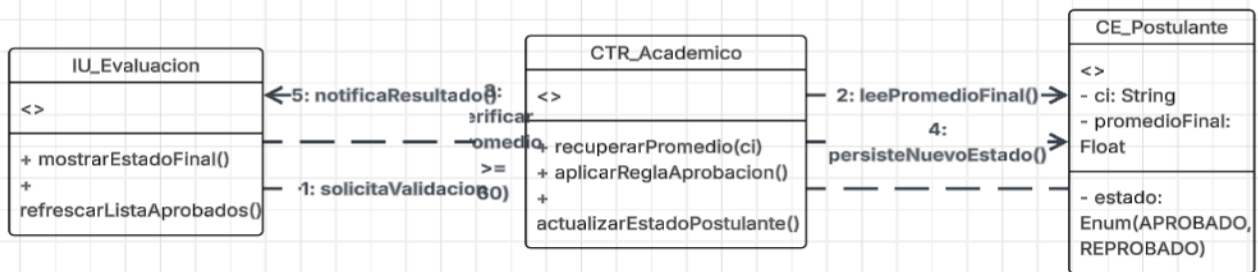
CU13 - Registrar Notas



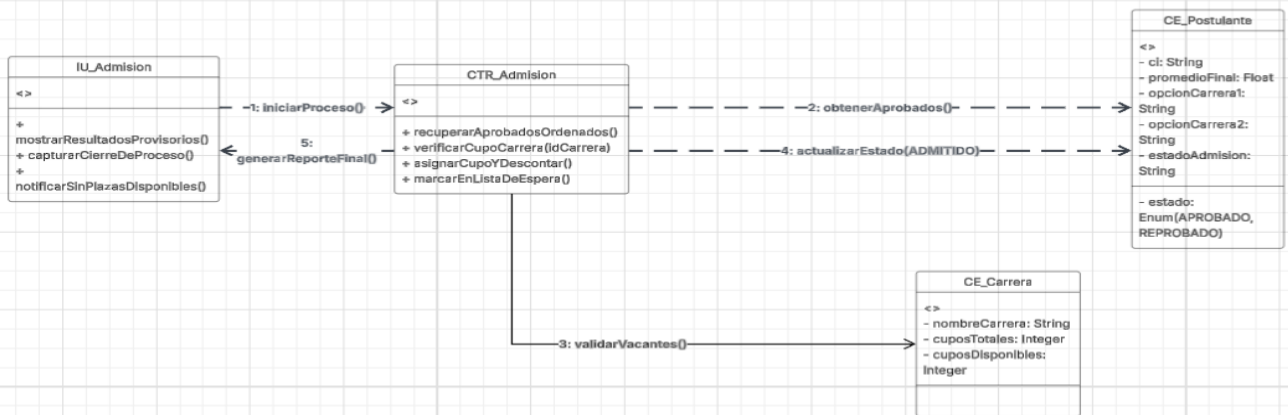
CU15 - Calcular Promedio



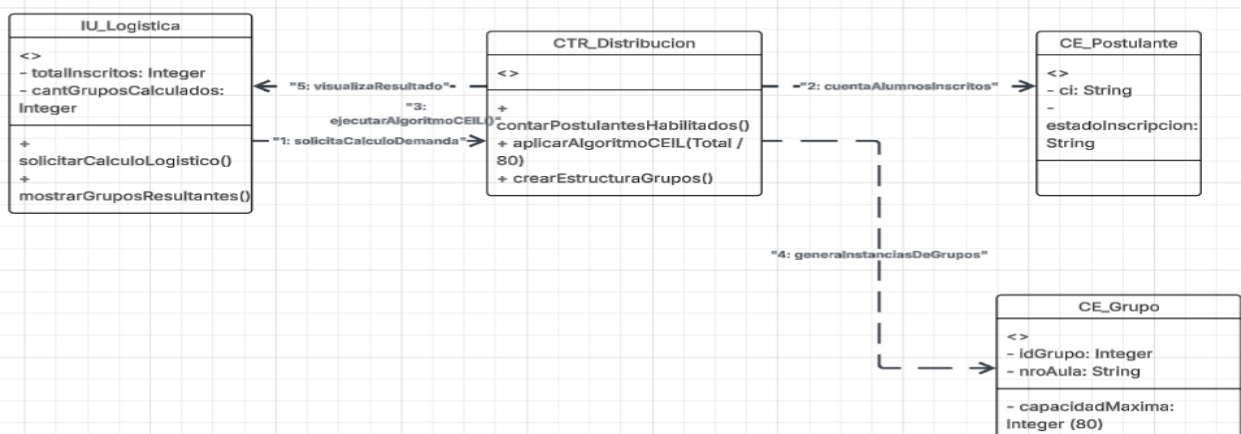
CU16 - Verificar Aprobación



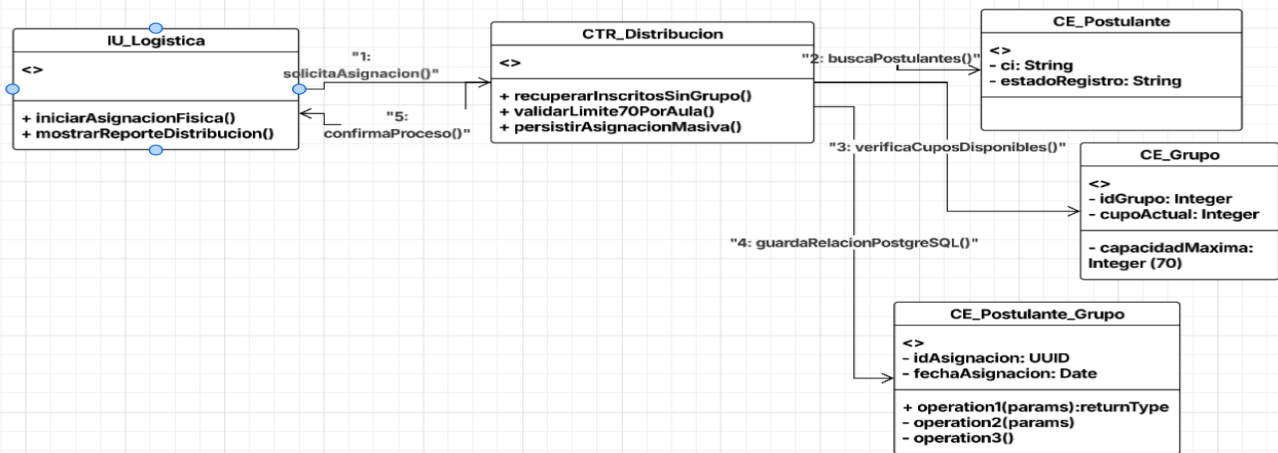
Realización: CU17 - Algoritmo de Admisión



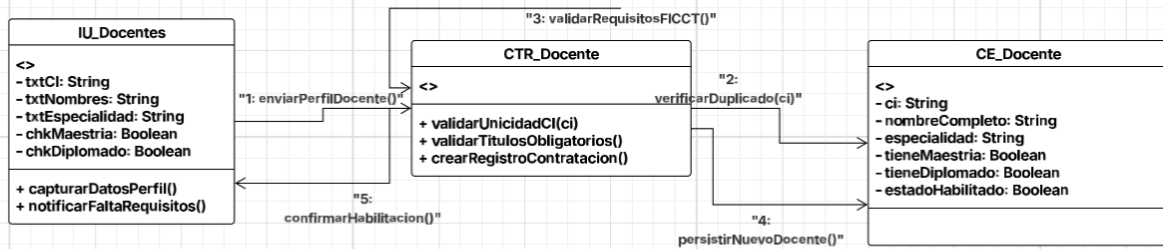
Realización: CU19 - Algoritmo Logístico CEIL



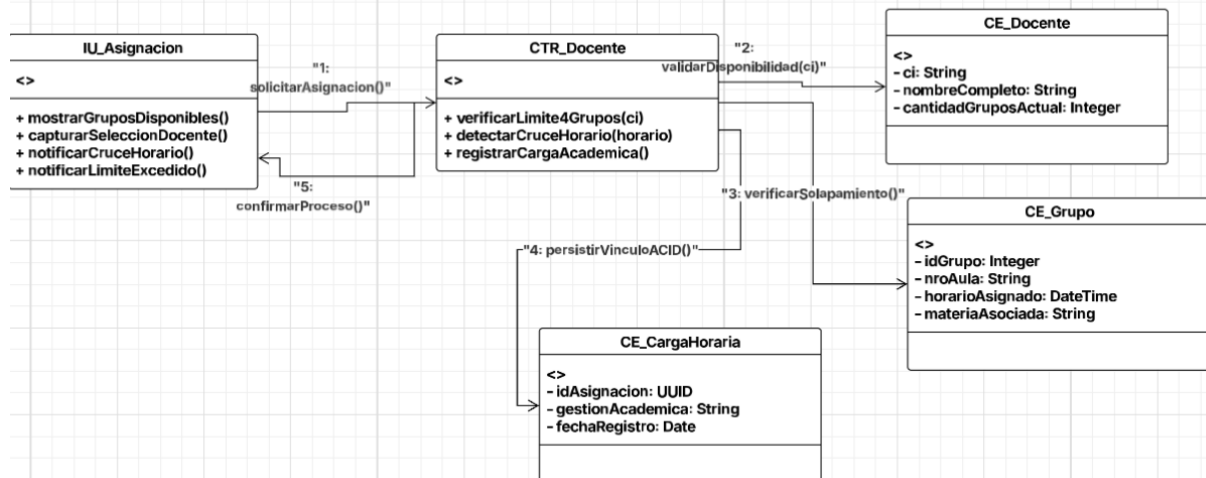
Realización: CU20 - Asignar Grupos



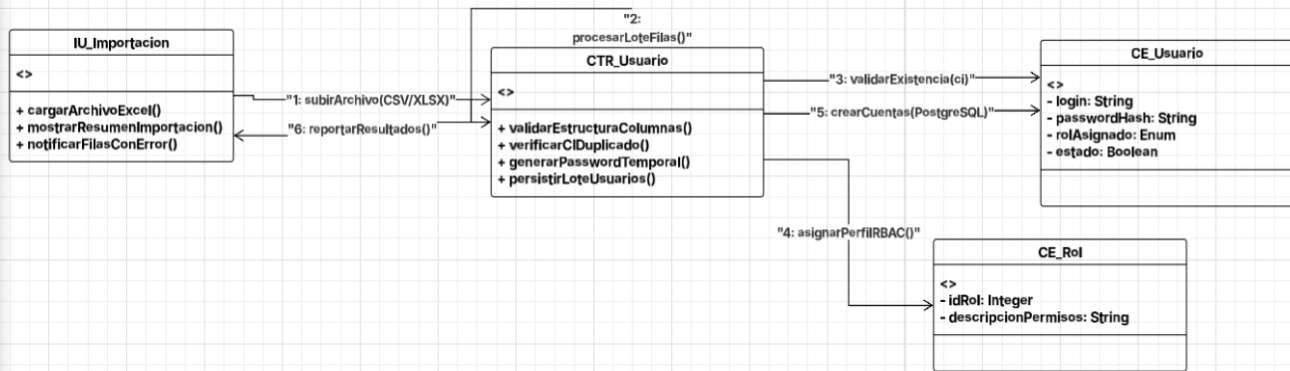
Realización: CU21 - Registro de Docentes



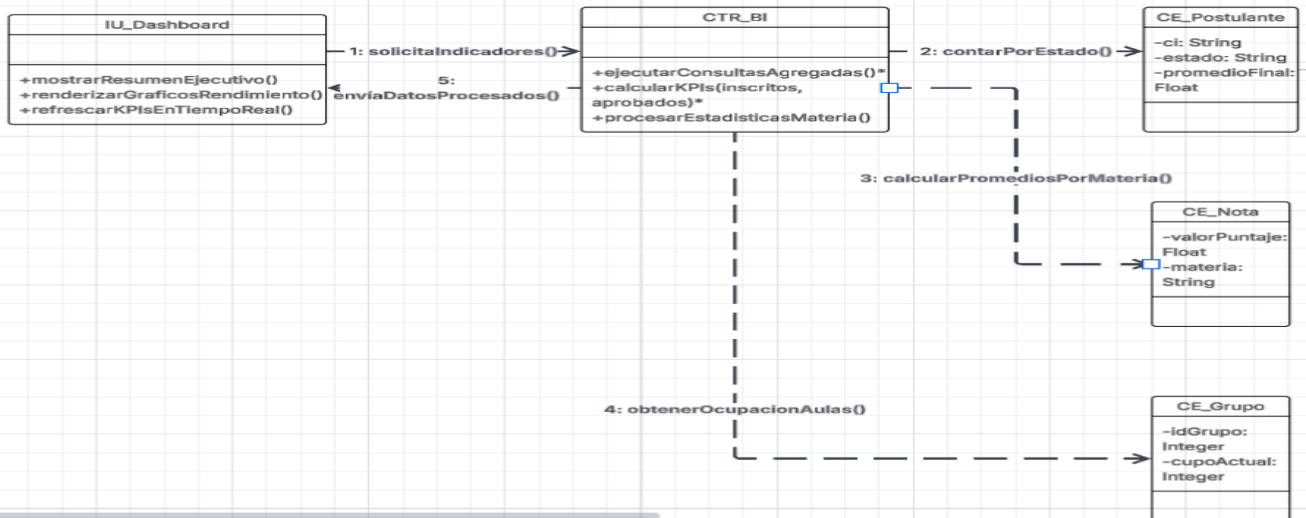
Realización: CU22 - Asignar Carga Horaria



Realización: CU31 - Importación Masiva

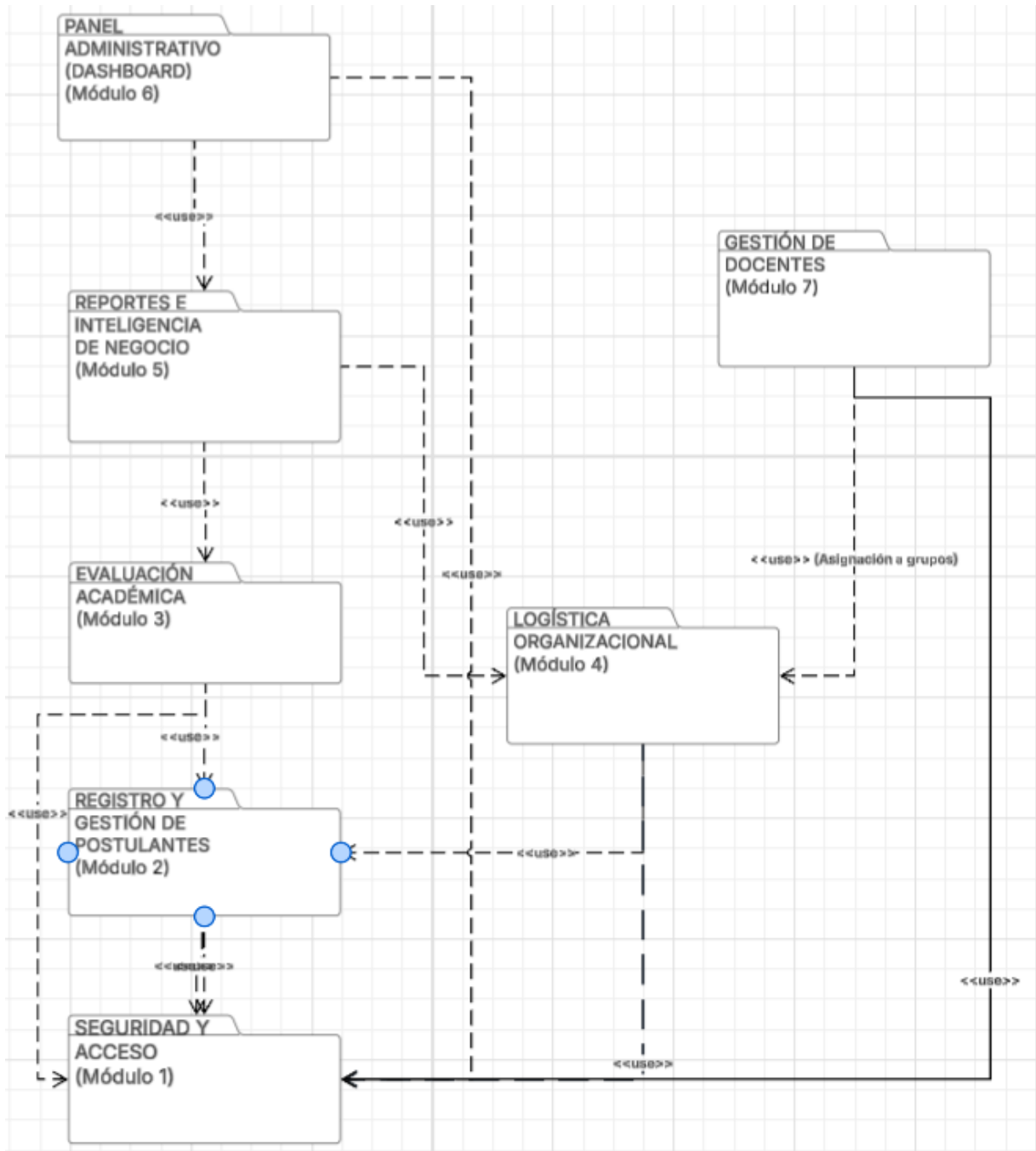


CU29 - Dashboard de KPIs



5.4 Analizar paquetes

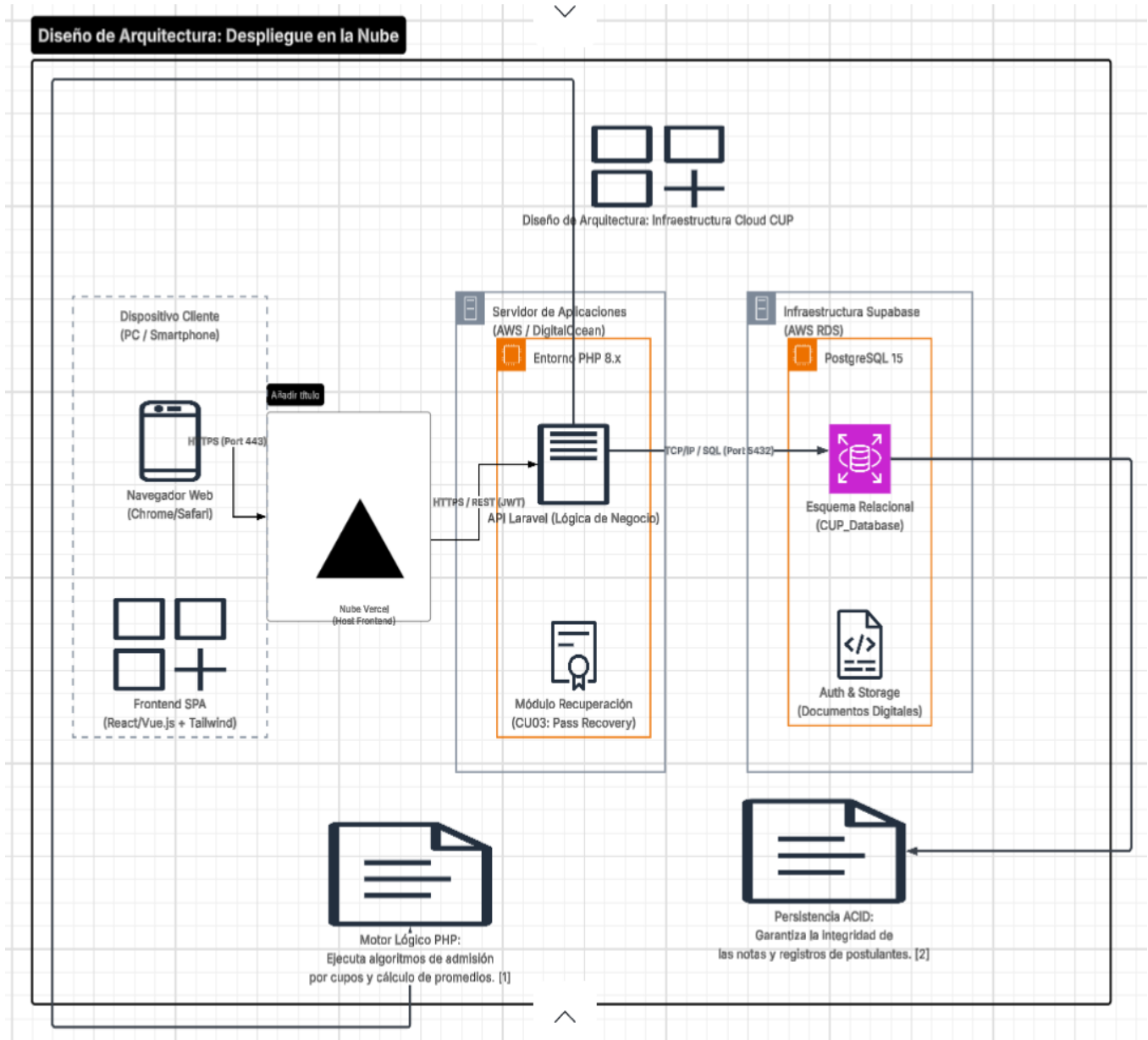
5.4.1 Ciclo #1



6 FLUJO DE TRABAJO: DISEÑO

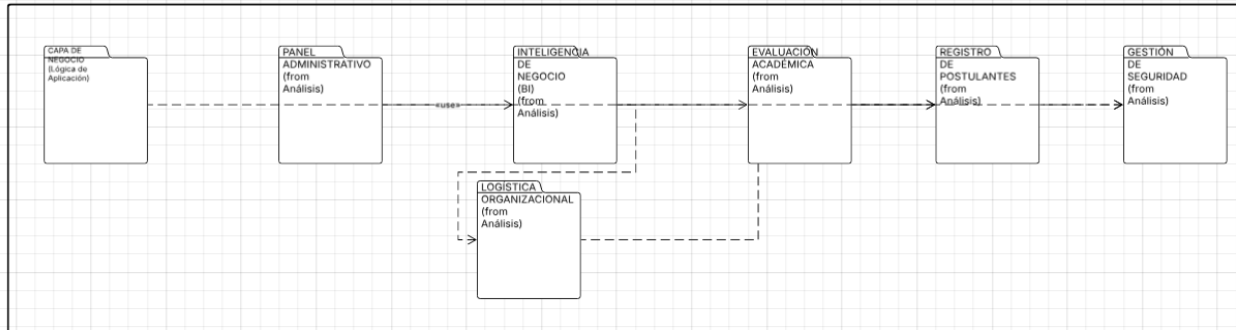
6.1 Diseño de arquitectura

6.1.1 Diseño físico (diagrama de despliegue)

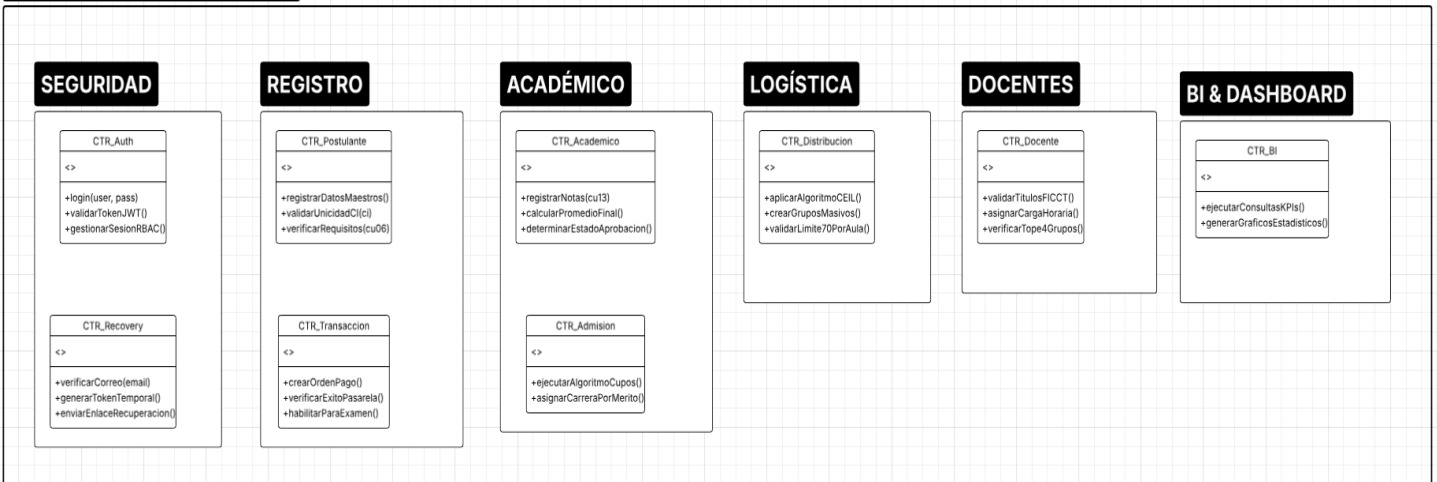


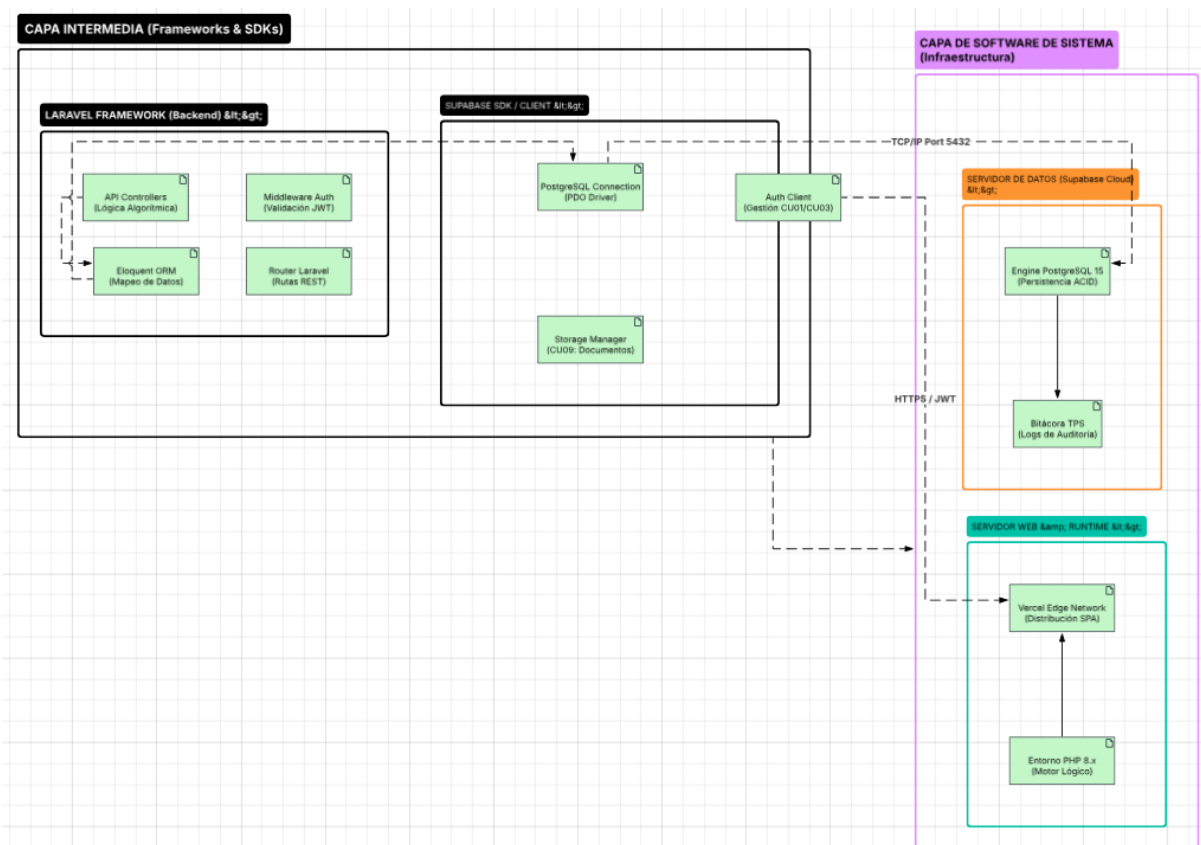
6.1.2 Diseño lógico (diagrama organizado en capas)

Capa de Negocio



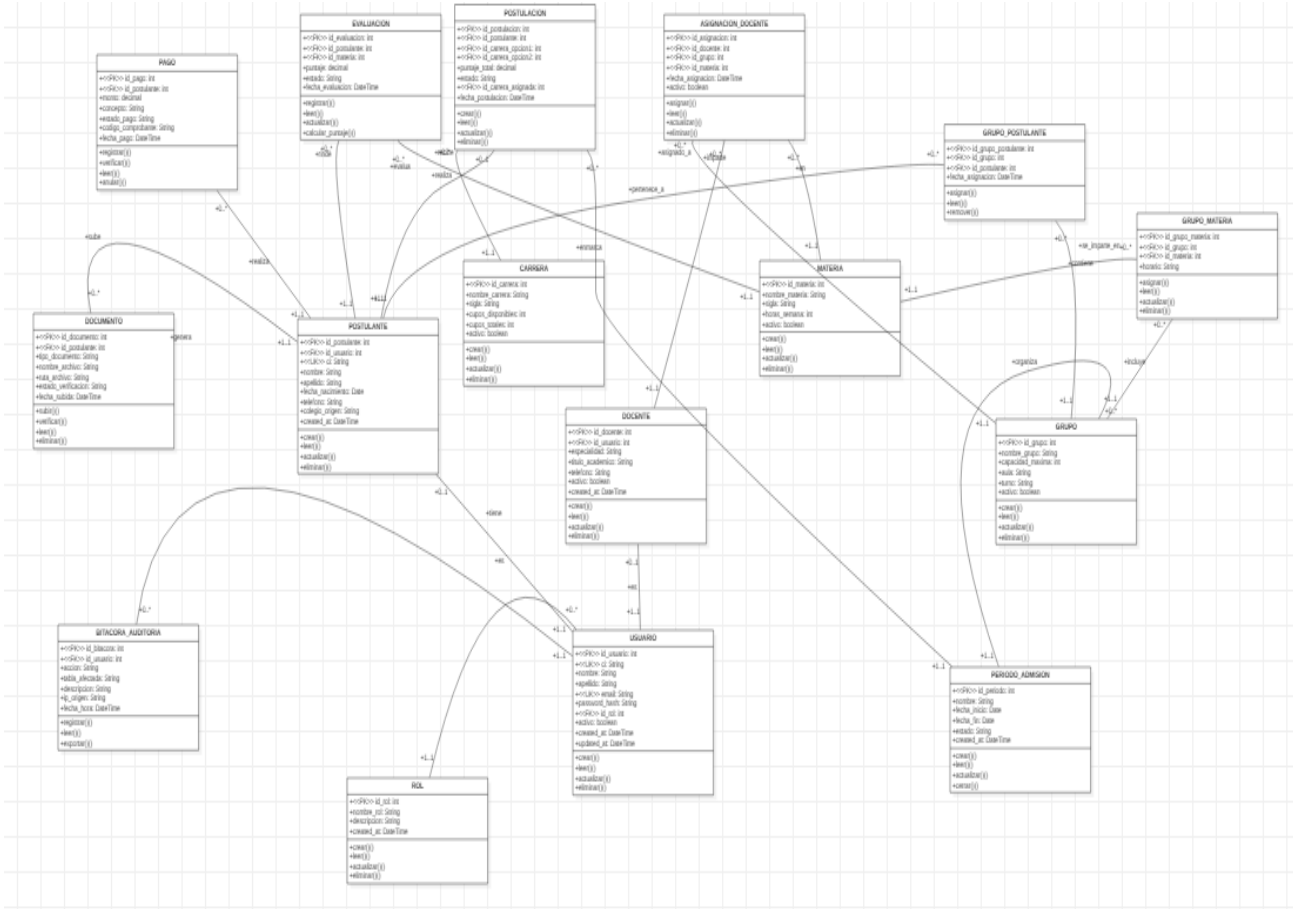
Capa de Controladores: Lógica de Procesos CUP





6.2 Diseño de datos

6.2.1 Diseño de datos lógico



Mapeo y normalización:

6.2.2 Diseño de datos físico

```
--  
=====
```

-- DISEÑO FÍSICO — Sistema CUP (FICCT - UAGRM)

-- Motor: PostgreSQL 15+

-- Gestión: 1-2026

--

=====

--

=====

-- EXTENSIONES

--

=====

CREATE EXTENSION IF NOT EXISTS "pgcrypto"; -- gen_random_uuid(), crypt()

CREATE EXTENSION IF NOT EXISTS "pg_trgm"; -- búsqueda full-text en nombres

--

=====

-- 1. ROL

--

=====

CREATE TABLE rol (

```

id_rol    SERIAL    PRIMARY KEY,

nombre_rol VARCHAR(50) NOT NULL UNIQUE,

descripcion VARCHAR(255),

created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()

);

```

```

COMMENT ON TABLE rol          IS 'Roles RBAC del sistema (Administrador, Docente,
Postulante).';

```

```

COMMENT ON COLUMN rol.nombre_rol IS 'Nombre único del rol, p.ej. ADMIN, DOCENTE,
POSTULANTE.';

```

```

--
=====

```

```

-- 2. USUARIO

```

```

--
=====

```

```

CREATE TABLE usuario (

```

```

    id_usuario SERIAL    PRIMARY KEY,

    ci          VARCHAR(20) NOT NULL UNIQUE,

    nombre      VARCHAR(100) NOT NULL,

    apellido    VARCHAR(100) NOT NULL,

    email       VARCHAR(150) NOT NULL UNIQUE,

    password_hash VARCHAR(255) NOT NULL,

    id_rol      INT        NOT NULL REFERENCES rol(id_rol)

```

```

        ON UPDATE CASCADE ON DELETE RESTRICT,

    activo      BOOLEAN      NOT NULL DEFAULT TRUE,

    created_at   TIMESTAMPTZ   NOT NULL DEFAULT NOW(),

    updated_at   TIMESTAMPTZ   NOT NULL DEFAULT NOW()

);

CREATE INDEX idx_usuario_rol    ON usuario(id_rol);

CREATE INDEX idx_usuario_email  ON usuario(email);

COMMENT ON TABLE  usuario      IS 'Cuenta de acceso al sistema. Cada persona (docente
o postulante) tiene exactamente un usuario.';

COMMENT ON COLUMN  usuario.ci    IS 'Cédula de identidad boliviana, sin extensión
departamental.';

COMMENT ON COLUMN  usuario.password_hash IS 'Hash bcrypt generado con
pgcrypto.crypt(). Nunca almacenar texto plano.';

COMMENT ON COLUMN  usuario.id_rol    IS 'FK al rol RBAC asignado.';

--
=====

-- 3. BITACORA_AUDITORIA

--
=====

CREATE TABLE bitacora_auditoria (

    id_bitacora    BIGSERIAL    PRIMARY KEY,

```

```

id_usuario    INT            REFERENCES usuario(id_usuario)
              ON UPDATE CASCADE ON DELETE SET NULL,
accion        VARCHAR(50)    NOT NULL
              CHECK          (accion IN
('LOGIN','LOGOUT','INSERT','UPDATE','DELETE','EXPORT')),
tabla_afectada VARCHAR(100),
descripcion   TEXT,
ip_origen     INET,
fecha_hora    TIMESTAMPTZ    NOT NULL DEFAULT NOW()
);

```

```

CREATE INDEX idx_bitacora_usuario  ON bitacora_auditoria(id_usuario);
CREATE INDEX idx_bitacora_fecha    ON bitacora_auditoria(fecha_hora DESC);

```

```

COMMENT ON TABLE bitacora_auditoria IS 'Registro inmutable de todas las acciones
relevantes del sistema (trazabilidad RBAC).';

```

```

COMMENT ON COLUMN bitacora_auditoria.ip_origen IS 'Tipo INET de PostgreSQL; soporta
IPv4 e IPv6 sin conversión.';

```

```

COMMENT ON COLUMN bitacora_auditoria.accion IS 'Conjunto cerrado de acciones
auditables.';

```

```
--
```

```
=====
```

```
-- 4. PARAMETRO_SISTEMA (no modelado en MDJ; inferido del doc)
```



```
--
=====

CREATE TABLE parametro_sistema (

    id_parametro  SERIAL      PRIMARY KEY,

    clave        VARCHAR(100) NOT NULL UNIQUE,

    valor        TEXT        NOT NULL,

    descripcion   VARCHAR(255),

    updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()

);

COMMENT ON TABLE parametro_sistema IS 'Parámetros configurables en caliente:
cupo_min_grupo, nota_aprobacion, etc.';
```

```
--
=====

-- 5. PERIODO_ADMISION

--
=====

CREATE TABLE periodo_admision (

    id_periodo  SERIAL      PRIMARY KEY,

    nombre     VARCHAR(100) NOT NULL,

    fecha_inicio DATE       NOT NULL,

    fecha_fin   DATE       NOT NULL,

    estado     VARCHAR(20)  NOT NULL DEFAULT 'PLANIFICADO'
```

```

        CHECK (estado IN ('PLANIFICADO','ACTIVO','CERRADO')),

created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

CONSTRAINT ck_periodo_fechas CHECK (fecha_fin > fecha_inicio)

);

COMMENT ON TABLE periodo_admision IS 'Convocatoria académica del CUP. Solo puede
existir un período ACTIVO a la vez (controlado en aplicación).';

COMMENT ON COLUMN periodo_admision.estado IS 'Ciclo de vida del período.';

--
=====

-- 6. CARRERA

--
=====

CREATE TABLE carrera (

    id_carrera SERIAL PRIMARY KEY,

    nombre_carrera VARCHAR(150) NOT NULL,

    sigla VARCHAR(10) NOT NULL UNIQUE,

    cupos_totales INT NOT NULL CHECK (cupos_totales > 0),

    cupos_disponibles INT NOT NULL CHECK (cupos_disponibles >= 0),

    activo BOOLEAN NOT NULL DEFAULT TRUE,

    CONSTRAINT ck_cupos CHECK (cupos_disponibles <= cupos_totales)

);

```

COMMENT ON TABLE carrera IS 'Carreras de la FICCT disponibles para postulación.';

COMMENT ON COLUMN carrera.sigla IS 'Código corto único, p.ej. ING-SIS, ING-IND.';

COMMENT ON COLUMN carrera.cupos_disponibles IS 'Se decrementa automáticamente al confirmar admisión (trigger o lógica de negocio).';

--

=====

-- 7. POSTULANTE

--

=====

CREATE TABLE postulante (

id_postulante SERIAL PRIMARY KEY,

id_usuario INT NOT NULL UNIQUE

REFERENCES usuario(id_usuario)

ON UPDATE CASCADE ON DELETE RESTRICT,

ci VARCHAR(20) NOT NULL UNIQUE,

nombre VARCHAR(100) NOT NULL,

apellido VARCHAR(100) NOT NULL,

fecha_nacimiento DATE NOT NULL,

telefono VARCHAR(20),

colegio_origen VARCHAR(200),

created_at TIMESTAMPTZ NOT NULL DEFAULT NOW())

);

CREATE INDEX idx_postulante_usuario ON postulante(id_usuario);

COMMENT ON TABLE postulante IS 'Datos académicos y personales del aspirante al CUP.';

COMMENT ON COLUMN postulante.id_usuario IS 'Relación 1-a-1 con USUARIO (UNIQUE). El usuario tiene rol POSTULANTE.';

--

=====

-- 8. POSTULACION

--

=====

CREATE TABLE postulacion (

id_postulacion SERIAL PRIMARY KEY,

id_postulante INT NOT NULL REFERENCES postulante(id_postulante)

ON UPDATE CASCADE ON DELETE RESTRICT,

id_carrera_opcion1 INT NOT NULL REFERENCES carrera(id_carrera)

ON UPDATE CASCADE ON DELETE RESTRICT,

id_carrera_opcion2 INT REFERENCES carrera(id_carrera)

ON UPDATE CASCADE ON DELETE SET NULL,

puntaje_total NUMERIC(5,2) CHECK (puntaje_total BETWEEN 0 AND 100),

estado VARCHAR(20) NOT NULL DEFAULT 'PENDIENTE'

```

CHECK (estado IN
('PENDIENTE','APROBADO','REPROBADO','ADMITIDO')),
id_carrera_asignada INT REFERENCES carrera(id_carrera)
ON UPDATE CASCADE ON DELETE SET NULL,
fecha_postulacion TIMESTAMPTZ NOT NULL DEFAULT NOW(),
CONSTRAINT ck_opciones_distintas CHECK (id_carrera_opcion1 <> id_carrera_opcion2)
);

```

```

CREATE INDEX idx_postulacion_postulante ON postulacion(id_postulante);

```

```

CREATE INDEX idx_postulacion_estado ON postulacion(estados);

```

```

COMMENT ON TABLE postulacion IS 'Solicitud formal de ingreso al CUP. Registra
las dos opciones de carrera y el resultado final.';

```

```

COMMENT ON COLUMN postulacion.puntaje_total IS 'Promedio ponderado calculado por
CU17 (Algoritmo Admisión).';

```

```

COMMENT ON COLUMN postulacion.id_carrera_asignada IS 'Carrera asignada al concluir el
algoritmo de admisión (CU17).';

```

```

--
=====

```

```

-- 9. DOCUMENTO

```

```

--
=====

```

```

CREATE TABLE documento (

```

```

id_documento    SERIAL    PRIMARY KEY,
id_postulante   INT       NOT NULL REFERENCES postulante(id_postulante)
                ON UPDATE CASCADE ON DELETE CASCADE,
tipo_documento  VARCHAR(50) NOT NULL
                CHECK (tipo_documento IN
('CI','CERTIFICADO_BACHILLER','FOTO','COMPROBANTE_PAGO','OTRO')),
nombre_archivo  VARCHAR(255) NOT NULL,
ruta_archivo    VARCHAR(500) NOT NULL,
estado_verificacion VARCHAR(20) NOT NULL DEFAULT 'PENDIENTE'
                CHECK (estado_verificacion IN
('PENDIENTE','VERIFICADO','RECHAZADO')),
fecha_subida    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

CREATE INDEX idx_documento_postulante ON documento(id_postulante);

```

```

COMMENT ON TABLE documento IS 'Archivos digitales requeridos durante la
inscripción del postulante.';

```

```

COMMENT ON COLUMN documento.ruta_archivo IS 'Ruta relativa en el sistema de
archivos o URL del bucket de almacenamiento en nube.';

```

```

COMMENT ON COLUMN documento.estado_verificacion IS 'Estado de revisión manual por el
Administrador.';

```

```

--
=====

-- 10. PAGO

--
=====

CREATE TABLE pago (
    id_pago          SERIAL          PRIMARY KEY,
    id_postulante     INT             NOT NULL REFERENCES postulante(id_postulante)
                                ON UPDATE CASCADE ON DELETE RESTRICT,
    monto             NUMERIC(8,2)   NOT NULL CHECK (monto > 0),
    concepto          VARCHAR(150)   NOT NULL,
    estado_pago       VARCHAR(20)    NOT NULL DEFAULT 'PENDIENTE'
                                CHECK (estado_pago IN
('PENDIENTE','CONFIRMADO','RECHAZADO')),
    codigo_comprobante VARCHAR(100)  UNIQUE,
    fecha_pago        TIMESTAMPTZ    NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_pago_postulante ON pago(id_postulante);

CREATE INDEX idx_pago_estado     ON pago(estado_pago);

COMMENT ON TABLE pago          IS 'Registro de pagos de inscripción procesados a
través de la pasarela externa (CU07).';

```

COMMENT ON COLUMN pago.codigo_comprobante IS 'Código único devuelto por la pasarela de pago. UNIQUE garantiza idempotencia.';

--

=====

-- 11. MATERIA

--

=====

CREATE TABLE materia (

id_materia SERIAL PRIMARY KEY,

nombre_materia VARCHAR(100) NOT NULL,

sigla VARCHAR(10) NOT NULL UNIQUE,

horas_semana INT NOT NULL CHECK (horas_semana > 0),

activo BOOLEAN NOT NULL DEFAULT TRUE

);

COMMENT ON TABLE materia IS 'Materias del plan de estudios del CUP (Matemáticas, Física, Química, etc.).';

--

=====

-- 12. EVALUACION

--

=====


```

CREATE TABLE evaluacion (
    id_evaluacion SERIAL PRIMARY KEY,
    id_postulante INT NOT NULL REFERENCES postulante(id_postulante)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    id_materia INT NOT NULL REFERENCES materia(id_materia)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    nota1 NUMERIC(5,2) CHECK (nota1 BETWEEN 0 AND 100),
    nota2 NUMERIC(5,2) CHECK (nota2 BETWEEN 0 AND 100),
    nota3 NUMERIC(5,2) CHECK (nota3 BETWEEN 0 AND 100),
    promedio NUMERIC(5,2) GENERATED ALWAYS AS
        (ROUND((COALESCE(nota1,0) + COALESCE(nota2,0) +
COALESCE(nota3,0)) / 3.0, 2))
        STORED,
    estado VARCHAR(20) NOT NULL DEFAULT 'PENDIENTE'
        CHECK (estado IN ('PENDIENTE','APROBADO','REPROBADO')),
    fecha_evaluacion TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT uq_evaluacion UNIQUE (id_postulante, id_materia)
);

CREATE INDEX idx_evaluacion_postulante ON evaluacion(id_postulante);
CREATE INDEX idx_evaluacion_materia ON evaluacion(id_materia);

```

COMMENT ON TABLE evaluacion IS 'Calificaciones del postulante por materia. El promedio es una columna GENERATED ALWAYS AS STORED (calculada automáticamente por PostgreSQL).';

COMMENT ON COLUMN evaluacion.promedio IS 'Columna calculada: (nota1+nota2+nota3)/3. No requiere trigger ni CU15 adicional; PostgreSQL la actualiza al hacer INSERT/UPDATE.';

COMMENT ON COLUMN evaluacion.estado IS 'Derivado del promedio: APROBADO si promedio >= 60, REPROBADO en caso contrario (actualizado por trigger o lógica CU16).';

--

=====

-- 13. GRUPO

--

=====

CREATE TABLE grupo (

id_grupo SERIAL PRIMARY KEY,

nombre_grupo VARCHAR(50) NOT NULL UNIQUE,

capacidad_maxima INT NOT NULL DEFAULT 80 CHECK (capacidad_maxima > 0),

aula VARCHAR(50),

turno VARCHAR(10) NOT NULL

CHECK (turno IN ('MAÑANA','TARDE','NOCHE')),

activo BOOLEAN NOT NULL DEFAULT TRUE

);

COMMENT ON TABLE grupo IS 'Grupo de estudio del CUP. La capacidad máxima (80) es el divisor del algoritmo CEIL en CU19.';

COMMENT ON COLUMN grupo.nombre_grupo IS 'Identificador visual del grupo, p.ej. A-MAÑ-2026, B-TAR-2026.';

--

=====

-- 14. GRUPO_POSTULANTE

--

=====

CREATE TABLE grupo_postulante (

id_grupo_postulante SERIAL PRIMARY KEY,

id_grupo INT NOT NULL REFERENCES grupo(id_grupo)

ON UPDATE CASCADE ON DELETE RESTRICT,

id_postulante INT NOT NULL REFERENCES postulante(id_postulante)

ON UPDATE CASCADE ON DELETE RESTRICT,

fecha_asignacion TIMESTAMPTZ NOT NULL DEFAULT NOW(),

CONSTRAINT uq_grupo_postulante UNIQUE (id_grupo, id_postulante)

);

CREATE INDEX idx_gp_grupo ON grupo_postulante(id_grupo);

CREATE INDEX idx_gp_postulante ON grupo_postulante(id_postulante);

COMMENT ON TABLE grupo_postulante IS 'Tabla de unión N:M entre GRUPO y POSTULANTE.
Un postulante pertenece a un solo grupo (controlado en aplicación).';

--

=====

-- 15. GRUPO_MATERIA

--

=====

```
CREATE TABLE grupo_materia (  
    id_grupo_materia SERIAL    PRIMARY KEY,  
    id_grupo      INT          NOT NULL REFERENCES grupo(id_grupo)  
                                ON UPDATE CASCADE ON DELETE CASCADE,  
    id_materia    INT          NOT NULL REFERENCES materia(id_materia)  
                                ON UPDATE CASCADE ON DELETE RESTRICT,  
    horario       VARCHAR(100),  
    CONSTRAINT uq_grupo_materia UNIQUE (id_grupo, id_materia)  
);
```

COMMENT ON TABLE grupo_materia IS 'Materias asignadas a cada grupo con su horario de
clase.';

--

=====

-- 16. DOCENTE

--

=====

```
CREATE TABLE docente (  
    id_docente    SERIAL    PRIMARY KEY,  
    id_usuario    INT       NOT NULL UNIQUE  
                    REFERENCES usuario(id_usuario)  
                    ON UPDATE CASCADE ON DELETE RESTRICT,  
    especialidad  VARCHAR(150),  
    titulo_academico VARCHAR(50) NOT NULL  
                    CHECK (titulo_academico IN  
('LICENCIATURA','MAESTRIA','DOCTORADO','DIPLOMADO')),  
    telefono      VARCHAR(20),  
    activo        BOOLEAN   NOT NULL DEFAULT TRUE,  
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

```
CREATE INDEX idx_docente_usuario ON docente(id_usuario);
```

```
COMMENT ON TABLE docente IS 'Perfil académico del docente. La validación de  
título (maestría o superior) se realiza en CU21.';
```

```
COMMENT ON COLUMN docente.titulo_academico IS 'Conjunto cerrado de títulos aceptados  
por el reglamento FICCT.';
```

```

--
=====

-- 17. ASIGNACION_DOCENTE

--
=====

CREATE TABLE asignacion_docente (

    id_asignacion  SERIAL      PRIMARY KEY,

    id_docente     INT         NOT NULL REFERENCES docente(id_docente)

                                ON UPDATE CASCADE ON DELETE RESTRICT,

    id_grupo       INT         NOT NULL REFERENCES grupo(id_grupo)

                                ON UPDATE CASCADE ON DELETE RESTRICT,

    id_materia     INT         NOT NULL REFERENCES materia(id_materia)

                                ON UPDATE CASCADE ON DELETE RESTRICT,

    fecha_asignacion TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    activo         BOOLEAN     NOT NULL DEFAULT TRUE,

    CONSTRAINT uq_asignacion UNIQUE (id_grupo, id_materia)  -- un docente por grupo-
materia

);

CREATE INDEX idx_asignacion_docente ON asignacion_docente(id_docente);

CREATE INDEX idx_asignacion_grupo  ON asignacion_docente(id_grupo);

```

COMMENT ON TABLE asignacion_docente IS 'Carga horaria: un docente asignado a un grupo y una materia específica (CU22). La restricción UNIQUE garantiza que no haya duplicados por grupo-materia.';

--

=====

-- FIN DEL DISEÑO FÍSICO

--

=====

6.2.3 Población de datos

--

=====

-- POBLACIÓN DE DATOS — Sistema CUP (FICCT - UAGRM)

-- Motor: PostgreSQL 15+

-- Gestión: 1-2026

-- Nota: passwords en texto real se mostrarían como hash bcrypt

-- en producción usar: crypt('password', gen_salt('bf'))

--

=====

--

=====

-- 1. ROL

--

=====

```

INSERT INTO rol (id_rol, nombre_rol, descripcion) VALUES

(1, 'ADMIN',    'Administrador con acceso total al sistema CUP'),

(2, 'DOCENTE',  'Docente con acceso a registro de notas y grupos asignados'),

(3, 'POSTULANTE', 'Aspirante al curso de nivelación con acceso a su perfil y resultados');


--

=====

-- 2. USUARIO (passwords: Admin123!, Docente01!, Postulante01! ... en hash bcrypt simulado)

--

=====

INSERT INTO usuario (id_usuario, ci, nombre, apellido, email, password_hash, id_rol, activo)
VALUES

-- Administradores

(1,  '5012347',    'Carlos',      'Vaca  Diez',      'cvacadiez@ficct.uagrm.edu.bo',
'$2a$12$AdmHash00000000000001', 1, TRUE),

(2,  '5089231',    'Lorena',      'Antelo  Suárez',  'lantelo@ficct.uagrm.edu.bo',
'$2a$12$AdmHash00000000000002', 1, TRUE),

-- Docentes

(3,  '4123456',    'Hugo',        'Rocha  Vargas',   'hrocha@ficct.uagrm.edu.bo',
'$2a$12$DocHash00000000000001', 2, TRUE),

(4,  '4234567',    'Patricia',    'Méndez  Torres',  'pmendez@ficct.uagrm.edu.bo',
'$2a$12$DocHash00000000000002', 2, TRUE),

(5,  '4345678',    'Rodrigo',     'Salinas  Paz',     'rsalinas@ficct.uagrm.edu.bo',
'$2a$12$DocHash00000000000003', 2, TRUE),

```



```
(6, '4456789', 'Carla', 'Ibáñez Vaca', 'cibanez@ficct.uagrm.edu.bo',  
'$2a$12$DocHash00000000000004', 2, TRUE),
```

```
(7, '4567890', 'Marcelo', 'Torrez Aguilar', 'mtorrez@ficct.uagrm.edu.bo',  
'$2a$12$DocHash00000000000005', 2, TRUE),
```

-- Postulantes

```
(8, '8123456', 'Ana', 'Flores Quiroga', 'ana.flores@gmail.com',  
'$2a$12$PosHash00000000000001', 3, TRUE),
```

```
(9, '8234567', 'Luis', 'Peredo Castro', 'luis.peredo@gmail.com',  
'$2a$12$PosHash00000000000002', 3, TRUE),
```

```
(10, '8345678', 'Valeria', 'Guzmán Ríos', 'valeria.guzman@gmail.com',  
'$2a$12$PosHash00000000000003', 3, TRUE),
```

```
(11, '8456789', 'Diego', 'Montero Suárez', 'diego.montero@gmail.com',  
'$2a$12$PosHash00000000000004', 3, TRUE),
```

```
(12, '8567890', 'Daniela', 'Rivero Mamani', 'daniela.rivero@gmail.com',  
'$2a$12$PosHash00000000000005', 3, TRUE),
```

```
(13, '8678901', 'Sergio', 'Álvarez Pinto', 'sergio.alvarez@gmail.com',  
'$2a$12$PosHash00000000000006', 3, TRUE),
```

```
(14, '8789012', 'Camila', 'Durán Heredia', 'camila.duran@gmail.com',  
'$2a$12$PosHash00000000000007', 3, TRUE),
```

```
(15, '8890123', 'Fernanda', 'Cossío Velasco', 'fernanda.cossio@gmail.com',  
'$2a$12$PosHash00000000000008', 3, TRUE);
```

```
SELECT setval('usuario_id_usuario_seq', 15);
```

--

=====

-- 3. BITACORA_AUDITORIA

--

=====

INSERT INTO bitacora_auditoria (id_usuario, accion, tabla_afectada, descripcion, ip_origen, fecha_hora) VALUES

(1, 'LOGIN', NULL, 'Inicio de sesión exitoso - Admin', '192.168.1.10', '2026-03-01 08:00:00-04'),

(1, 'INSERT', 'carrera', 'Registro de carrera ING-SIS', '192.168.1.10', '2026-03-01 08:15:00-04'),

(1, 'INSERT', 'periodo_admision', 'Creación del período Gestión 1-2026', '192.168.1.10', '2026-03-01 08:30:00-04'),

(8, 'LOGIN', NULL, 'Inicio de sesión - Postulante Ana Flores', '190.121.44.12', '2026-03-05 09:10:00-04'),

(8, 'INSERT', 'postulante', 'Registro de postulante Ana Flores Quiroga', '190.121.44.12', '2026-03-05 09:12:00-04'),

(3, 'LOGIN', NULL, 'Inicio de sesión - Docente Hugo Rocha', '192.168.1.21', '2026-03-10 07:45:00-04'),

(3, 'INSERT', 'evaluacion', 'Registro de notas materia MAT - grupo A', '192.168.1.21', '2026-03-10 10:00:00-04'),

(1, 'UPDATE', 'postulacion', 'Ejecución algoritmo admisión CU17 - 8 postulantes', '192.168.1.10', '2026-04-01 14:00:00-04'),

(1, 'EXPORT', 'reporte', 'Exportación Dashboard KPIs - PDF', '192.168.1.10', '2026-04-02 09:00:00-04'),

```
(2, 'LOGIN', NULL, 'Inicio de sesión - Admin Lorena Antelo', '192.168.1.11', '2026-04-05 08:00:00-04');
```

```
--
```

```
=====
```

```
-- 4. PARAMETRO_SISTEMA
```

```
--
```

```
=====
```

```
INSERT INTO parametro_sistema (clave, valor, descripcion) VALUES
```

```
('nota_minima_aprobacion', '60', 'Promedio mínimo para aprobar el CUP'),
```

```
('capacidad_maxima_grupo', '80', 'Divisor del algoritmo CEIL(n/80) para calcular grupos (CU19)'),
```

```
('monto_inscripcion_bs', '150', 'Monto de inscripción en bolivianos (CU07)'),
```

```
('periodo_activo_id', '1', 'ID del período de admisión actualmente activo'),
```

```
('max_notas_por_materia', '3', 'Número máximo de calificaciones por materia por postulante'),
```

```
('intentos_login_max', '5', 'Máximo de intentos fallidos antes de bloquear cuenta');
```

```
--
```

```
=====
```

```
-- 5. PERIODO_ADMISION
```

```
--
```

```
=====
```

```
INSERT INTO periodo_admision (id_periodo, nombre, fecha_inicio, fecha_fin, estado) VALUES
```

```
(1, 'Gestión 1-2026', '2026-03-01', '2026-06-30', 'ACTIVO');
```

```

--
=====

-- 6. CARRERA

--
=====

INSERT INTO carrera (id_carrera, nombre_carrera, sigla, cupos_totales, cupos_disponibles,
activo) VALUES

(1, 'Ingeniería en Sistemas Computacionales', 'ING-SIS', 200, 155, TRUE),
(2, 'Ingeniería Industrial', 'ING-IND', 150, 120, TRUE),
(3, 'Ingeniería Civil', 'ING-CIV', 180, 145, TRUE),
(4, 'Ingeniería Eléctrica', 'ING-ELE', 100, 88, TRUE),
(5, 'Ingeniería Electrónica y Telecomunicaciones', 'ING-ELC', 120, 105, TRUE);

--
=====

-- 7. POSTULANTE

--
=====

INSERT INTO postulante (id_postulante, id_usuario, ci, nombre, apellido, fecha_nacimiento,
telefono, colegio_origen) VALUES

(1, 8, '8123456', 'Ana', 'Flores Quiroga', '2007-04-12', '77812345', 'Colegio Nacional Florida'),
(2, 9, '8234567', 'Luis', 'Peredo Castro', '2007-08-23', '77823456', 'Colegio Santa Ana'),
(3, 10, '8345678', 'Valeria', 'Guzmán Ríos', '2006-11-05', '77834567', 'Colegio Juan Pablo II'),

```

```
(4, 11, '8456789', 'Diego', 'Montero Suárez', '2007-01-30', '77845678', 'U.E. La Salle'),
(5, 12, '8567890', 'Daniela', 'Rivero Mamani', '2006-07-17', '77856789', 'U.E. Santo Domingo'),
(6, 13, '8678901', 'Sergio', 'Álvarez Pinto', '2007-02-28', '77867890', 'Colegio Nacional Florida'),
(7, 14, '8789012', 'Camila', 'Durán Heredia', '2006-09-14', '77878901', 'U.E. Santa Teresa'),
(8, 15, '8890123', 'Fernanda', 'Cossío Velasco', '2007-05-03', '77889012', 'Colegio Juan Pablo II');
```

```
SELECT setval('postulante_id_postulante_seq', 8);
```

```
--
```

```
=====
```

```
-- 8. POSTULACION
```

```
--
```

```
=====
```

```
INSERT INTO postulacion (id_postulacion, id_postulante, id_carrera_opcion1,
id_carrera_opcion2, puntaje_total, estado, id_carrera_asignada, fecha_postulacion) VALUES
```

```
(1, 1, 1, 2, 78.33, 'ADMITIDO', 1, '2026-03-05 09:20:00-04'),
```

```
(2, 2, 1, 3, 65.00, 'ADMITIDO', 1, '2026-03-06 10:00:00-04'),
```

```
(3, 3, 2, 1, 82.67, 'ADMITIDO', 2, '2026-03-07 11:15:00-04'),
```

```
(4, 4, 1, 5, 55.33, 'REPROBADO', NULL, '2026-03-08 08:30:00-04'),
```

```
(5, 5, 3, 2, 70.00, 'ADMITIDO', 3, '2026-03-09 09:45:00-04'),
```

```
(6, 6, 1, 4, 91.00, 'ADMITIDO', 1, '2026-03-10 10:30:00-04'),
```

```
(7, 7, 4, 1, 60.33, 'ADMITIDO', 4, '2026-03-11 11:00:00-04'),
```

```
(8, 8, 2, 3, 48.00, 'REPROBADO', NULL, '2026-03-12 08:00:00-04');
```

```
--
=====

-- 9. DOCUMENTO

--
=====

INSERT INTO documento (id_postulante, tipo_documento, nombre_archivo, ruta_archivo,
estado_verificacion) VALUES

(1, 'CI',          'ci_ana_flores.pdf',    '/uploads/docs/ci/ci_ana_flores.pdf',    'VERIFICADO'),

(1,                                     'CERTIFICADO_BACHILLER','bach_ana_flores.pdf',
'/uploads/docs/bach/bach_ana_flores.pdf', 'VERIFICADO'),

(1,          'COMPROBANTE_PAGO',                                     'pago_ana_flores.pdf',
'/uploads/docs/pagos/pago_ana_flores.pdf', 'VERIFICADO'),

(2, 'CI',          'ci_luis_peredo.pdf',    '/uploads/docs/ci/ci_luis_peredo.pdf',    'VERIFICADO'),

(2,                                     'CERTIFICADO_BACHILLER','bach_luis_peredo.pdf',
'/uploads/docs/bach/bach_luis_peredo.pdf', 'VERIFICADO'),

(3, 'CI',          'ci_valeria_guzman.pdf',  '/uploads/docs/ci/ci_valeria_guzman.pdf',
'VERIFICADO'),

(3, 'CERTIFICADO_BACHILLER','bach_valeria.pdf',      '/uploads/docs/bach/bach_valeria.pdf',
'PENDIENTE'),

(4, 'CI',          'ci_diego_montero.pdf',   '/uploads/docs/ci/ci_diego_montero.pdf',
'VERIFICADO'),

(5, 'CI',          'ci_daniela_rivero.pdf',  '/uploads/docs/ci/ci_daniela_rivero.pdf', 'VERIFICADO'),

(5, 'COMPROBANTE_PAGO', 'pago_daniela.pdf',    '/uploads/docs/pagos/pago_daniela.pdf',
'VERIFICADO');
```

--

=====

-- 10. PAGO

--

=====

INSERT INTO pago (id_postulante, monto, concepto, estado_pago, codigo_comprobante, fecha_pago) VALUES

(1, 150.00, 'Inscripción CUP Gestión 1-2026', 'CONFIRMADO', 'TIGO-20260305-000001', '2026-03-05 09:15:00-04'),

(2, 150.00, 'Inscripción CUP Gestión 1-2026', 'CONFIRMADO', 'TIGO-20260306-000002', '2026-03-06 09:55:00-04'),

(3, 150.00, 'Inscripción CUP Gestión 1-2026', 'CONFIRMADO', 'BNB-20260307-000003', '2026-03-07 11:10:00-04'),

(4, 150.00, 'Inscripción CUP Gestión 1-2026', 'CONFIRMADO', 'TIGO-20260308-000004', '2026-03-08 08:25:00-04'),

(5, 150.00, 'Inscripción CUP Gestión 1-2026', 'CONFIRMADO', 'BNB-20260309-000005', '2026-03-09 09:40:00-04'),

(6, 150.00, 'Inscripción CUP Gestión 1-2026', 'CONFIRMADO', 'TIGO-20260310-000006', '2026-03-10 10:25:00-04'),

(7, 150.00, 'Inscripción CUP Gestión 1-2026', 'CONFIRMADO', 'BNB-20260311-000007', '2026-03-11 10:55:00-04'),

(8, 150.00, 'Inscripción CUP Gestión 1-2026', 'RECHAZADO', NULL, '2026-03-12 07:55:00-04');

```

--
=====

-- 11. MATERIA

--
=====

INSERT INTO materia (id_materia, nombre_materia, sigla, horas_semana, activo) VALUES

(1, 'Matemáticas', 'MAT', 6, TRUE),

(2, 'Física', 'FIS', 4, TRUE),

(3, 'Química', 'QUI', 4, TRUE);

--
=====

-- 12. EVALUACION

-- (promedio es GENERATED ALWAYS AS, no se inserta)

--
=====

INSERT INTO evaluacion (id_postulante, id_materia, nota1, nota2, nota3, estado,
fecha_evaluacion) VALUES

-- Ana Flores — Aprobada (prom 78.33)

(1, 1, 80.00, 75.00, 82.00, 'APROBADO', '2026-03-25 10:00:00-04'),

(1, 2, 77.00, 78.00, 76.00, 'APROBADO', '2026-03-25 10:00:00-04'),

(1, 3, 79.00, 80.00, 78.00, 'APROBADO', '2026-03-25 10:00:00-04'),

-- Luis Peredo — Aprobado (prom 65)

(2, 1, 60.00, 70.00, 65.00, 'APROBADO', '2026-03-25 10:30:00-04'),

```


(2, 2, 62.00, 68.00, 65.00, 'APROBADO', '2026-03-25 10:30:00-04'),
 (2, 3, 65.00, 63.00, 67.00, 'APROBADO', '2026-03-25 10:30:00-04'),
 -- Valeria Guzmán — Aprobada (prom 82.67)
 (3, 1, 85.00, 80.00, 84.00, 'APROBADO', '2026-03-25 11:00:00-04'),
 (3, 2, 82.00, 83.00, 81.00, 'APROBADO', '2026-03-25 11:00:00-04'),
 (3, 3, 84.00, 82.00, 83.00, 'APROBADO', '2026-03-25 11:00:00-04'),
 -- Diego Montero — Reprobado (prom 55.33)
 (4, 1, 50.00, 58.00, 55.00, 'REPROBADO', '2026-03-25 11:30:00-04'),
 (4, 2, 52.00, 60.00, 54.00, 'REPROBADO', '2026-03-25 11:30:00-04'),
 (4, 3, 55.00, 57.00, 53.00, 'REPROBADO', '2026-03-25 11:30:00-04'),
 -- Daniela Rivero — Aprobada (prom 70)
 (5, 1, 70.00, 68.00, 72.00, 'APROBADO', '2026-03-25 12:00:00-04'),
 (5, 2, 71.00, 69.00, 70.00, 'APROBADO', '2026-03-25 12:00:00-04'),
 (5, 3, 70.00, 70.00, 70.00, 'APROBADO', '2026-03-25 12:00:00-04'),
 -- Sergio Álvarez — Aprobado (prom 91)
 (6, 1, 92.00, 90.00, 93.00, 'APROBADO', '2026-03-25 12:30:00-04'),
 (6, 2, 90.00, 92.00, 91.00, 'APROBADO', '2026-03-25 12:30:00-04'),
 (6, 3, 91.00, 90.00, 91.00, 'APROBADO', '2026-03-25 12:30:00-04'),
 -- Camila Durán — Aprobada (prom 60.33)
 (7, 1, 60.00, 61.00, 60.00, 'APROBADO', '2026-03-25 13:00:00-04'),
 (7, 2, 60.00, 60.00, 61.00, 'APROBADO', '2026-03-25 13:00:00-04'),
 (7, 3, 60.00, 60.00, 61.00, 'APROBADO', '2026-03-25 13:00:00-04'),
 -- Fernanda Cossío — Reprobada (prom 48)

```
(8, 1, 45.00, 50.00, 48.00, 'REPROBADO', '2026-03-25 13:30:00-04'),
(8, 2, 47.00, 49.00, 48.00, 'REPROBADO', '2026-03-25 13:30:00-04'),
(8, 3, 48.00, 47.00, 49.00, 'REPROBADO', '2026-03-25 13:30:00-04');
```

```
--
```

```
=====
```

```
-- 13. GRUPO
```

```
-- 8 postulantes aprobados →  $\text{CEIL}(6/80) = 1$  grupo
```

```
-- Se crean 2 grupos de demostración
```

```
--
```

```
=====
```

```
INSERT INTO grupo (id_grupo, nombre_grupo, capacidad_maxima, aula, turno, activo) VALUES
(1, 'A-MAÑ-2026', 80, 'Aula 101 - FICCT', 'MAÑANA', TRUE),
(2, 'B-TAR-2026', 80, 'Aula 205 - FICCT', 'TARDE', TRUE);
```

```
--
```

```
=====
```

```
-- 14. GRUPO_POSTULANTE (solo los 6 admitidos)
```

```
--
```

```
=====
```

```
INSERT INTO grupo_postulante (id_grupo, id_postulante, fecha_asignacion) VALUES
(1, 1, '2026-04-01 14:30:00-04'), -- Ana
(1, 2, '2026-04-01 14:30:00-04'), -- Luis
(1, 3, '2026-04-01 14:30:00-04'), -- Valeria
```

```

(1, 5, '2026-04-01 14:30:00-04'), -- Daniela

(2, 6, '2026-04-01 14:30:00-04'), -- Sergio

(2, 7, '2026-04-01 14:30:00-04'); -- Camila


--
=====

-- 15. GRUPO_MATERIA

--
=====

INSERT INTO grupo_materia (id_grupo, id_materia, horario) VALUES

(1, 1, 'Lunes-Miércoles-Viernes 08:00-10:00'),

(1, 2, 'Martes-Jueves 08:00-10:00'),

(1, 3, 'Lunes-Miércoles 10:00-12:00'),

(2, 1, 'Lunes-Miércoles-Viernes 14:00-16:00'),

(2, 2, 'Martes-Jueves 14:00-16:00'),

(2, 3, 'Lunes-Miércoles 16:00-18:00');


--
=====

-- 16. DOCENTE

--
=====

INSERT INTO docente (id_docente, id_usuario, especialidad, titulo_academico, telefono, activo)
VALUES

```

```

(1, 3, 'Cálculo Diferencial e Integral', 'MAESTRIA', '70312345', TRUE),
(2, 4, 'Física Mecánica y Ondas', 'MAESTRIA', '70323456', TRUE),
(3, 5, 'Química General e Inorgánica', 'LICENCIATURA', '70334567', TRUE),
(4, 6, 'Álgebra Lineal', 'DOCTORADO', '70345678', TRUE),
(5, 7, 'Electromagnetismo', 'MAESTRIA', '70356789', TRUE);

--
=====

-- 17. ASIGNACION_DOCENTE

--
=====

INSERT INTO asignacion_docente (id_docente, id_grupo, id_materia, fecha_asignacion, activo)
VALUES

(1, 1, 1, '2026-04-02 08:00:00-04', TRUE), -- Hugo → Grupo A, Matemáticas
(2, 1, 2, '2026-04-02 08:00:00-04', TRUE), -- Patricia → Grupo A, Física
(3, 1, 3, '2026-04-02 08:00:00-04', TRUE), -- Rodrigo → Grupo A, Química
(4, 2, 1, '2026-04-02 08:00:00-04', TRUE), -- Carla → Grupo B, Matemáticas
(5, 2, 2, '2026-04-02 08:00:00-04', TRUE), -- Marcelo → Grupo B, Física
(3, 2, 3, '2026-04-02 08:00:00-04', TRUE); -- Rodrigo → Grupo B, Química (carga compartida)

--
=====

-- VERIFICACIÓN RÁPIDA (ejecutar después del INSERT)

```

```
--
=====

-- SELECT COUNT(*) FROM usuario;      -- esperado: 15
-- SELECT COUNT(*) FROM postulante;    -- esperado: 8
-- SELECT COUNT(*) FROM evaluacion;    -- esperado: 24
-- SELECT COUNT(*) FROM grupo_postulante; -- esperado: 6
-- SELECT id_postulante, id_materia, nota1, nota2, nota3, promedio
--     FROM evaluacion ORDER BY id_postulante, id_materia; -- promedio calculado
automáticamente
```

```
--
=====

-- FIN DE LA POBLACIÓN DE DATOS
```

```
--
=====
```

6.2.4 Consultas

```
--
=====

-- CONSULTAS SQL — Sistema CUP (FICCT - UAGRM)
-- Motor: PostgreSQL 15+
-- Gestión: 1-2026
-- Organizadas por módulo del sistema (según casos de uso)
--
=====
```

--

=====

-- MÓDULO 1: AUTENTICACIÓN (CU01, CU02, CU04)

--

=====

-- CU01: Iniciar sesión — verificar credenciales por email

SELECT u.id_usuario,

u.ci,

u.nombre,

u.apellido,

u.email,

u.password_hash,

u.activo,

r.nombre_rol

FROM usuario u

JOIN rol r ON r.id_rol = u.id_rol

WHERE u.email = 'ana.flores@gmail.com'

AND u.activo = TRUE;

-- CU01: Registrar evento de login en bitácora

```
INSERT INTO bitacora_auditoria (id_usuario, accion, tabla_afectada, descripcion, ip_origen)
VALUES (8, 'LOGIN', NULL, 'Inicio de sesión exitoso', '190.121.44.12');
```

-- CU02: Cerrar sesión — registrar logout en bitácora

```
INSERT INTO bitacora_auditoria (id_usuario, accion, tabla_afectada, descripcion, ip_origen)
VALUES (8, 'LOGOUT', NULL, 'Cierre de sesión', '190.121.44.12');
```

-- CU04: Cambiar contraseña — actualizar hash

```
UPDATE usuario
SET password_hash = '$2a$12$NuevoHashBcrypt',
    updated_at = NOW()
WHERE id_usuario = 8
AND activo = TRUE;
```

-- CU01: Bloquear cuenta tras N intentos fallidos (verificación de estado)

```
SELECT activo FROM usuario WHERE email = 'ana.flores@gmail.com';
```

-- Consulta de auditoría: últimos 10 intentos de login de un usuario

```
SELECT accion, descripcion, ip_origen, fecha_hora
FROM bitacora_auditoria
WHERE id_usuario = 8
AND accion IN ('LOGIN','LOGOUT')
ORDER BY fecha_hora DESC
```

```
LIMIT 10;
```

```
--
```

```
=====
```

```
-- MÓDULO 2: GESTIÓN DE CUENTAS RBAC (CU05, CU31)
```

```
--
```

```
=====
```

```
-- CU05: Listar todos los usuarios con su rol
```

```
SELECT u.id_usuario,  
       u.ci,  
       u.nombre || ' ' || u.apellido AS nombre_completo,  
       u.email,  
       r.nombre_rol,  
       u.activo,  
       u.created_at
```

```
FROM   usuario u
```

```
JOIN   rol r ON r.id_rol = u.id_rol
```

```
ORDER BY r.nombre_rol, u.apellido;
```

```
-- CU05: Crear nuevo usuario con rol asignado
```

```
INSERT INTO usuario (ci, nombre, apellido, email, password_hash, id_rol)
```



```
VALUES ('9012345', 'Nuevo', 'Usuario', 'nuevo@ficct.edu.bo', '$2a$12$HashNuevo', 2);
```

```
-- CU05: Actualizar rol de un usuario (cambio de permisos)
```

```
UPDATE usuario
```

```
SET id_rol = 1,
```

```
    updated_at = NOW()
```

```
WHERE id_usuario = 5;
```

```
-- CU05: Desactivar cuenta (baja lógica, nunca DELETE)
```

```
UPDATE usuario
```

```
SET activo = FALSE,
```

```
    updated_at = NOW()
```

```
WHERE id_usuario = 10;
```

```
-- CU05: Reactivar cuenta
```

```
UPDATE usuario
```

```
SET activo = TRUE,
```

```
    updated_at = NOW()
```

```
WHERE id_usuario = 10;
```

```
-- CU05: Listar usuarios por rol específico
```

```
SELECT u.id_usuario, u.nombre, u.apellido, u.email, u.activo
```

```
FROM usuario u
```

```

JOIN rol r ON r.id_rol = u.id_rol

WHERE r.nombre_rol = 'DOCENTE'

ORDER BY u.apellido;

```

```
-- CU31: Importar cuentas masivas desde Excel/CSV
```

```
--      (simulación de inserción masiva con ON CONFLICT para idempotencia)
```

```
INSERT INTO usuario (ci, nombre, apellido, email, password_hash, id_rol)
```

```
VALUES
```

```
    ('91111111', 'Pedro', 'Mamani', 'pmamani@ficct.edu.bo', '$2a$12$Hash001', 2),
```

```
    ('92222222', 'Sandra', 'Cuellar', 'scuellar@ficct.edu.bo', '$2a$12$Hash002', 2),
```

```
    ('93333333', 'Jorge', 'Vaca', 'jvaca@ficct.edu.bo', '$2a$12$Hash003', 2)
```

```
ON CONFLICT (email) DO UPDATE
```

```
    SET nombre = EXCLUDED.nombre,
```

```
        apellido = EXCLUDED.apellido,
```

```
        updated_at = NOW();
```

```
-- CU31: Verificar cuántos usuarios se importaron en el último lote
```

```
SELECT COUNT(*) AS total_importados
```

```
FROM bitacora_auditoria
```

```
WHERE accion = 'INSERT'
```

```
    AND tabla_afectada = 'usuario'
```

```
    AND fecha_hora >= NOW() - INTERVAL '1 hour';
```

```

--
=====

-- MÓDULO 3: REGISTRO DE POSTULANTES (CU07, CU08)

--
=====

-- CU08: Registrar nuevo postulante (se hace junto con el usuario en transacción)

BEGIN;

INSERT INTO usuario (ci, nombre, apellido, email, password_hash, id_rol)
VALUES ('89999999', 'Mario', 'Chávez', 'mario.chavez@gmail.com', '$2a$12$HashPos', 3)
RETURNING id_usuario;

-- Usar el id_usuario retornado en la siguiente INSERT

INSERT INTO postulante (id_usuario, ci, nombre, apellido, fecha_nacimiento, telefono,
colegio_origen)
VALUES (currval('usuario_id_usuario_seq'), '89999999', 'Mario', 'Chávez', '2007-03-10',
'703999999', 'U.E. La Salle');

COMMIT;

-- CU08: Verificar si el CI ya está registrado (evitar duplicados)

SELECT COUNT(*) FROM postulante WHERE ci = '89999999';

-- CU08: Ver perfil completo de un postulante

```

```

SELECT p.id_postulante,

       p.ci,

       p.nombre || ' ' || p.apellido AS nombre_completo,

       p.fecha_nacimiento,

       EXTRACT(YEAR FROM AGE(p.fecha_nacimiento)) AS edad,

       p.telefono,

       p.colegio_origen,

       u.email,

       u.activo,

       p.created_at

FROM   postulante p

JOIN   usuario u ON u.id_usuario = p.id_usuario

WHERE  p.id_postulante = 1;

```

-- CU08: Listar todos los postulantes registrados

```

SELECT p.id_postulante,

       p.ci,

       p.nombre || ' ' || p.apellido AS nombre_completo,

       p.colegio_origen,

       u.email,

       p.created_at

FROM   postulante p

JOIN   usuario u ON u.id_usuario = p.id_usuario

```

```
ORDER BY p.apellido, p.nombre;
```

```
-- CU07: Registrar pago de inscripción
```

```
INSERT INTO pago (id_postulante, monto, concepto, estado_pago, codigo_comprobante)
```

```
VALUES (1, 150.00, 'Inscripción CUP Gestión 1-2026', 'PENDIENTE', NULL);
```

```
-- CU07: Confirmar pago (respuesta de pasarela externa)
```

```
UPDATE pago
```

```
SET estado_pago = 'CONFIRMADO',
```

```
    codigo_comprobante = 'TIGO-20260315-000099',
```

```
    fecha_pago = NOW()
```

```
WHERE id_pago = 1;
```

```
-- CU07: Verificar si un postulante ya pagó
```

```
SELECT estado_pago, codigo_comprobante, fecha_pago
```

```
FROM pago
```

```
WHERE id_postulante = 1
```

```
    AND estado_pago = 'CONFIRMADO';
```

```
-- CU07: Historial de pagos de un postulante
```

```
SELECT id_pago, monto, concepto, estado_pago, codigo_comprobante, fecha_pago
```

```
FROM pago
```

```
WHERE id_postulante = 1
```

```
ORDER BY fecha_pago DESC;
```

```
-- CU08: Subir documento del postulante
```

```
INSERT INTO documento (id_postulante, tipo_documento, nombre_archivo, ruta_archivo)  
VALUES (1, 'CI', 'ci_mario_chavez.pdf', '/uploads/docs/ci/ci_mario_chavez.pdf');
```

```
-- CU08: Verificar documentos pendientes de un postulante
```

```
SELECT tipo_documento, nombre_archivo, estado_verificacion, fecha_subida  
FROM documento  
WHERE id_postulante = 1  
ORDER BY fecha_subida;
```

```
-- CU08: Verificar/rechazar documento (Admin)
```

```
UPDATE documento  
SET estado_verificacion = 'VERIFICADO'  
WHERE id_documento = 1;
```

```
-- CU08: Postulantes con documentación incompleta
```

```
SELECT p.id_postulante,  
       p.nombre || ' ' || p.apellido AS nombre_completo,  
       COUNT(d.id_documento) FILTER (WHERE d.estado_verificacion = 'PENDIENTE') AS  
docs_pendientes,
```

```

COUNT(d.id_documento) FILTER (WHERE d.estado_verificacion = 'RECHAZADO') AS
docs_rechazados

FROM postulante p

LEFT JOIN documento d ON d.id_postulante = p.id_postulante

GROUP BY p.id_postulante, p.nombre, p.apellido

HAVING COUNT(d.id_documento) FILTER (WHERE d.estado_verificacion IN
('PENDIENTE','RECHAZADO')) > 0

ORDER BY p.apellido;

```

```
--
```

```
=====
```

```
-- MÓDULO 4: EXÁMENES Y NOTAS (CU13, CU15, CU16)
```

```
--
```

```
=====
```

```
-- CU13: Listar postulantes de un grupo para registrar notas (vista del docente)
```

```

SELECT p.id_postulante,
       p.ci,
       p.nombre || ' ' || p.apellido AS nombre_completo,
       gp.fecha_asignacion
FROM grupo_postulante gp
JOIN postulante p ON p.id_postulante = gp.id_postulante
WHERE gp.id_grupo = 1

```

```
ORDER BY p.apellido, p.nombre;
```

```
-- CU13: Registrar notas de un postulante en una materia
```

```
INSERT INTO evaluacion (id_postulante, id_materia, nota1, nota2, nota3, estado)
```

```
VALUES (1, 1, 80.00, 75.00, 82.00, 'PENDIENTE')
```

```
ON CONFLICT (id_postulante, id_materia) DO UPDATE
```

```
SET nota1      = EXCLUDED.nota1,
```

```
    nota2      = EXCLUDED.nota2,
```

```
    nota3      = EXCLUDED.nota3,
```

```
    fecha_evaluacion = NOW());
```

```
-- CU13: Verificar que una nota esté en rango (validación en DB como respaldo)
```

```
-- La restricción CHECK ya lo garantiza, pero se puede consultar:
```

```
SELECT id_evaluacion, nota1, nota2, nota3, promedio
```

```
FROM   evaluacion
```

```
WHERE  id_postulante = 1 AND id_materia = 1;
```

```
-- CU15: Consultar promedio calculado (GENERATED ALWAYS AS STORED)
```

```
-- El promedio se actualiza solo; no requiere UPDATE manual.
```

```
SELECT e.id_evaluacion,
```

```
       p.nombre || ' ' || p.apellido AS postulante,
```

```
       m.nombre_materia,
```

```
       e.nota1, e.nota2, e.nota3,
```



```

        e.promedio,
        e.estado
FROM   evaluacion e
JOIN   postulante p ON p.id_postulante = e.id_postulante
JOIN   materia   m ON m.id_materia   = e.id_materia
WHERE  e.id_postulante = 1
ORDER BY m.nombre_materia;

```

-- CU15: Calcular promedio global de un postulante entre todas las materias

```

SELECT p.id_postulante,
       p.nombre || ' ' || p.apellido AS nombre_completo,
       ROUND(AVG(e.promedio), 2)   AS promedio_global
FROM   evaluacion e
JOIN   postulante p ON p.id_postulante = e.id_postulante
WHERE  e.id_postulante = 1
GROUP BY p.id_postulante, p.nombre, p.apellido;

```

-- CU16: Actualizar estado de aprobación según promedio (≥ 60 = APROBADO)

```

UPDATE evaluacion
SET   estado = CASE WHEN promedio >= 60 THEN 'APROBADO' ELSE 'REPROBADO' END
WHERE id_postulante = 1;

```

-- CU16: Actualizar estado de TODAS las evaluaciones en lote (cierre de período)

```
UPDATE evaluacion
```

```
SET estado = CASE WHEN promedio >= 60 THEN 'APROBADO' ELSE 'REPROBADO' END
```

```
WHERE estado = 'PENDIENTE';
```

```
-- CU13: Vista completa de notas de un grupo con promedios
```

```
SELECT p.nombre || ' ' || p.apellido AS postulante,
```

```
    m.sigla,
```

```
    e.nota1, e.nota2, e.nota3,
```

```
    e.promedio,
```

```
    e.estado
```

```
FROM evaluacion e
```

```
JOIN postulante p ON p.id_postulante = e.id_postulante
```

```
JOIN materia m ON m.id_materia = e.id_materia
```

```
JOIN grupo_postulante gp ON gp.id_postulante = e.id_postulante
```

```
WHERE gp.id_grupo = 1
```

```
ORDER BY p.apellido, m.sigla;
```

```
-- CU13: Verificar que no se ingresen más de 3 notas por materia
```

```
-- (la tabla tiene nota1, nota2, nota3 — estructura fija; restricción en diseño)
```

```
SELECT COUNT(*) FROM evaluacion
```

```
WHERE id_postulante = 1 AND id_materia = 1;
```

```

--
=====

-- MÓDULO 5: ALGORITMO DE ADMISIÓN (CU17)

--
=====

-- CU17: Calcular puntaje total de cada postulante (promedio de promedios por materia)

SELECT p.id_postulante,
       p.nombre || ' ' || p.apellido   AS nombre_completo,
       ROUND(AVG(e.promedio), 2)      AS puntaje_total,
       CASE WHEN AVG(e.promedio) >= 60
            THEN 'APROBADO' ELSE 'REPROBADO' END AS resultado
FROM   evaluacion e
JOIN   postulante p ON p.id_postulante = e.id_postulante
GROUP BY p.id_postulante, p.nombre, p.apellido
ORDER BY puntaje_total DESC;

-- CU17: Actualizar puntaje_total y estado en POSTULACION

UPDATE postulacion po
SET   puntaje_total = sub.puntaje_total,
      estado       = CASE WHEN sub.puntaje_total >= 60 THEN 'APROBADO' ELSE 'REPROBADO'
END
FROM (

```

```

SELECT id_postulante, ROUND(AVG(promedio), 2) AS puntaje_total
FROM   evaluacion
GROUP BY id_postulante
) sub
WHERE  po.id_postulante = sub.id_postulante;

```

-- CU17: Asignar carrera según opción 1 (si hay cupos disponibles)

```

UPDATE postulacion
SET   id_carrera_asignada = id_carrera_opcion1,
      estado              = 'ADMITIDO'
WHERE estado = 'APROBADO'
AND   id_carrera_opcion1 IN (
      SELECT id_carrera FROM carrera WHERE cupos_disponibles > 0
);

```

-- CU17: Si no hay cupos en opción 1, intentar con opción 2

```

UPDATE postulacion
SET   id_carrera_asignada = id_carrera_opcion2,
      estado              = 'ADMITIDO'
WHERE estado = 'APROBADO'
AND   id_carrera_asignada IS NULL
AND   id_carrera_opcion2 IN (
      SELECT id_carrera FROM carrera WHERE cupos_disponibles > 0
);

```

```

);

-- CU17: Decrementar cupos al confirmar admisión

UPDATE carrera

SET   cupos_disponibles = cupos_disponibles - 1

WHERE id_carrera IN (

    SELECT id_carrera_asignada

    FROM   postulacion

    WHERE  estado = 'ADMITIDO'

    AND   id_carrera_asignada IS NOT NULL

);

-- CU17: Listado final de admitidos con carrera asignada

SELECT p.nombre || ' ' || p.apellido AS postulante,

       po.puntaje_total,

       c1.sigla                      AS opcion_1,

       c2.sigla                      AS opcion_2,

       ca.nombre_carrera             AS carrera_asignada,

       po.estado

FROM   postulacion po

JOIN   postulante p  ON p.id_postulante = po.id_postulante

JOIN   carrera  c1  ON c1.id_carrera   = po.id_carrera_opcion1

LEFT   JOIN carrera c2 ON c2.id_carrera = po.id_carrera_opcion2

```

```
LEFT JOIN carrera ca ON ca.id_carrera = po.id_carrera_asignada
```

```
ORDER BY po.puntaje_total DESC;
```

```
--
```

```
=====
```

```
-- MÓDULO 6: GRUPOS (CU19, CU20)
```

```
--
```

```
=====
```

```
-- CU19: Calcular número de grupos necesarios — CEIL(total_aprobados / 80)
```

```
SELECT CEIL(COUNT(*)::NUMERIC / 80) AS grupos_necesarios
```

```
FROM postulacion
```

```
WHERE estado IN ('APROBADO', 'ADMITIDO');
```

```
-- CU19: Registrar nuevos grupos calculados
```

```
INSERT INTO grupo (nombre_grupo, capacidad_maxima, aula, turno)
```

```
VALUES ('A-MAÑ-2026', 80, 'Aula 101 - FICCT', 'MAÑANA'),
```

```
      ('B-TAR-2026', 80, 'Aula 205 - FICCT', 'TARDE');
```

```
-- CU20: Asignar postulantes a grupos (distribución ordenada por puntaje DESC)
```

```
-- Asigna grupo según número de fila / capacidad
```

```
INSERT INTO grupo_postulante (id_grupo, id_postulante)
```

```

SELECT g.id_grupo,
       sub.id_postulante
FROM (
    SELECT po.id_postulante,
           ROW_NUMBER() OVER (ORDER BY po.puntaje_total DESC) AS fila
    FROM   postulacion po
    WHERE  po.estado IN ('APROBADO','ADMITIDO')
) sub
JOIN grupo g ON g.id_grupo = CEIL(sub.fila::NUMERIC / 80)
ON CONFLICT (id_grupo, id_postulante) DO NOTHING;

```

-- CU20: Ver distribución de postulantes por grupo

```

SELECT g.nombre_grupo,
       g.turno,
       COUNT(gp.id_postulante) AS total_asignados,
       g.capacidad_maxima,
       g.capacidad_maxima - COUNT(gp.id_postulante) AS cupos_restantes
FROM   grupo g
LEFT   JOIN grupo_postulante gp ON gp.id_grupo = g.id_grupo
WHERE  g.activo = TRUE
GROUP BY g.id_grupo, g.nombre_grupo, g.turno, g.capacidad_maxima
ORDER BY g.nombre_grupo;

```

-- CU20: Ver listado de postulantes de un grupo específico

```
SELECT g.nombre_grupo,
       p.ci,
       p.nombre || ' ' || p.apellido AS nombre_completo,
       po.puntaje_total,
       gp.fecha_asignacion
FROM   grupo_postulante gp
JOIN   grupo      g ON g.id_grupo      = gp.id_grupo
JOIN   postulante p ON p.id_postulante = gp.id_postulante
JOIN   postulacion po ON po.id_postulante = gp.id_postulante
WHERE  gp.id_grupo = 1
ORDER BY po.puntaje_total DESC;
```

--

=====

-- MÓDULO 7: GESTIÓN DE DOCENTES (CU21, CU22)

--

=====

-- CU21: Registrar nuevo docente (transacción con usuario)

BEGIN;

INSERT INTO usuario (ci, nombre, apellido, email, password_hash, id_rol)


```
VALUES ('4999999', 'Elena', 'Soria', 'esoria@ficct.edu.bo', '$2a$12$DocHashNew', 2)
```

```
RETURNING id_usuario;
```

```
INSERT INTO docente (id_usuario, especialidad, titulo_academico, telefono)
```

```
VALUES (currval('usuario_id_usuario_seq'), 'Estadística Aplicada', 'MAESTRIA', '70399888');
```

```
COMMIT;
```

```
-- CU21: Validar que el docente tenga título requerido (MAESTRIA o superior)
```

```
SELECT id_docente, titulo_academico
```

```
FROM docente
```

```
WHERE id_docente = 1
```

```
AND titulo_academico IN ('MAESTRIA','DOCTORADO');
```

```
-- CU21: Listar todos los docentes activos con sus datos
```

```
SELECT d.id_docente,
```

```
u.ci,
```

```
u.nombre || ' ' || u.apellido AS nombre_completo,
```

```
d.especialidad,
```

```
d.titulo_academico,
```

```
d.telefono,
```

```
u.email,
```

```
d.activo
```

```
FROM docente d
```

```
JOIN usuario u ON u.id_usuario = d.id_usuario
```

```
WHERE d.activo = TRUE
```

```
ORDER BY u.apellido;
```

```
-- CU22: Asignar docente a grupo-materia
```

```
INSERT INTO asignacion_docente (id_docente, id_grupo, id_materia)
```

```
VALUES (1, 1, 1)
```

```
ON CONFLICT (id_grupo, id_materia) DO UPDATE
```

```
SET id_docente = EXCLUDED.id_docente,
```

```
    fecha_asignacion = NOW(),
```

```
    activo = TRUE;
```

```
-- CU22: Ver carga horaria de un docente
```

```
SELECT d.id_docente,
```

```
       u.nombre || ' ' || u.apellido AS docente,
```

```
       g.nombre_grupo,
```

```
       g.turno,
```

```
       m.nombre_materia,
```

```
       m.horas_semana,
```

```
       gm.horario,
```

```
       ad.fecha_asignacion
```

```
FROM asignacion_docente ad
```

```
JOIN docente d ON d.id_docente = ad.id_docente
```

```

JOIN usuario u ON u.id_usuario = d.id_usuario
JOIN grupo g ON g.id_grupo = ad.id_grupo
JOIN materia m ON m.id_materia = ad.id_materia
LEFT JOIN grupo_materia gm ON gm.id_grupo = ad.id_grupo AND gm.id_materia =
ad.id_materia
WHERE ad.id_docente = 1 AND ad.activo = TRUE
ORDER BY g.nombre_grupo, m.nombre_materia;

```

-- CU22: Docentes sin asignación (para detectar sin carga horaria)

```

SELECT d.id_docente,
       u.nombre || ' ' || u.apellido AS docente,
       d.especialidad
FROM docente d
JOIN usuario u ON u.id_usuario = d.id_usuario
WHERE d.activo = TRUE
AND d.id_docente NOT IN (
    SELECT DISTINCT id_docente FROM asignacion_docente WHERE activo = TRUE
);

```

-- CU22: Grupos sin docente asignado en alguna materia

```

SELECT g.nombre_grupo,
       m.nombre_materia
FROM grupo g

```

```

CROSS JOIN materia m

WHERE g.activo = TRUE AND m.activo = TRUE

AND NOT EXISTS (

    SELECT 1 FROM asignacion_docente ad

    WHERE ad.id_grupo = g.id_grupo

        AND ad.id_materia = m.id_materia

        AND ad.activo = TRUE

)

ORDER BY g.nombre_grupo, m.nombre_materia;

```

```
--
```

```
=====
```

```
-- MÓDULO 8: DASHBOARD KPIs (CU29)
```

```
--
```

```
=====
```

```
-- CU29: KPI 1 — Total de postulantes inscritos
```

```
SELECT COUNT(*) AS total_inscritos FROM postulante;
```

```
-- CU29: KPI 2 — Total de aprobados
```

```
SELECT COUNT(*) AS total_aprobados
```

```
FROM postulacion
```

```
WHERE estado IN ('APROBADO', 'ADMITIDO');
```

```
-- CU29: KPI 3 — Total de reprobados
```

```
SELECT COUNT(*) AS total_reprobados
```

```
FROM postulacion
```

```
WHERE estado = 'REPROBADO';
```

```
-- CU29: KPI 4 — Total de grupos habilitados
```

```
SELECT COUNT(*) AS total_grupos FROM grupo WHERE activo = TRUE;
```

```
-- CU29: KPI 5 — Total de admitidos con carrera asignada
```

```
SELECT COUNT(*) AS total_admitidos
```

```
FROM postulacion
```

```
WHERE estado = 'ADMITIDO';
```

```
-- CU29: KPI 6 — Total recaudado en pagos confirmados
```

```
SELECT SUM(monto) AS total_recaudado_bs
```

```
FROM pago
```

```
WHERE estado_pago = 'CONFIRMADO';
```

```
-- CU29: Todos los KPIs en una sola consulta (para el panel principal)
```

```
SELECT
```

```
(SELECT COUNT(*) FROM postulante)
```

```
AS total_inscritos,
```

```

        (SELECT COUNT(*) FROM postulacion WHERE estado IN ('APROBADO','ADMITIDO')) AS
total_aprobados,

        (SELECT COUNT(*) FROM postulacion WHERE estado = 'REPROBADO')                AS
total_reprobados,

        (SELECT COUNT(*) FROM grupo WHERE activo = TRUE)                            AS total_grupos,

        (SELECT COUNT(*) FROM postulacion WHERE estado = 'ADMITIDO')                AS
total_admitidos,

        (SELECT COALESCE(SUM(monto),0) FROM pago WHERE estado_pago = 'CONFIRMADO') AS
total_recaudado_bs;

```

-- CU29: Gráfico — Rendimiento promedio por materia

```

SELECT m.nombre_materia,

        ROUND(AVG(e.promedio), 2)                                AS promedio_general,

        COUNT(*) FILTER (WHERE e.estado = 'APROBADO')           AS aprobados,

        COUNT(*) FILTER (WHERE e.estado = 'REPROBADO')           AS reprobados,

        ROUND(COUNT(*) FILTER (WHERE e.estado = 'APROBADO')::NUMERIC

        / NULLIF(COUNT(*),0) * 100, 1)                          AS porcentaje_aprobacion

FROM   evaluacion e

JOIN   materia m ON m.id_materia = e.id_materia

GROUP BY m.id_materia, m.nombre_materia

ORDER BY m.nombre_materia;

```

-- CU29: Gráfico — Distribución de postulantes por carrera de opción 1

```

SELECT c.sigla,

```

```

        c.nombre_carrera,

        COUNT(po.id_postulacion) AS total_postulaciones,

        c.cupos_totales,

        c.cupos_disponibles

FROM   carrera c

LEFT   JOIN postulacion po ON po.id_carrera_opcion1 = c.id_carrera

GROUP BY c.id_carrera, c.sigla, c.nombre_carrera, c.cupos_totales, c.cupos_disponibles

ORDER BY total_postulaciones DESC;

```

-- CU29: Gráfico — Postulantes por colegio de origen (top 10)

```

SELECT colegio_origen,

        COUNT(*) AS total

FROM   postulante

WHERE  colegio_origen IS NOT NULL

GROUP BY colegio_origen

ORDER BY total DESC

LIMIT 10;

```

-- CU29: Gráfico — Evolución de inscripciones por día

```

SELECT DATE(created_at)      AS fecha,

        COUNT(*)            AS inscripciones_del_dia,

        SUM(COUNT(*)) OVER (ORDER BY DATE(created_at)) AS acumulado

FROM   postulante

```

```
GROUP BY DATE(created_at)
```

```
ORDER BY fecha;
```

```
-- CU29: Ranking de postulantes por puntaje (top 10)
```

```
SELECT ROW_NUMBER() OVER (ORDER BY po.puntaje_total DESC) AS puesto,
```

```
    p.nombre || ' ' || p.apellido AS nombre_completo,
```

```
    po.puntaje_total,
```

```
    ca.sigla AS carrera_asignada,
```

```
    po.estado
```

```
FROM   postulacion po
```

```
JOIN   postulante p   ON p.id_postulante = po.id_postulante
```

```
LEFT   JOIN carrera ca ON ca.id_carrera = po.id_carrera_asignada
```

```
ORDER BY po.puntaje_total DESC
```

```
LIMIT 10;
```

```
--
```

```
=====
```

```
-- MÓDULO 9: AUDITORÍA Y BITÁCORA
```

```
--
```

```
=====
```

```
-- Consultar toda la bitácora ordenada por fecha
```



```

SELECT ba.id_bitacora,

       u.nombre || ' ' || u.apellido AS usuario,

       r.nombre_rol,

       ba.accion,

       ba.tabla_afectada,

       ba.descripcion,

       ba.ip_origen,

       ba.fecha_hora

FROM   bitacora_auditoria ba

LEFT   JOIN usuario u ON u.id_usuario = ba.id_usuario

LEFT   JOIN rol     r ON r.id_rol     = u.id_rol

ORDER  BY ba.fecha_hora DESC

LIMIT 50;

```

-- Filtrar bitácora por tipo de acción

```

SELECT * FROM bitacora_auditoria

WHERE accion = 'LOGIN'

ORDER BY fecha_hora DESC;

```

-- Filtrar bitácora por rango de fechas

```

SELECT ba.id_bitacora,

       u.nombre || ' ' || u.apellido AS usuario,

       ba.accion,

```

```

        ba.descripcion,
        ba.ip_origen,
        ba.fecha_hora
FROM   bitacora_auditoria ba
LEFT   JOIN usuario u ON u.id_usuario = ba.id_usuario
WHERE  ba.fecha_hora BETWEEN '2026-03-01' AND '2026-04-30'
ORDER BY ba.fecha_hora DESC;

```

-- Acciones por usuario (resumen)

```

SELECT u.nombre || ' ' || u.apellido AS usuario,
        ba.accion,
        COUNT(*) AS total
FROM   bitacora_auditoria ba
JOIN   usuario u ON u.id_usuario = ba.id_usuario
GROUP BY u.id_usuario, u.nombre, u.apellido, ba.accion
ORDER BY usuario, ba.accion;

```

-- IPs con más actividad (detección de accesos sospechosos)

```

SELECT ip_origen,
        COUNT(*) AS total_accesos,
        COUNT(DISTINCT id_usuario) AS usuarios_distintos
FROM   bitacora_auditoria
WHERE  accion = 'LOGIN'

```

GROUP BY ip_origen

ORDER BY total_accesos DESC;

--

=====

-- MÓDULO 10: REPORTES Y CONSULTAS GENERALES DEL SISTEMA

--

=====

-- Reporte general: estado completo de cada postulante

SELECT p.ci,

 p.nombre || ' ' || p.apellido AS nombre_completo,

 p.colegio_origen,

 pg.estado_pago,

 po.puntaje_total,

 po.estado AS estado_postulacion,

 ca.sigla AS carrera_asignada,

 g.nombre_grupo AS grupo_asignado

FROM postulante p

LEFT JOIN pago pg ON pg.id_postulante = p.id_postulante

 AND pg.estado_pago = 'CONFIRMADO'

LEFT JOIN postulacion po ON po.id_postulante = p.id_postulante

```

LEFT JOIN carrera ca ON ca.id_carrera = po.id_carrera_asignada
LEFT JOIN grupo_postulante gp ON gp.id_postulante = p.id_postulante
LEFT JOIN grupo g ON g.id_grupo = gp.id_grupo
ORDER BY p.apellido, p.nombre;

```

-- Reporte de notas completo por postulante y materia

```

SELECT p.nombre || ' ' || p.apellido AS postulante,
       m.nombre_materia,
       e.nota1, e.nota2, e.nota3,
       e.promedio,
       e.estado
FROM   evaluacion e
JOIN   postulante p ON p.id_postulante = e.id_postulante
JOIN   materia m ON m.id_materia = e.id_materia
ORDER BY p.apellido, p.nombre, m.nombre_materia;

```

-- Parámetros del sistema

```

SELECT clave, valor, descripcion, updated_at
FROM   parametro_sistema
ORDER BY clave;

```

-- Actualizar un parámetro del sistema

```

UPDATE parametro_sistema

```

```
SET valor = '65',
```

```
updated_at = NOW()
```

```
WHERE clave = 'nota_minima_aprobacion';
```

```
-- Consulta de carreras con cupos
```

```
SELECT sigla, nombre_carrera, cupos_totales, cupos_disponibles,
```

```
cupos_totales - cupos_disponibles AS cupos_ocupados,
```

```
ROUND((cupos_totales - cupos_disponibles)::NUMERIC / cupos_totales * 100, 1) AS  
porcentaje_ocupacion
```

```
FROM carrera
```

```
WHERE activo = TRUE
```

```
ORDER BY sigla;
```

```
-- Materias activas del CUP
```

```
SELECT id_materia, nombre_materia, sigla, horas_semana
```

```
FROM materia
```

```
WHERE activo = TRUE
```

```
ORDER BY nombre_materia;
```

```
-- Estado del período de admisión activo
```

```
SELECT id_periodo, nombre, fecha_inicio, fecha_fin, estado,
```

```
fecha_fin - CURRENT_DATE AS dias_restantes
```

```
FROM periodo_admision
```

```
WHERE estado = 'ACTIVO';
```

```
--
```

```
=====
```

```
-- FIN DE CONSULTAS
```

```
--
```

```
=====
```

6.3 Diseñar un caso de uso (diagrama de secuencia)

6.3.1 Ciclo #1

7 FLUJO DE TRABAJO: IMPLEMENTACION